

Syslab Julia 语法入门

Syslab 语言组

苏州同元软控信息技术有限公司

2024-07-29



TONGYUAN
Software and Control

课程须知

本课程课程目标：

本课程面向科学计算新用户，讲述 Julia 基本使用、基础语法、常用数据类型、编程风格以及与其他语言的差异性等，帮助用户深入了解 Julia 语言、掌握 Julia 语言编程能力。

本课程基于云化版本：Syslab 2024a 构建

@苏州同元软控信息技术有限公司版权所有，侵权必究

本课程所有示例见附件，课程内代码为伪代码片段

目录

1. Julia 导论

2. Julia 语法全貌

3. 函数与数据类型

4. 模块化

5. 核心风格

6. 文件 IO

7. 与其他语言的差异

Syslab Julia 导论

Syslab 语言组

苏州同元软控信息技术有限公司

2024-07-29



TONGYUAN
Software and Control

@苏州同元软控信息技术有限公司版权所有，侵权必究
未经许可，不得复印、传播或以其他方式使用

目录

1. Julia 语言速览

2. Julia REPL

01 Julia 语言速览

©苏州同元软控信息技术有限公司版权所有，未经许可，不得复印、传播或以其他方式使用

@苏州同元软控信息技术有限公司
版权所有，侵权必究

1. Julia 语言速览

- Julia 是专为科学计算场景设计的编程语言，并具有以下几个特点

高性能

易用性

工具箱支持

可扩展性

@苏州同元软控信息技术有限公司
版权所有，侵权必究

1. Julia 语言速览——高性能

优化后的代码接近 C 语言的性能

示例：累积求和 cumsum 函数

```
function cumsum(x)
    ret = similar(x)
    _sum = 0.0
    @inbounds for i in eachindex(x)
        _sum += x[i]
        ret[i] = _sum
    end
    return ret
end
x = rand(100);
using BenchmarkTools
@btime ret = cumsum(x);
```



```
调试控制台 终端
julia> 正在运行 cumsum.jl
203.845 ns (1 allocation: 896 bytes)

julia>
```


1. Julia 语言速览——易用性

简洁的语法

动态类型

交互式命令行

多重派发

- 语法简洁，近似数学表达式；

- 支持动态类型，函数定义灵活；

- 内置方便的交互式命令行，支持包管理，查看帮助等；

- 多重派发支持函数根据参数的运行时类型，动态选择最适合的函数实现

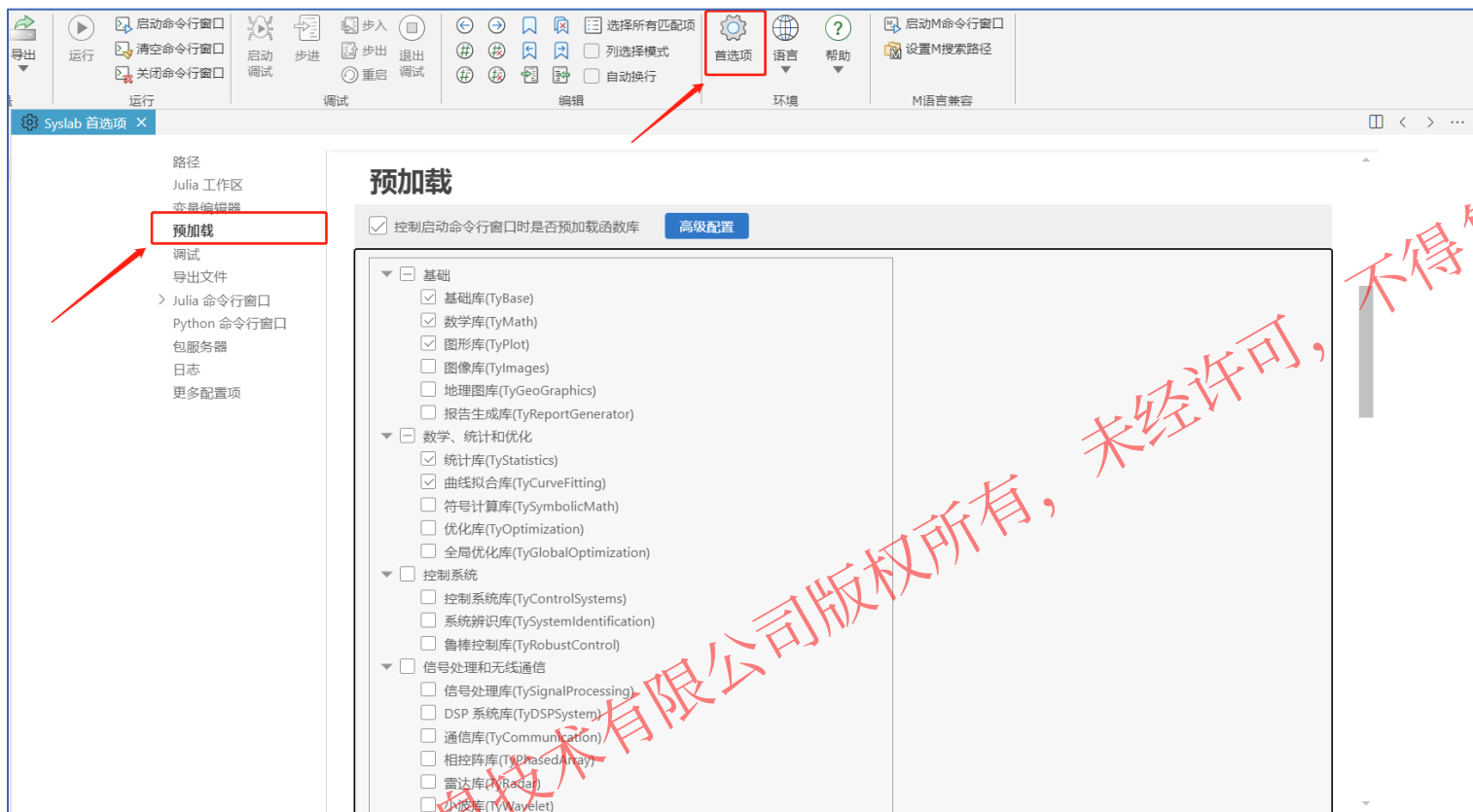
1. Julia 语言速览——丰富的工具箱生态

- 除了基础工具箱、绘图工具箱外，Syslab 还提供了图形，信号处理，数学与优化，控制系统等多个工具箱。
- 通过设置 **首选项——预加载**，或者在脚本中使用 **using** 语句即可使用。

工 具 箱	信号处理与通信		数学与优化		AI	控制系统	
	通信	相控阵	PDE	全局优化	强化学习	系统辨识	PID调节器
	DSP	雷达	曲线拟合	优化	深度学习	鲁棒控制	线性系统分析器
	信号处理	小波	符号数学	统计	机器学习	控制系统	

1. Julia 语言速览——丰富的工具箱生态

- 可以在 **首选项——预加载** 中，勾选工具箱，启动命令行时自动加载



- 也可以在命令行或者脚本中使用 **using** 语法导入工具箱

```
julia> using TyMathCore  
julia> fft(rand(10));
```

1. Julia 语言速览——可扩展性

调用 Python/C/C++

静态代码生成

宏和元编程

...

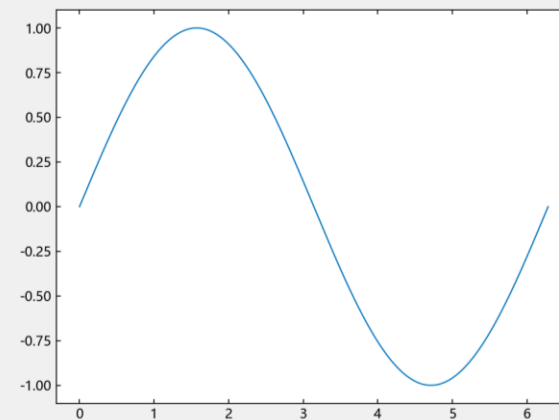
1. Julia 语言速览——可扩展性

- 调用 C 函数

```
julia> cpath = @ccall getenv("PATH"::Cstring)::Cstring  
Cstring(0x00007ffc706d1a81)  
  
julia> unsafe_string(cpath)  
"/usr/local/share/TongYuan/..."
```

- 调用 Python 函数

```
using PyCall  
using TyPlot # 同元绘图库  
  
# 导入python库  
@pyimport numpy as np  
x = np.linspace(0, 2pi, 1000)  
y = np.sin(x)  
  
# 绘图  
plot(x, y)
```



02 Julia REPL

©苏州同元软控信息技术有限公司版权所有，未经许可，不得复印、传播或以其他方式使用

@苏州同元软控信息技术有限公司
版权所有，侵权必究

2. Julia REPL

Julia REPL 是一个交互式命令行工具，通过 REPL 可以实时输入并执行 Julia 代码。可以通过在终端中输入 `julia` 命令进入 Julia REPL，通过快捷键 `Ctrl + D` 或者调用 `exit()` 即可退出 REPL。

[illegible]

2. Julia REPL

在 REPL 中执行 Julia 程序

进入 REPL 后即可在窗口中输入任何需要执行的 Julia 程序，按下回车键即可在 REPL 的全局作用域执行输入的程序。

```
julia> a = "Hello, World" # 定义变量  
"Hello, World"
```

```
julia> a # 展示变量  
Hello, World
```

```
julia> function f(x, y) # 定义函数  
    return x + y  
end
```

```
julia> for i = 1:10 # 执行循环语句  
    a = i * 2  
    print(a)  
end
```

注意:

- 输入变量名则会显示该变量的值
- Julia中不会自动创建变量，在使用变量之前需要确保其已经被定义

2. Julia REPL

在 REPL 中执行 Julia 程序

循环中的赋值在REPL和在文件中的语义有所不同：

1. 在 REPL 的循环语句中对变量赋值时，Julia 将尝试寻找**全局作用域**中的变量并**对其赋值**；
2. 在文件中的循环语句对变量赋值时，Julia 将在循环语句的局部作用域**创建局部变量**，不会对全局作用域中的变量产生影响。

```
julia> x = 1

julia> for i in 1:10
    x = i
end

julia> x
10
```

在REPL中的循环语句将对全局作用域中的变量进行赋值

```
x = 1

for i in 1:10
    # 触发错误，x未定义
    x = i
end

println(x)
```

在文件中运行循环语句，无法对全局变量进行赋值

```
x = 1

for i in 1:10
    # 显式声明全局变量
    global x
    x = i
end

println(x)
```

通过global在循环中显式声明全局变量

2. Julia REPL

在 REPL 中查找帮助文档

用户可以通过输入问号 ? 进入 REPL 的帮助模式。用户在此模式下输入内容时，Julia 将基于输入内容提供相应的帮助文档。

```
julia> ? # 输入 ? 后, REPL 开头的提示符会变化为 help?>
```

```
help?> +
```

```
search: +
```

```
+(x, y...)
```

Addition operator. $x+y+z+\dots$ calls this function with all arguments, i.e. $+(x, y, z, \dots)$.

Examples

```
=====
```

```
julia> 1 + 20 + 4  
25
```

```
julia> +(1, 20, 4)  
25
```

在帮助模式下输入 + 后，Julia 在 REPL 中展示了加法函数的帮助文档及其使用示例。

2. Julia REPL

在 REPL 中安装包

用户可以通过在默认模式中按下] 键进入包管理模式，通过按下 CTRL+C 或在行首按退格键退出该模式。

在该模式中，用户可以通过执行包管理命令加载或安装 Julia 包。

```
(@v1.9) pkg> add JSON
```

```
(@v1.9) pkg> status
```

```
(@v1.9) pkg> rm JSON
```

通过 status 命令列举当前系统中已经安装的 Julia 包

通过 rm 命令删除已经安装的 Julia 包。

提示：输入 ? 即可进入包管理模式专有的帮助模式，从而查看所有包管理模式下的可用命令。

2. Julia REPL

高效的使用 Julia REPL

基于历史回滚输入：

Julia REPL可以基于已经输入的内容回滚历史输入（快捷键：上方向键），用户可以通过该特性高效的查找历史输入。

```
julia>function compute_area(length,  
width)  
    area = length + width  
    println("The area is $area")  
end
```

以前定义的函数存在错误

```
julia>function compute_area(length,  
width)  
    area = length * width  
    println("The area is $area")  
end
```

通过**上方向键**重现该函数，并将光标移至出错的位置对其进行修改

这些历史命令在关闭 REPL 后依然会保留，因此，即使用户退出了 REPL，历史输入也可以在下次启动时被查找到。

2. Julia REPL

高效的使用 Julia REPL

使用 TAB 进行自动补全:

在 REPL 中可以输入一个函数或类型的前几个字符，然后按下 Tab 键触发自动补全功能，当 REPL 检测到多种补全项后，将展示所有匹配项的列表:

```
julia> test = 1 # 定义名为 test 的变量
```

```
1
```

```
julia> te[按下TAB] # 展示当前作用域中所有由 te 开头的变量和类型  
tempdir tempname test text textwidth
```

```
julia> tes[按下TAB] # 当前作用域中只有变量 test 由 tes 开头，REPL将自动补全输入为 test  
julia> test
```

Syslab Julia 语法全貌

Syslab 语言组

苏州同元软控信息技术有限公司

2024-07-31



TONGYUAN
Software and Control

@苏州同元软控信息技术有限公司版权所有，侵权必究
未经许可，不得复印、传播或以其他方式使用

目录

1. Julia 语法全貌概览

2. Julia 基础语法 – 定义

3. Julia 基础语法 – 语句

01

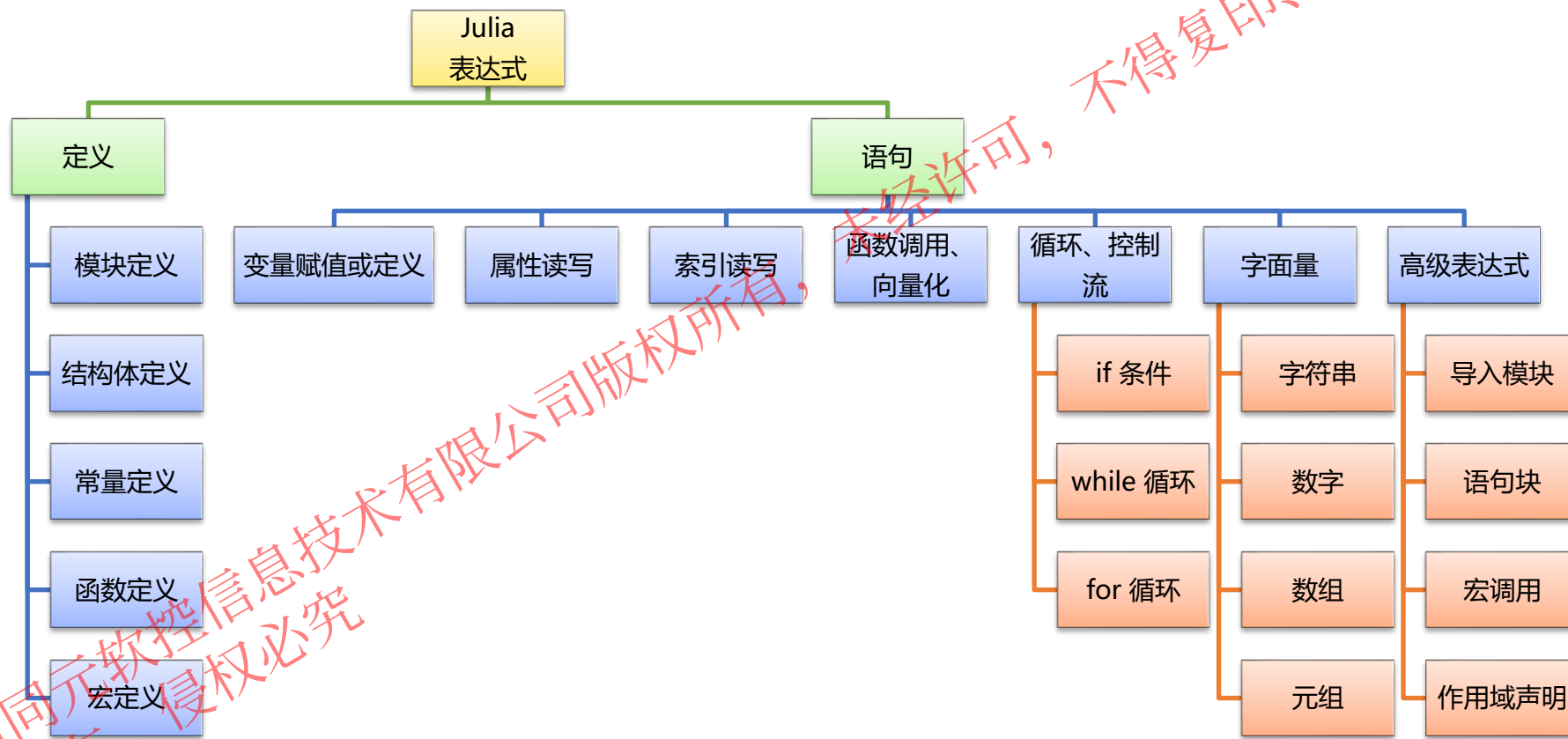
Julia 语法全貌概览

未经许可，不得复印、传播或以其他方式使用

@苏州同元软控信息技术有限公司版权所有，侵权必究

1. Julia 语法全貌概览

- Julia 的语法接近 Python、MATLAB 或 Ruby，但 Julia 语法与其他语言不完全相同。
- 表达式优先：所有 Julia 语法均为表达式，用以增强程序的可组合性。



02

Julia 基础语法 – 定义

未经许可，不得复印、传播或以其他方式使用

@苏州同元软控信息技术有限公司版权所有，侵权必究

2. Julia 基础语法 – 定义

模块定义：Julia 使用模块来组织代码

my_module.jl

```
module ModuleA
export add1
add1(x) = x + 1
end

module ModuleB
import LinearAlgebra: norm
export my_hypot
my_hypot(x, y) = norm([x, y])
end
```

test.jl

```
include("my_module.jl")
# 使用全名
@show ModuleA.add1(10)
# 导入模块符号
using .ModuleB
@show my_hypot(3, 4)

import .ModuleA
@show ModuleA.add1(10)
```

```
julia> 正在运行 test.jl
ModuleA.add1(10) = 11
my_hypot(3, 4) = 5.0
ModuleA.add1(10) = 11
11
```

1. 调用 include 函数，在当前模块顶层加载 Julia 文件
2. 使用 module 定义模块，使用 export 标记模块导出符号
3. 模块不能直接访问外部符号，如变量名、函数名与类型名，需使用 using 或 import 导入符号
4. 使用 using 导入 Julia 包，污染名字空间；使用 import 导入 Julia 包，不污染名字空间；
特别地，using .XXX 导入当前模块下的子模块 XXX

2. Julia 基础语法 – 定义

结构体定义：Julia 定义结构体并创建新类型

my_struct.jl

```
struct Data1
    x::Float64
    y::String
end

Base.@kwdef struct Data2{T}
    x::T
    y::String = "default"
end

mutable struct Data3{T}
    x::T
    y::String
end
```

1. struct 关键字创建不可变结构体, mutable struct 创建可变结构体
2. Base.@kwdef 修饰结构体后, 构造器可支持关键字与默认参数
3. 可定义泛型结构体

Julia REPL

```
julia> include("my_struct.jl")

julia> d1 = Data1(1.2, "str")
Data1{Float64, String}(1.2, "str")

julia> d2 = Data2{Float64}(x = 1.2)
Data2{Float64, String}(1.2, "default")

julia> d3 = Data3{Float64}(1.2, "str")
Data3{Float64, String}(1.2, "str")

julia> d3.x = 2.4;
julia> d3
Data3{Float64, String}(2.4, "str")

julia> d2.x = 2.4
```

ERROR: setfield!: immutable struct of type Data2 cannot be changed

2. Julia 基础语法 – 定义

常量定义: `const` 关键字为当前模块定义常量

my_const.jl

```
const string_const = "hello"  
const number_const = 1  
const data_const = [1.2, 5.7]  
const data_const2 = [  
    data_const,  
    data_const  
]  
push!(data_const, 1.2)  
@show data_const2  
  
data_const = 2
```

```
julia> 正在运行 my_const.jl  
data_const2 = [[1.2, 5.7, 1.2], [1.2, 5.7, 1.2]]  
ERROR: invalid redefinition of constant data_const  
Stacktrace:  
 [1] top-level scope  
      @ ~/SyslabUserData/SyslabDrive/examples/02-Julia Grammar/01_definatio  
n/03_const/my_const.jl:11
```

1. `const` 关键字在当前模块定义常量
2. 常量对 Julia 性能提升有重要作用
3. `const` 符号不可指向其他数据; 数据内容仍可修改, 但不可重新绑定 `const` 符号

2. Julia 基础语法 – 定义

函数定义：使用数学等式或 `function` 关键字定义函数

my_functions.jl

```
# 使用数学等式定义函数
myfunc1(x) = x^2 + 3 * x + 1
r1 = myfunc1(2)
println("myfunc1(2) = $r1")

# 使用 function 关键字定义函数
function myfunc2(x)
    x^2 + 3 * x + 1
end
r2 = myfunc2(2)
println("myfunc1(2) = $r2")

function myfunc3(x)
    return x^2 + 3 * x + 1
end
r3 = myfunc3(2)
println("myfunc1(2) = $r3")
```

```
julia> 正在运行 my_functions.jl
myfunc1(2) = 11
myfunc1(2) = 11
myfunc1(2) = 11
```

1. 在 Julia 中定义函数可以用简洁的“赋值形式”来定义，在赋值形式下，函数体必须是一个一行的表达式
2. 可以使用 `function` 关键字定义函数，默认情况下函数返回函数体的最后一个表达式，也可以使用 `return` 关键字让函数立即返回值

2. Julia 基础语法 – 定义

宏定义：使用 `macro` 关键字定义宏

my_macro.jl

```
macro sayhello(name)
    return :(println("Hello, ", $name))
end
```

1. 在 Julia 中宏将参数映射到返回的表达式，并且直接编译结果表达式，而不需要运行时 `eval` 调用。宏参数可以是任何有效的 Julia 表达式。
2. 宏在编译时展开，因此，宏允许程序员在运行完整的程序之前生成并包含自定义代码的片段
3. 宏的调用方式：`@macroname(expr1, expr2, ...)`
或 `@macroname expr1 expr2`

Julia REPL

```
julia> include("my_macro.jl")
@sayhello (macro with 1 method)

julia> @sayhello "Julia"
Hello, Julia

julia> @sayhello("Julia")
Hello, Julia

julia> @sayhello :Julia
Hello, Julia

julia> @sayhello "Ju" * "lia"
Hello, Julia
```

03

Julia 基础语法 – 语句

未经许可，不得复印、传播或以其他方式使用

@苏州同元软控信息技术有限公司版权所有，侵权必究

3. Julia 基础语法 – 语句

变量赋值或定义

```
julia> x = 123
123

julia> x = [1, 2, 3, 4];
julia> y = x;
julia> y[1] = 0;
julia> x
4-element Vector{Int64}:
 0
 2
 3
 4

julia> y = copy(x);
julia> y[2] = 0;
julia> x
4-element Vector{Int64}:
 0
 2
 3
 4
```

1. 在 Julia 中给变量赋值，即将变量名与一个内存中的内容联系起来，也称为绑定（binding），使用等号 “=”，等号左边写变量名，右边写要保存到变量中的值
2. 变量的类型是由它保存的（指向的内存中的）值的类型决定的，Julia 允许说明变量类型，但一般不需要，仅在编写函数时为了提高运行效率可以说明自变量和返回值类型。
3. 如果 x 是一个变量，y = x 并不是复制 x 的值给变量 y，对于可变类型的 x，对 y 做修改时 x 也会被修改，如果需要复制 x 的值需要用 y = copy(x) 或者 y = deepcopy(x)

3. Julia 基础语法 – 语句

属性读写：结构体的属性读写

1. struct 关键字创建不可变结构体, 可以通过点操作读属性
2. mutable struct 创建可变结构体, 可以点操作读属性和写属性

```
julia> struct Data1
    x::Float64
    y::String
end

julia> d1 = Data1(1.2, "str")
Data1(1.2, "str")

julia> d1.x = 2.4
ERROR: setfield!: immutable struct of type Data1
cannot be changed
```

```
julia> mutable struct Data2
    x::Float64
    y::String
end

julia> d2 = Data2(1.2, "str")
Data2(1.2, "str")

julia> d2.x = 2.4;

julia> d2
Data2(2.4, "str")
```

3. Julia 基础语法 – 语句

索引读写：数组的索引读写

```
julia> A = [1 2 3; 4 5 6; 7 8 9];
julia> A[1] # 数组的第一个元素
1

julia> A[end] # 数组的最后一个元素
9

julia> A[:, 2] # 数组的第 2 列
3-element Vector{Int64}:
 2
 5
 8

julia> A[2, :] # 数组的第 2 行
3-element Vector{Int64}:
 4
 5
 6

julia> A[:]
9-element Vector{Int64}:
 1
 4
 ⋮
 6
```

1. Julia 中第一个索引为 1。
2. end 除了是终止 for、while、try、if 等语句的关键字以外，也是数组的最后一个索引。例如，X[end] 是 X 的最后一个元素，X[3:end] 选择 X 的第三个到最后一个元素。
3. A[:,n] 是矩阵 A 的第 n 列。
4. A[m,:] 是矩阵 A 的第 m 行。
5. A[:] 将 A 中的所有元素重构成一个列向量。如果 A 已经是列向量，则此表达式没有任何作用。
6. A[j:k] 使用向量 j:k 对 A 进行索引，因此相当于向量 [A[j], A[j+1], ..., A[k]]。
7. A[:,j:k] 包含第一个维度中的所有下标，但使用向量 j:k 对第二个维度进行索引。这样将返回包含列 [A[:,j], A[:,j+1], ..., A[:,k]] 的矩阵。
8. A[:,:,p] 是三维数组 A 的第 p 页。

3. Julia 基础语法 – 语句

索引读写：数组的索引读写

```
julia> A = [1 2 3; 4 5 6; 7 8 9];
julia> A[2:3] # 使用向量 j:k 对 A 进行索引
2-element Vector{Int64}:
 4
 7
julia> A[:, 2:3] # 数组的第 2 到 3 列
3×2 Matrix{Int64}:
 2 3
 5 6
 8 9
julia> A[2:3, :] # 数组的第 2 到 3 行
2×3 Matrix{Int64}:
 4 5 6
 7 8 9

# 数组的第 1 到 2 行, 第 1 到 2 列
julia> A[1:2, 1:2]
2×2 Matrix{Int64}:
 1 2
 4 5

julia> A = ones(2,2,2);
julia> A[:, :, 2] # 三维数组 A 的第 2 页
2×2 Matrix{Float64}:
 1.0 1.0
 1.0 1.0
```

1. Julia 中第一个索引为 1。
2. end 除了是终止 for、while、try、if 等语句的关键字以外，也是数组的最后一个索引。例如，X[end] 是 X 的最后一个元素，X[3:end] 选择 X 的第三个到最后一个元素。
3. A[:,n] 是矩阵 A 的第 n 列。
4. A[m,:] 是矩阵 A 的第 m 行。
5. A[:] 将 A 中的所有元素重构成一个列向量。如果 A 已经是列向量，则此表达式没有任何作用。
6. A[j:k] 使用向量 j:k 对 A 进行索引，因此相当于向量 [A[j], A[j+1], ..., A[k]]。
7. A[:,j:k] 包含第一个维度中的所有下标，但使用向量 j:k 对第二个维度进行索引。这样将返回包含列 [A[:,j], A[:,j+1], ..., A[:,k]] 的矩阵。
8. A[:, :, p] 是三维数组 A 的第 p 页。

3. Julia 基础语法 – 语句

索引读写：数组的索引读写

```
julia> A = [1 2 3; 4 5 6; 7 8 9];
julia> A[1] = 0; # 将索引为 1 的元素修改为 0
julia> A
3×3 Matrix{Int64}:
 0  2  3
 4  5  6
 7  8  9

julia> A[2:3, 2:3] = [1 2; 3 4];
julia> A # 修改数组的第 2 到 3 行，第 2 到 3 列
3×3 Matrix{Int64}:
 0  2  3
 4  1  2
 7  3  4

julia> A[2:3, 2:3] .= 0;
julia> A
3×3 Matrix{Int64}:
 0  2  3
 4  0  0
 7  0  0
```

1. 读写数组的索引都使用 []。
2. 修改索引时等号左右的数组大小应该一样，也可以使用广播修改数组的每一个索引。

3. Julia 基础语法 – 语句

索引读写：元组的索引读写

1. 访问元组的索引都可以使用 [], 命名元组也可以使用点操作来访问。
2. 元组都是不可变的, 所以不支持写索引

```
julia> T1 = (1, 2, 3)
(1, 2, 3)
```

```
julia> typeof(T1)
Tuple{Int64, Int64, Int64}
```

```
julia> T[1] # 元组的第一个元素
1
```

```
julia> T1[1] = 2
ERROR: MethodError: no method matching
setindex!...
```

```
julia> T2 = (a=1, b=2)
(a = 1, b = 2)
```

```
julia> T2.a
1
```

```
julia> T2[:a]
1
```

```
julia> T2[1]
1
```

```
julia> T2[1] = 2
ERROR: MethodError: no method matching
setindex!...
```

3. Julia 基础语法 – 语句

函数调用、向量化

- 函数调用

- 包含一个输出的函数

average.jl

```
function average(x)
    return sum(x[:])/length(x)
end

# function average(x)
#     sum(x[:])/length(x)
# end
```

Julia REPL

```
julia> include("average.jl")

julia> z = 1:99;
julia> ave = average(z)
50.0
```

- 包含多个输出的函数：当需要多个返回值时，可以通过返回元组从函数返回多个值

stat.jl

```
function stat(x)
    n = length(x)
    m = sum(x)/n
    s = sqrt(sum((x.-m).^2/n))
    return m, s
end
```

Julia REPL

```
julia> include("stat.jl")
julia> values = [12.7, 45.4, 98.9, 26.6, 53.1];
julia> ave, stdev = stat(values);
julia> ave
47.339999999999996
julia> stdev
29.412419145660223
julia> R = stat(values);
julia> _, stdev = stat(values);
```

3. Julia 基础语法 – 语句

函数调用、向量化

- 向量化

向量化 (vectorization) 是指在不显式编写循环的情况下，对数组或矩阵进行操作的过程。它通常通过使用高效的底层库来实现，从而提高代码的性能。Julia 语言支持向量化操作，允许用户编写简洁而高效的代码。

```
# 创建两个数组
a = [1, 2, 3, 4, 5]
b = [10, 20, 30, 40, 50]

# 向量化操作：逐元素相减
c = a .- 1
# 向量化操作：逐元素相加
d = a .+ b

println(c) # 输出: [0, 1, 2, 3, 4]
println(d) # 输出: [11, 22, 33, 44, 55]
```

```
julia> 正在运行 vectorization.jl
[0, 1, 2, 3, 4]
[11, 22, 33, 44, 55]
```

这里的 `.-`, `.+` 是一个向量化操作符，表示对两个数组的每个元素进行相减或相加。Julia 中的点运算符（如 `.-`, `.*`, `./` 等）用于实现向量化操作。通过这种方式，可以避免显式的循环，提高代码的可读性和执行效率。

3. Julia 基础语法 – 语句

循环、控制流 – if 条件

if 语句可以根据布尔表达式的值，让部分代码被执行或者不被执行。

test.jl

```
function test(x, y)
    if x < y
        println("x is less than y")
    elseif x > y
        println("x is greater than y")
    else
        println("x is equal to y")
    end
end
```

Julia REPL

```
julia> include("test.jl")
julia> test(1, 2)
x is less than y

julia> test(2, 1)
x is greater than y

julia> test(1, 1)
x is equal to y
```

1. 如果表达式 $x < y$ 是 true，那么对应的代码块会被执行；否则判断条件表达式 $x > y$ ，如果它是 true，则执行对应的代码块；如果没有表达式是 true，则执行 else 代码块。
2. elseif 和 else 代码块是可选的，并且可以使用任意多个 elseif 代码块。

3. Julia 基础语法 – 语句

循环、控制流 – if 条件

if 代码块是“有渗漏的”，也就是说它们不会引入局部作用域。这意味着在 if 语句中新定义的变量依然可以在 if 代码块之后使用，尽管这些变量没有在 if 语句之前定义过。所以，我们可以将 test 函数定义为：

test.jl

Julia REPL

```
function test(x,y)
    if x < y
        relation = "less than"
    elseif x == y
        relation = "equal to"
    else
        relation = "greater than"
    end
    println("x is ", relation, " y.")
end
```

```
julia> include("test.jl")
julia> test(1, 2)
x is less than y

julia> test(2, 1)
x is greater than y

julia> test(1, 1)
x is equal to y
```

变量 relation 是在 if 代码块内部声明的，但可以在外部使用。然而，在利用这种行为的时候，要保证变量在所有的分支下都进行了定义。否则会导致运行时错误。

3. Julia 基础语法 – 语句

循环、控制流 – if 条件

1. if 代码块也会返回一个值，这可能对于一些从其他语言转过来的用户来说不是很直观。这个返回值就是被执行的分支中最后一个被执行的语句的返回值。
2. 在 Julia 中的 if 语句中，一个条件表达式的值如果不是 true 或者 false 的话，会返回错误。
3. 在 Julia 中，并没有直接的 switch case 语句，但你可以使用多个 if-elseif-else 语句

```
julia> x = 3
3

julia> if x > 0
    "positive!"
else
    "negative..."
end
"positive!"
```

```
julia> if 1
    println("true")
end
ERROR: TypeError: non-boolean (Int64) used in
boolean context
```

3. Julia 基础语法 – 语句

循环、控制流 – 三元运算符

所谓的 "三元运算符", `?:`, 很类似 if-else 语法, 它用于选择性获取单个表达式的值, 而不是选择性执行大段的代码块。

```
a ? b : c
```

在 `?` 之前的表达式 `a`, 是一个条件表达式, 如果条件 `a` 是 `true`, 三元运算符计算在 `:` 之前的表达式 `b`; 如果条件 `a` 是 `false`, 则执行 `:` 后面的表达式 `c`。注意, `?` 和 `:` 旁边的空格是强制的。重要地是, 与 if-else 类似, `:` 之前和之后的表达式只有在条件表达式为 `true` 或者 `false` 时才会被相应地执行。

```
julia> x = 1; y = 2;  
julia> println(x < y ? "less than" : "not less than")  
less than  
  
julia> x = 1; y = 0;  
julia> println(x < y ? "less than" : "not less than")  
not less than
```

如果表达式 `x < y` 为真, 整个三元运算符会执行字符串 "less than", 否则执行字符串 "not less than"。

3. Julia 基础语法 – 语句

循环、控制流 – while 、 for 循环

while 循环会执行条件表达式，只要它为 true，就一直执行while 循环的主体部分。当 while 循环第一次执行时，如果条件表达式为 false，那么主体代码就一次也不会被执行。for 循环使得常见的重复执行代码写起来更容易。 while 循环中用到的向上和向下计数是可以用 for 循环更简明地表达。

```
julia> i = 1;
julia> while i <= 5
    println(i)
    global i += 1
end
1
2
3
4
5
```

```
julia> for i = 1:5
    println(i)
end
1
2
3
4
5
```

这里的 1:5 是一个范围对象，代表数字 1, 2, 3, 4, 5 的序列。for 循环在这些值之中迭代，对每一个变量 i 进行赋值。for 循环与之前 while 循环的一个非常重要区别是作用域，即变量的可见性。如果变量 i 没有在另一个作用域里引入，在 for 循环内，它就只在 for 循环内部可见，在外部和后面均不可见。

3. Julia 基础语法 – 语句

循环、控制流 – while 、 for 循环

一般来说，for 循环组件可以用于迭代任一容器。在这种情况下，相比 `=`，另外的（但完全相同）关键字 `in` 或者 `∈` 则更常用，因为它使得代码更清晰：

```
julia> for i in [1,4,0]
           println(i)
       end
1
4
0
```

```
julia> for s ∈ ["foo","bar","baz"]
           println(s)
       end
foo
bar
baz
```

3. Julia 基础语法 – 语句

循环、控制流 – while 、 for 循环

多个嵌套的 for 循环可以合并到一个外部循环，可以用来创建其迭代对象的笛卡尔积。每次内层循环运行的时候，变量 (i 和 j) 会被赋值为他们当前的迭代变量值。所以对 i 的赋值对于接下来的迭代是不可见的。

```
julia> for i = 1:2, j = 3:4
    println((i, j))
end
(1, 3)
(1, 4)
(2, 3)
(2, 4)
```

```
julia> for i = 1:2, j = 3:4
    println((i, j))
    i = 0
end
(1, 3)
(1, 4)
(2, 3)
(2, 4)
```

可以使用 zip 在单个 for 循环中同时迭代多个容器：

```
julia> for (j, k) in zip([1 2 3], [4 5 6 7])
    println((j, k))
end
(1, 4)
(2, 5)
(3, 6)
```

3. Julia 基础语法 – 语句

循环、控制流 – while 、 for 循环

为了方便，我们可能会在条件不成立之前终止一个 while 循环，或者在访问到迭代对象的结尾之前停止一个 for 循环，这可以用关键字 **break** 来完成：

```
julia> while true
    println(i)
    if i >= 5
        break
    end
    global i += 1
end
```

```
1
2
3
4
5
```

```
julia> for j = 1:1000
    println(j)
    if j >= 5
        break
    end
end
```

```
1
2
3
4
5
```

对于多个嵌套的 for 循环合并成的外部循环（见上一页），循环里面使用 break 语句则会跳出整个嵌套循环，不仅仅是内层循环。

3. Julia 基础语法 – 语句

循环、控制流 – while 、 for 循环

在某些场景下，需要直接结束此次迭代，并立刻进入下次迭代，**continue** 关键字可以用来完成此功能：

```
julia> i = 1;
julia> while i <= 10
    global i += 1
    if i % 3 != 0
        continue
    end
    println(i)
end
3
6
9
```

```
julia> for i = 1:10
    if i % 3 != 0
        continue
    end
    println(i)
end
3
6
9
```

3. Julia 基础语法 – 语句

字面量 – 字符

- 类似 C 和 Java，但是和大多数动态语言不同的是，Julia 有优秀的表示单字符的类型，即 `AbstractChar`。`Char` 是 `AbstractChar` 的内置子类型，它能表示任何 Unicode 字符的 32 位原始类型（基于 UTF-8 编码）。
- 字符使用单引号分隔。

```
julia> Chr = 'C'  
'C': ASCII/Unicode U+0043 (category Lu: Letter, uppercase)
```

- 可以将 `Char` 转换为其对应的整数值，即 Unicode 代码，也可以将一个整数值转回 `Char`：

```
julia> c = Int(Chr)  
67  
  
julia> typeof(c) # 在 32 位的计算机中，typeof(c) 将显示为 Int32。  
Int64  
  
julia> Char(67)  
'C': ASCII/Unicode U+0043 (category Lu: Letter, uppercase)
```

3. Julia 基础语法 – 语句

字面量 – 字符

- 可以在单引号中输入任何 Unicode 字符，通过使用 \u 加上至多 4 个十六进制数字或者 \U 加上至多 8 个十六进制数：

```
julia> '\u78'  
'x': ASCII/Unicode U+0078 (category Ll: Letter, lowercase)  
  
julia> '\U10ffff'  
\U10ffff': Unicode U+10FFFF (category Cn: Other, not assigned)
```

- 可以对 Char 的值进行比较和有限的算术运算：

```
julia> 'A' < 'a'  
true  
  
julia> 'A' <= 'a' <= 'Z'  
false  
  
julia> 'A' <= 'X' <= 'Z'  
true  
  
julia> 'x' - 'a'  
23  
  
julia> 'A' + 1  
'B': ASCII/Unicode U+0042 (category Lu: Letter, uppercase)
```

3. Julia 基础语法 – 语句

字面量 – 字符串

- 字符串字面量：由双引号或三重双引号分隔。

```
julia> str = "Hello, world.\n"  
"Hello, world.\n"
```

```
julia> """Contains "quote" characters"""  
"Contains \"quote\" characters"
```

- 字符串索引：如果要字符串中提取字符，可以对其进行索引，你也可以用范围索引来提取子字符串。索引是按照字节计算的。

```
julia> str[begin]  
'H': ASCII/Unicode U+0048 (category Lu: Letter, uppercase)
```

```
julia> str[begin]  
'H': ASCII/Unicode U+0048 (category Lu: Letter, uppercase)
```

```
julia> str[6]  
,': ASCII/Unicode U+002C (category Po: Punctuation, other)
```

```
julia> str[end]  
'\n': ASCII/Unicode U+000A (category Cc: Other, control)
```

```
julia> str[4:9]  
"lo, wo "
```

```
julia> str[6]  
,': ASCII/Unicode U+002C (category Po: Punctuation, other)
```

```
julia> str[6:6]  
,'
```

注意，表达式 `str[k]` 和 `str[k:k]` 不会给出相同的结果，前者是一个 `Char` 类型的单个字符，而后者是一个恰好只包含一个字符的字符串。

3. Julia 基础语法 – 语句

字面量 – 字符串

- 文件路径处理

在处理文件路径时，不同操作系统使用不同的路径分隔符。Windows 使用反斜杠 (\)，而 Unix/Linux 和 macOS 使用正斜杠 (/)。为了避免因路径分隔符导致的运行报错，可以使用以下几种方法来处理路径：

1. 使用 joinpath 函数：joinpath 函数会根据操作系统自动选择正确的路径分隔符

joinpath.jl

```
path = joinpath("folder", "subfolder", "file.txt")
println(path)
# 在 Windows 上输出: folder\subfolder\file.txt
# 在 Unix/Linux 和 macOS 上输出: folder/subfolder/file.txt
```

```
julia> 正在运行 joinpath.jl
folder/subfolder/file.txt
```

2. 使用 raw 字符串宏：在 Windows 上，可以使用 raw 字符串宏来避免转义字符的问题。

raw.jl

```
path = raw"C:\folder\subfolder\file.txt"
println(path) # 输出: C:\folder\subfolder\file.txt
```

```
julia> 正在运行 raw.jl
C:\folder\subfolder\file.txt
```

3. Julia 基础语法 – 语句

字面量 – 字符串

- 文件路径处理

3. 替换斜杠：你可以手动替换路径中的斜杠，确保路径符合当前操作系统的要求。

replace.jl

```
if Sys.iswindows()
    path = replace("folder/subfolder/file.txt", "/" => "\\")
else
    path = "folder/subfolder/file.txt"
end
```

```
julia> 正在运行 replace.jl
"folder/subfolder/file.txt"
```

4. 使用 Base.Filesystem.path_separator: Base.Filesystem.path_separator 返回当前操作系统的路径分隔符，你可以用它来动态构建路径。

path_separator.jl

```
sep = Base.Filesystem.path_separator
path = "folder" * sep * "subfolder" * sep * "file.txt"
```

```
julia> 正在运行 path_separator.jl
"folder/subfolder/file.txt"
```

3. Julia 基础语法 – 语句

字面量 – 字符串

- 字符串拼接

```
julia> greet = "Hello"
"Hello"

julia> whom = "world"
"world"

julia> string(greet, ", ", whom, ".\n")
"Hello, world.\n "

julia> greet * ", " * whom * ".\n"
"Hello, world.\n"
```

- 字符串插值：Julia 允许像 Perl 中一样使用 `$` 对字符串字面量进行插值，在 `$` 之后最短的完整表达式被视为插入其值于字符串中的表达式。

```
julia> "$greet, $whom.\n"
"Hello, world.\n "

julia> "1 + 2 = $(1 + 2)"
"1 + 2 = 3"
```

3. Julia 基础语法 – 语句

字面量 – 字符串

- 你可以使用标准的比较操作符按照字典顺序比较字符串：

```
julia> "abracadabra" < "xylophone"
true

julia> "abracadabra" == "xylophone"
false

julia> "Hello, world." != "Goodbye, world."
true

julia> "1 + 2 = 3" == "1 + 2 = $(1 + 2)"
true
```

- 你可以使用 **findfirst** 与 **findlast** 函数搜索特定字符的索引：

```
julia> findfirst(isequal('o'), "xylophone")
4

julia> findlast(isequal('o'), "xylophone")
7

julia> findfirst(isequal('z'), "xylophone")
```


3. Julia 基础语法 – 语句

字面量 – 字符串

- 你可以带上第三个参数，用 **findnext** 与 **findprev** 函数来在给定偏移量处搜索字符：

```
julia> findnext(isequal('o'), "xylophone", 1)
```

```
4
```

```
julia> findnext(isequal('o'), "xylophone", 5)
```

```
7
```

```
julia> findprev(isequal('o'), "xylophone", 5)
```

```
4
```

```
julia> findnext(isequal('o'), "xylophone", 8)
```

- 你可以用 **occursin** 函数检查在字符串中某子字符串可否找到。

```
julia> occursin("world", "Hello, world.")
```

```
true
```

```
julia> occursin("a", "Xylophon")
```

```
false
```

```
julia> occursin('o', "Xylophon")
```

```
true
```

3. Julia 基础语法 – 语句

字面量 – 字符串

- 正则表达式：在 Julia 中正则表达式的输入使用了前缀各类以 `r` 开头的标识符的非标准字符串字面量。

```
julia> re = r"^\s*(?:#|$)"  
r"^\s*(?:#|$)"  
  
julia> typeof(re)  
Regex
```

- 若要检查正则表达式是否匹配某字符串，就用 **occursin**。

```
julia> occursin(r"^\s*(?:#|$)", "not a comment")  
false  
  
julia> occursin(r"^\s*(?:#|$)", "# a comment")  
true
```

可以看到，`occursin` 只返回正确或错误，表明给定正则表达式是否在该字符串中出现。然而，通常我们不只想知道字符串是否匹配，更想了解它是如何匹配的。

3. Julia 基础语法 – 语句

字面量 – 字符串

- 要捕获匹配的信息，可以改用 **match** 函数

```
julia> m = match(r"(a|b)(c)?(d)", "ad")
RegexMatch("ad", 1="a", 2=nothing, 3="d")

julia> m.match
"ad"

julia> m.captures
3-element Vector{Union{Nothing, SubString{String}}}:
"a"
nothing
"d "

julia> m.offset
1

julia> m.offsets
3-element Vector{Int64}:
1
0
2
```

如果正则表达式匹配，`match` 的返回值是一个 `RegexMatch` 对象。这些对象记录了表达式是如何匹配的，包括该模式匹配的子字符串和任何可能捕获的子字符串。

你可以从 `RegexMatch` 对象中提取如下信息：

- 匹配的整个子字符串：`m.match`
- 作为字符串数组捕获的子字符串：`m.captures`
- 整个匹配开始处的偏移：`m.offset`
- 作为向量的捕获子字符串的偏移：`m.offsets`

若正则表达式与给定字符串不匹配，`match` 返回 `nothing`——在交互式提示框中不打印任何东西的特殊值。除了不打印，它是一个完全正常的值，这可以用程序来测试。

```
m = match(r"^\s*(?:#|$)", "abc")
if m === nothing
    println("not a comment")
else
    println("blank or comment")
end
```

3. Julia 基础语法 – 语句

字面量 – 数字

- 整数类型

IntN 和 UIntN, 且 $N \in \{8, 16, 32, 64, 128\}$, BigInt

整型字面量的默认类型取决于目标系统是 32 位还是 64 位架构。

```
# 32 位系统:  
julia> typeof(1)  
Int32
```

```
# 64 位系统:  
julia> typeof(1)  
Int64
```

Julia 也定义了 Int 与 UInt 类型，它们分别是系统有符号和无符号的原生整数类型的别名。

```
# 32 位系统:  
julia> Int  
Int32  
julia> UInt  
UInt32
```

```
# 64 位系统:  
julia> Int  
Int64  
julia> UInt  
UInt64
```

3. Julia 基础语法 – 语句

字面量 – 数字

- 整数类型

十六进制、二进制、八进制字面量：

```
julia> x = 0x1 # 十六进制
0x01
```

```
julia> typeof(x)
UInt8
```

```
julia> x = 0x123 # 十六进制
0x0123
```

```
julia> typeof(x)
UInt16
```

```
julia> x = 0b10 # 二进制
0x02
```

```
julia> typeof(x)
UInt8
```

```
julia> x = 0o010 # 八进制
0x08
```

```
julia> typeof(x)
UInt8
```

二进制、八进制和十六进制的字面量前面加一个负号 -, 这样可以产生一个和原字面量有着同样位数而值为原数的补码的数（二补数）：

```
julia> -0x2
0xfe
```

```
julia> -0x0002
0xfffe
```

3. Julia 基础语法 – 语句

字面量 – 数字

- 整数类型

整型等原始数值类型的最小和最大可表示的值可用 `typemin` 和 `typemax` 函数得到：

```
julia> (typemin(Int32), typemax(Int32))
(-2147483648, 2147483647)

julia> for T in [Int8,Int16,Int32,Int64,Int128,UInt8,UInt16,UInt32,UInt64,UInt128]
    println("$(\lpad(T,7)): [$(typemin(T)),$(typemax(T))]" )
end
Int8: [-128,127]
Int16: [-32768,32767]
Int32: [-2147483648,2147483647]
Int64: [-9223372036854775808,9223372036854775807]
Int128: [-170141183460469231731687303715884105728,170141183460469231731687303715884105727]
UInt8: [0,255]
UInt16: [0,65535]
UInt32: [0,4294967295]
UInt64: [0,18446744073709551615]
UInt128: [0,340282366920938463463374607431768211455]
```

3. Julia 基础语法 – 语句

字面量 – 数字

- 整数类型

溢出行为：在 Julia 里，超出一个类型可表示的最大值会导致环绕 (wraparound) 行为：

```
julia> x = typemax(Int64)
9223372036854775807

julia> x + 1
-9223372036854775808

julia> x + 1 == typemin(Int64)
true
```

推荐使用任意精度算术中的 BigInt 类型作为替代。

```
julia> 10^19
-8446744073709551616

julia> big(10)^19
100000000000000000000
```

3. Julia 基础语法 – 语句

字面量 – 数字

- 浮点类型

FloatN 且 $N \in \{16, 32, 64\}$, BigFloat

浮点数字面量以标准格式表示，必要时可使用E-表示法。使用 f 替代 e 可以得到 Float32 类型的字面量。

```
julia> 1.0  
1.0
```

```
julia> .5  
0.5
```

```
julia> -1.23  
-1.23
```

```
julia> 1e10  
1.0e10
```

```
julia> 2.5e-4  
0.00025
```

```
julia> x = 0.5f0  
0.5f0
```

```
julia> typeof(x)  
Float32
```

```
julia> 2.5f-4  
0.00025f0
```

数值可以很容易地转换为 Float32 类型

```
julia> x = Float32(-1.5)  
-1.5f0
```

```
julia> typeof(x)  
Float32
```


3. Julia 基础语法 – 语句

字面量 – 数字

- 浮点类型

特殊的浮点值：Julia 有三种特定的标准浮点值不和实数轴上任何一点对应

Float16	Float32	Float64	名称
Inf16	Inf32	Inf	正无穷
0	0	-Inf	负无穷
NaN16	NaN32	NaN	不是数 (Not a Number)

根据 IEEE 754 标准，这些浮点值是某些算术运算的结果：

```
julia> 1/Inf
0.0

julia> 1/0
Inf

julia> 0/0
NaN
```

```
julia> 500 + Inf
Inf
```

```
julia> 500 - Inf
-Inf
```

```
julia> Inf - Inf
NaN
```

```
julia> 0 * Inf
NaN
```

```
julia> NaN == NaN
false
```

```
julia> isequal(NaN, NaN)
true
```

```
julia> NaN < NaN
false
```

```
julia> NaN > NaN
false
```

```
julia> (typemin(Float16), typemax(Float16))
(-Inf16, Inf16)
```

```
julia> (typemin(Float32), typemax(Float32))
(-Inf32, Inf32)
```

```
julia> (typemin(Float64), typemax(Float64))
(-Inf, Inf)
```

== 是用于比较两个值是否相等的通用运算符。其行为通常符合数学上的相等性，并且会根据类型的不同进行相应的重载。

- 对于浮点数，== 遵循 IEEE 标准，认为 NaN 不等于任何值，包括它自己。

- 对于缺失值 (missing)，== 认为两个 missing 不相等。

isequal 用于比较两个值是否相等，并且在处理浮点数和缺失值 (missing) 时具有特定的行为。它主要用于哈希表和集合中，以确保相同的值具有相同的哈希值。

- 对于浮点数，isequal 会认为 NaN (“不是一个数字”) 是相等的。

- 对于缺失值 (missing)，isequal 会认为两个 missing 是相等的。

3. Julia 基础语法 – 语句

字面量 – 数字

- 浮点类型

Julia 提供了 `eps` 函数，它可以给出 1.0 与下一个 Julia 能表示的浮点数之间的差值，通常被叫做机器精度。

```
julia> eps(Float32)
1.1920929f-7

julia> eps(Float64)
2.220446049250313e-16

julia> eps() # 与 eps(Float64) 相同
2.220446049250313e-16

julia> eps(1.0)
2.220446049250313e-16

julia> eps(1000.)
1.1368683772161603e-13

julia> eps(1e-27)
1.793662034335766e-43
```

3. Julia 基础语法 – 语句

字面量 – 数字

- 复数

全局常量 `im` 被绑定到复数 `i`, 表示 -1 的主平方根。

```
julia> 1+2im
1 + 2im

# 对复数进行各种标准算术操作
julia> (1 + 2im)*(2 - 3im)
8 + 1im

julia> (1 + 2im) + (1 - 2im)
2 + 0im

julia> (-1 + 2im)^2.5
2.729624464784009 - 6.9606644595719im

julia> (-1 + 2im)^(1 + 1im)
-0.27910381075826657 + 0.08708053414102428im

julia> 3(2 - 5im)^2
-63 - 60im
```

类型提升机制也确保你可以使用不同类型的操作数的组合

```
julia> 2(1 - 1im)
2 - 2im

julia> (2 + 3im) - 1
1 + 3im

julia> (2 + 3im) - 0.5im
2.0 + 2.5im

julia> 0.75(1 + 2im)
0.75 + 1.5im

julia> (1 - 3im) / (2 + 2im)
-0.5 - 1.0im

julia> 2im^2
-2 + 0im
```

3. Julia 基础语法 – 语句

字面量 – 数字

- 复数

Julia 提供了一些操作复数的标准函数

```
julia> z = 1 + 2im  
1 + 2im
```

```
julia> real(1 + 2im) # z 的实部  
1
```

```
julia> imag(1 + 2im) # z 的虚部  
2
```

```
julia> conj(1 + 2im) # z 的复共轭  
1 - 2im
```

```
julia> abs(1 + 2im) # z 的绝对值  
2.23606797749979
```

```
julia> angle(1 + 2im) # 以弧度为单位的相位角  
1.1071487177940904
```

```
julia> sqrt(1im)  
0.7071067811865476 + 0.7071067811865475im
```

```
julia> sqrt(1 + 2im)  
1.272019649514069 + 0.7861513777574233im
```

```
julia> cos(1 + 2im)  
2.0327230070196656 - 3.0518977991517997im
```

```
julia> sqrt(-1 + 0im)  
0.0 + 1.0im
```

```
julia> a = 1; b = 2; complex(a, b)  
1 + 2im
```

Inf 和 NaN 可能出现在复数的实部和虚部

```
julia> 1 + Inf*im  
1.0 + Inf*im
```

```
julia> 1 + NaN*im  
1.0 + NaN*im
```

3. Julia 基础语法 – 语句

字面量 – 数字

- 有理数

Julia 有一个用于表示整数精确比值的分数类型。
分数通过 `//` 运算符构建：

```
julia> 2//3
2//3

julia> 6//9
2//3

julia> -4//8
-1//2

julia> 5//-15
-1//3

julia> -4//-12
1//3
```

分数值的标准化分子和分母可以使用 `numerator` 和 `denominator` 函数得到：

```
julia> numerator(2//3)
2

julia> denominator(2//3)
3
```

分数值的比较和标准算术操作：

```
julia> 2//3 == 6//9
true

julia> 3//7 < 1//2
true

julia> 2//4 + 1//6
2//3

julia> 5//12 - 1//4
1//6

julia> 5//8 * 3//12
5//32
```

3. Julia 基础语法 – 语句

字面量 – 数字

- 有理数

分数可以很容易地转换成浮点数:

```
julia> float(3//4)
0.75

julia> a = 1; b = 2;

julia> isequal(float(a//b), a/b)
true
```

Julia接受构建无穷分数值:

```
julia> 5//0
1//0

julia> x = -3//0
-1//0

julia> typeof(x)
Rational{Int64}
```

类型提升系统使得分数可以轻松同其它数值类型进行交互:

```
julia> 3//5 + 1
8//5

julia> 2//7 * (1 + 2im)
2//7 + 4//7*im

julia> 2//7 * (1.5 + 2im)
0.42857142857142855 + 0.5714285714285714im

julia> 1//2 + 2im
1//2 + 2//1*im

julia> 0.33 == 1//3
false

julia> 0.33 < 1//3
true

julia> 1//3 - 0.33
0.00333333333333332993
```

3. Julia 基础语法 – 语句

字面量 – 数组

数组是存储在多维网格中对象的集合。与大多数科学计算语言一样，Julia 提供原生的数组实现。

函数	描述
<code>eltype(A)</code>	A 中元素的类型
<code>length(A)</code>	A 中元素的数量
<code>ndims(A)</code>	A 的维数
<code>size(A)</code>	一个包含 A 各个维度上元素数量的元组
<code>size(A,n)</code>	A 第 n 维中的元素数量
<code>axes(A)</code>	一个包含 A 有效索引的元组
<code>axes(A,n)</code>	第 n 维有效索引的范围
<code>eachindex(A)</code>	一个访问 A 中每一个位置的高效迭代器
<code>stride(A,k)</code>	在第 k 维上的间隔 (stride) (相邻元素间的线性索引距离)
<code>strides(A)</code>	包含每一维上的间隔 (stride) 的元组

3. Julia 基础语法 – 语句

字面量 – 数组

- 数组常量

数组也可以直接用方括号来构造; 语法为 `[A, B, C, ...]` 创建一个一维数组(即一个向量), 该一维数组的元素用逗号分隔。所创建的数组中元素的类型(eltype) 自动由括号内参数的类型确定。如果所有参数类型都相同, 则该类型称为数组的 eltype。如果所有元素都有相同的 promotion type, 那么每个元素都由 convert 转换成该类型并且该类型为数组的 eltype。否则, 生成一个可以包含任意类型的异构数组——`Vector{Any}`; 该构造方法包含字符 `[]`, 此时构造过程无参数给出。

```
julia> [1,2,3] # 元素类型为 Int 的向量
```

```
3-element Vector{Int64}:
```

```
1
2
3
```

```
julia> promote(1, 2.3, 4//5) # Int, Float64 以及 Rational 类型
```

```
放在一起则会提升到 Float64
```

```
(1.0, 2.3, 0.8)
```

```
julia> [1, 2.3, 4//5] # 从而Float64就是这个数组的元素类型
```

```
3-element Vector{Float64}:
```

```
1.0
2.3
0.8
```

```
julia> []  
Any[]
```


3. Julia 基础语法 – 语句

字面量 – 数组

- 数组拼接

如果方括号里的参数不是由逗号分隔，而是由单个分号(;) 或者换行符分隔，那么每一个参数就不再解析为一个单独的数组元素，而是纵向拼接起来。如果这些参数是被制表符、空格符或者两个分号所分隔，那么它们的内容就横向拼接在一起。

```
julia> [1:2, 4:5] # 这里有一个逗号，因此并不会发生矩阵的拼接。
```

```
range 本身就是元素
```

```
2-element Vector{UnitRange{Int64}}:
```

```
1:2
```

```
4:5
```

```
julia> [1:2; 4:5]
```

```
4-element Vector{Int64}:
```

```
1
```

```
2
```

```
4
```

```
5
```

```
julia> [1:2 4:5 7:8]
```

```
2×3 Matrix{Int64}:
```

```
1 4 7
```

```
2 5 8
```

```
julia> [[1,2] [4,5] [7,8]]
```

```
2×3 Matrix{Int64}:
```

```
1 4 7
```

```
2 5 8
```

```
julia> [1 2 3] # 数字可以被横向拼接
```

```
1×3 Matrix{Int64}:
```

```
1 2 3
```

```
julia> [1;; 2;; 3;; 4]
```

```
1×4 Matrix{Int64}:
```

```
1 2 3 4
```

3. Julia 基础语法 – 语句

字面量 – 数组

- 数组拼接

单个分号（或换行符）和空格（或制表符）可以被结合起来使用进行横向或者纵向的拼接，空格（和制表符）的优先级高于分号，首先执行纵向拼接，然后拼接结果。另一方面，使用双分号进行水平连接时，先纵向拼接再横向拼接。

```
julia> [1 2  
        3 4]  
2×2 Matrix{Int64}:  
 1 2  
 3 4
```

```
julia> [zeros(Int, 2, 2) [1; 2]  
        [3 4]          5]  
3×3 Matrix{Int64}:  
 0 0 1  
 0 0 2  
 3 4 5
```

```
julia> [[1 1]; 2 3; [4 4]]  
3×2 Matrix{Int64}:  
 1 1  
 2 3  
 4 4
```

```
julia> [zeros(Int, 2, 2) ; [3 4] ;; [1; 2] ; 5]  
3×3 Matrix{Int64}:  
 0 0 1  
 0 0 2  
 3 4 5
```

```
julia> [1:2; 4;; 1; 3:4]  
3×2 Matrix{Int64}:  
 1 1  
 2 3  
 4 4
```

3. Julia 基础语法 – 语句

字面量 – 数组

- 数组拼接

正如 ; 和 ;; 在第一维和第二维中拼接一样，使用更多的分号扩展了相同的通用方案。分隔符中的分号数指定了特定的维度，因此;;; 在第三个维度中拼接，;; 在第四个维度中，依此类推。较少的分号优先级高，因此较低的维度通常首先拼接。

像之前一样，用于水平拼接的空格（和制表符）的优先级高于任何数量的分号。因此，高维数组也可以通过首先指定它们的行来编写，它们的元素以类似于它们的布局的方式进行文本排列。

```
julia> [1; 2;; 3; 4;; 5; 6;;;  
        7; 8;; 9; 10;; 11; 12]
```

```
2×3×2 Array{Int64, 3}:
```

```
[:, :, 1] =
```

```
1 3 5
```

```
2 4 6
```

```
[:, :, 2] =
```

```
7 9 11
```

```
8 10 12
```

```
julia> [1 3 5
```

```
        2 4 6;;;
```

```
        7 9 11
```

```
        8 10 12]
```

```
2×3×2 Array{Int64, 3}:
```

```
[:, :, 1] =
```

```
1 3 5
```

```
2 4 6
```

```
[:, :, 2] =
```

```
7 9 11
```

```
8 10 12
```

2. Julia 基础语法 – 定义

字面量 – 数组

- 数组拼接

可以通过cat 函数来实现数组元素的拼接功能。 以下这些的语法为这些函数的简写形式，它们本身也是非常方便使用的：

语法	函数	描述
	cat(A, B, dims=k)	沿着第k维拼接数组
[A; B; C; ...]	vcat(A,B,C,...)	纵向拼接
[A B C ...]	hcat(A,B,C,...)	横向拼接
[A B; C D; ...]	hvcat(rows, A, B, C, D)	同时沿垂直和水平方向拼接
[A; C;; B; D;;; ...]	hvncat	同时进行n维拼接，其中分号的数量表示拼接所在的维度

3. Julia 基础语法 – 语句

字面量 – 数组

(数组) 推导提供了构造数组的通用且强大的方法。其语法类似于数学中的集合构造的写法:

$A = [F(x,y,...) \text{ for } x=r_x, y=r_y, \dots]$

```
julia> x = rand(8)
8-element Array{Float64,1}:
 0.843025
 0.869052
 0.365105
 0.699456
 0.977653
 0.994953
 0.41084
 0.809411

julia> [ 0.25*x[i-1] + 0.5*x[i] + 0.25*x[i+1] for i=2:length(x)-1 ]
6-element Array{Float64,1}:
 0.736559
 0.57468
 0.685417
 0.912429
 0.8446
 0.656511
```

3. Julia 基础语法 – 语句

字面量 – 数组

索引 n 维数组 A 的一般语法是: $X = A[l_1, l_2, \dots, l_n]$

```
julia> A = reshape(collect(1:16), (2, 2, 2, 2))  
2×2×2×2 Array{Int64, 4}
```

```
julia> A[1, 2, 1, 1] # 全部为标量索引  
3
```

```
julia> A[[1, 2], [1], [1, 2], 1] # 标量与向量索引的混合使用  
2×1×2 Array{Int64, 3}
```

```
julia> x = reshape(1:16, 4, 4)  
4×4 reshape(::UnitRange{Int64}, 4, 4) with eltype  
Int64:
```

```
julia> x[2:3, 2:end-1]  
2×2 Matrix{Int64}:  
 6 10  
 7 11
```

在 n 维数组 A 中赋值的一般语法是: $A[l_1, l_2, \dots, l_n] = X$

```
julia> x = collect(reshape(1:9, 3, 3))  
3×3 Matrix{Int64}:
```

```
 1  4  7  
 2  5  8  
 3  6  9
```

```
julia> x[3, 3] = -9;
```

```
julia> x[1:2, 1:2] = [-1 -4; -2 -5];
```

```
julia> x  
3×3 Matrix{Int64}:  
 -1 -4  7  
 -2 -5  8  
  3  6 -9
```

3. Julia 基础语法 – 语句

字面量 – 数组

- 向量和矩阵的相互转化

1. 向量转矩阵

Vector2Matrix.jl

```
# 将向量转为列向量
v = [1, 2, 3] # Vector
column_matrix = reshape(v, :, 1) # 转为列 3x1 矩阵
println(column_matrix) # 输出: [1; 2; 3;]

# 将向量转为行向量
v = [1, 2, 3] # Vector
row_matrix = reshape(v, 1, :) # 转为 1x3 矩阵
println(row_matrix) # 输出: [1 2 3]

# 将向量转为指定形状的矩阵
v = [1, 2, 3, 4, 5, 6] # Vector
matrix_2x3 = reshape(v, 2, 3) # 转为 2x3 矩阵
println(matrix_2x3) # 输出: [1 2 3; 4 5 6]
```

```
julia> 正在运行 Vector2Matrix.jl
[1; 2; 3;]
[1 2 3]
[1 3 5; 2 4 6]
```

3. Julia 基础语法 – 语句

字面量 – 数组

- 向量和矩阵的相互转化

2. 矩阵转向量

Matrix2Vector.jl

```
# 使用 vec 函数  
A = [1 2; 3 4]  
v = vec(A)  
println(v) # 输出: [1, 3, 2, 4]
```

```
# 使用 reshape 函数  
A = [1 2; 3 4]  
v = reshape(A, :)  
println(v) # 输出: [1, 3, 2, 4]
```

```
julia> 正在运行 Matrix2Vector.jl  
[1, 3, 2, 4]  
[1, 3, 2, 4]
```


3. Julia 基础语法 – 语句

字面量 – 元组

Julia 有一个和函数参数与返回值密切相关的内置数据结构叫做元组 (tuple)。一个元组是一个固定长度的容器，可以容纳任何值，但不可以被修改。元组通过圆括号和逗号来构造，其内容可以通过索引来访问，命名元组还可以使用点操作来访问。

```
julia> (1, 1+1)
(1, 2)

julia> (1,)
(1,)

julia> x = (0.0, "hello", 6*7)
(0.0, "hello", 42)

julia> x[2]
"hello"
```

```
julia> x = (a=2, b=1+2)
(a = 2, b = 3)

julia> x[1]
2

julia> x[:a]
2

julia> x.a
2
```

3. Julia 基础语法 – 语句

高级表达式 – 导入模块

命名空间管理是指语言提供的设施，用于使模块中的名称在其他模块中可用。

1. 加载模块最常见的方式是 `using ModuleName`。加载与 `ModuleName` 关联的代码，并引入
 - a. 模块名称
 - b. 和导出列表的元素到周围的全局命名空间中。

```
using ModuleName
```

2. 对应的，`import` 仅将模块名称带入作用域。当用户想要保持命名空间干净时，在上下文中可使用 `import ModuleName`。

```
import ModuleName
```

3. Julia 基础语法 – 语句

高级表达式 – 语句块

在 Julia 中，你可以使用 `begin` 和 `end` 关键字来定义一个代码块，其中 `do` 关键字常用于定义匿名函数或者与迭代器一起使用。

```
# 定义一个数组
numbers = [1, 2, 3, 4, 5]

# 使用 `do` 结构和 `map` 函数来对数组中的每个元素加倍
doubled_numbers = map(numbers) do number
    2 * number
end

# 使用 `begin` 和 `end` 来定义一个更复杂的代码块
total = begin
    su = 0
    for num in doubled_numbers
        global su += num
    end
    su # 返回总和
end

println("原始数字: ", numbers)
println("加倍后的数字: ", doubled_numbers)
println("数字总和: ", total)
```

```
julia> 正在运行 begin&do.jl
原始数字: [1, 2, 3, 4, 5]
加倍后的数字: [2, 4, 6, 8, 10]
数字总和: 30
```

3. Julia 基础语法 – 语句

高级表达式 – 宏调用

宏的通常调用语法: @name expr1 expr2 ... @name(expr1, expr2, ...)

```
julia> macro showarg(x)
    show(x)
    # ... remainder of macro, returning an expression
end
@showarg (macro with 1 method)
```

```
julia> @showarg(a)
:a
```

```
julia> @showarg(1+1)
:(1 + 1)
```

```
julia> @showarg 6
6
```

```
julia> @showarg(println("Yo!"))
:(println("Yo!"))
```

3. Julia 基础语法 – 语句

global&local.jl

高级表达式 – 作用域声明

在 Julia 中，`global` 和 `local` 关键字用于明确地声明变量的作用域。这在有嵌套作用域的情况下特别重要，如在函数内部或循环内部使用外部的全局变量使用 `global`，限定某些变量的作用域为本地时使用 `local`。

```
# 声明一个全局变量
global counter = 0

# 定义一个函数，演示局部变量和全局变量的使用
function increment()
    # 声明使用外部的全局变量
    global counter
    counter += 1

    # 声明一个局部变量
    local temp = 5

    # 对局部变量进行操作
    temp += 3
    println("局部变量 temp 的值: ", temp)
end

# 调用函数
increment()

println("全局变量 counter 的值: ", counter)
# 1

# 在循环中使用局部和全局变量
for i in 1:3
    # 修改全局变量
    global counter += 1
end

println("循环后的全局变量 counter 的值: ", counter)
# 4
```

```
julia> 正在运行 global&local.jl
局部变量 temp 的值: 8
全局变量 counter 的值: 1
循环后的全局变量 counter 的值: 4
```

Syslab Julia 函数与数据类型

Syslab 语言组

苏州同元软控信息技术有限公司

2024-08-02



TONGYUAN
Software and Control

目录

1. 函数基础

2. 类型与结构体

3. 多重分派

4. 参数化类型

5. 常用的宏

01 函数基础

@苏州同元软控信息技术有限公司版权所有，未经许可，不得复印、传播或以其他方式使用
版权所有，侵权必究

1 函数基础 – 函数定义

方式一，完整结构：

```
function functionname(args)
    expression
    expression
    expression
    ...
    expression
end
```

方式二，简化写法：

```
functionname(args) = expression
```

```
julia> function f(x, y)
    return x + y
end
```

```
julia> f(2, 3)
5
```

```
julia> f(x, y) = x + y
```

```
julia> f(2, 3)
5
```

1 函数基础 – 函数的输入参数

参数传递行为: passing-by-sharing (共享传参)

变量在传递时并不会被赋值, 在函数内对可变值的修改对调用者是可见的。

```
julia> function f(x, y)
    x[1] = 42 # mutates x
    y = 7 + y # new binding for y, no mutation
    return y
end
```

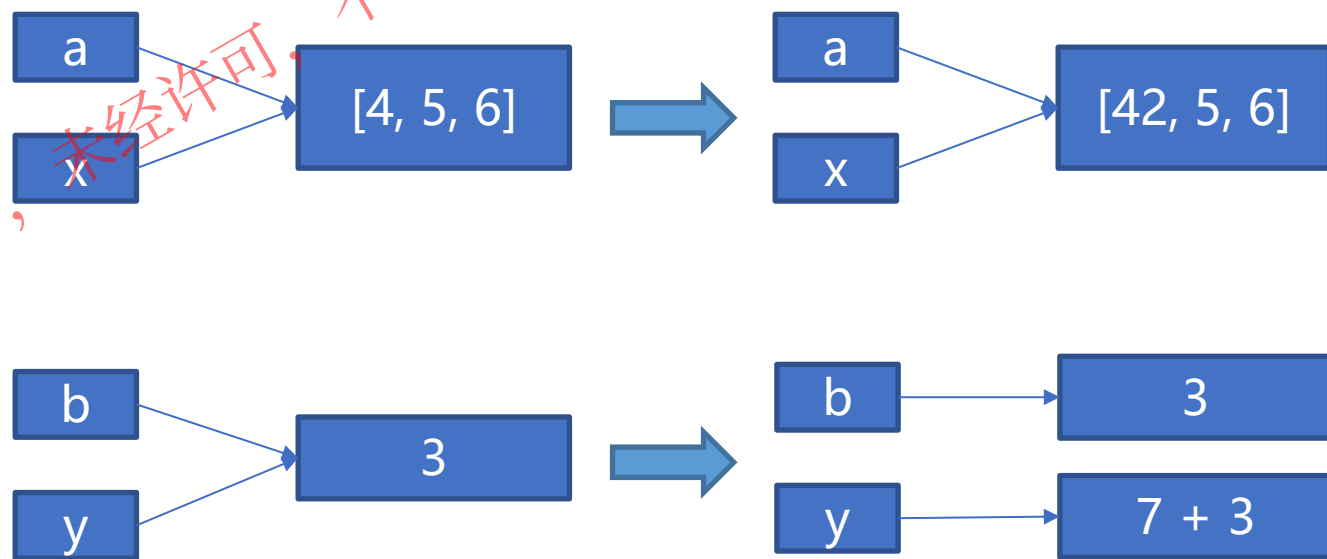
```
julia> a = [4,5,6];
```

```
julia> b = 3;
```

```
julia> f(a, b) # returns 7 + b == 10
10
```

```
julia> a # a[1] is changed to 42 by f
3-element Vector{Int64}:
 42
  5
  6
```

```
julia> b # not changed
3
```



1 函数基础 – 函数的输入参数

参数类型声明

可以通过将 `::TypeName` 附加到参数名称来声明函数参数的类型。

例如，下面的函数递归地计算斐波那契数：

```
fib(n::Integer) = n ≤ 2 ? one(n) : fib(n-1) + fib(n-2)
```

参数类型声明通常对性能没有影响，在 Julia 中声明参数类型最常见的原因是：

- 正确性：如果函数只为某些参数类型返回正确的结果，则类型声明会很有用。例如，如果我们省略参数类型并写成 `fib(n) = n ≤ 2 ? one(n) : fib(n-1) + fib(n-2)`，然后 `fib(1.5)` 会悄悄地给我们无意义的答案 1.0。
- 派发（详见 3.3 章节）：对于不同的参数类型，你可以有不同版本（“方法”）的函数，在这种情况下，参数类型用于确定调用哪个版本的函数。例如，你可以实现一个完全不同的算法 `fib(x::Number) = ...`，该算法扩展为了非整数值，适用于任何 `Number` 类型。
- 清晰性：类型声明可以作为一种关于预期参数的文档形式。

1 函数基础 – 函数的输入参数

变参函数

你可以通过在最后一个参数后增加省略号来定义一个变参函数:

```
# 变量 a 和 b 和以前一样被绑定给前两个参数, 后面的参数整个  
# 做为迭代集合被绑定到变量 x 上, x 为一个元组。
```

```
bar(a, b, x...) = (a, b, x)
```

```
# bar (generic function with 1 method)
```

```
bar(1, 2)
```

```
# (1, 2, ())
```

```
bar(1, 2, 3)
```

```
# (1, 2, (3,))
```

```
bar(1, 2, 3, 4)
```

```
# (1, 2, (3, 4))
```

```
bar(1, 2, 3, 4, 5, 6)
```

```
# (1, 2, (3, 4, 5, 6))
```

```
julia> bar(1, 2)  
(1, 2, ())
```

```
julia> bar(1, 2, 3)  
(1, 2, (3,))
```

```
julia> bar(1, 2, 3, 4)  
(1, 2, (3, 4))
```

```
julia> bar(1, 2, 3, 4, 5, 6)  
(1, 2, (3, 4, 5, 6))
```

1 函数基础 – 函数的输入参数

可选参数

可以为函数参数提供合理的默认值。这可以避免用户在每次调用时必须传递每个参数:

```
# 分别给变量 b, c 默认值为 2, 3, 当没有显式的传入参数 b 或 c 时,  
b 的值为 2, c 的值为 3。
```

```
f(a, b=2, c=3) = a + b + c
```

```
# f (generic function with 1 method)
```

```
f(1) # 等价于 f(1, 2, 3)  
# 6
```

```
f(1, 10) # 等价于 f(1, 10, 3)  
# 14
```

```
f(1, 10, 10)  
# 21
```

```
julia> f(a, b=2, c=3) = a + b + c  
f (generic function with 3 methods)
```

```
julia> f(1)  
6
```

```
julia> f(1, 10)  
14
```

```
julia> f(1, 10, 10)  
21
```



1 函数基础 – 函数的输入参数

关键字参数

在 Julia 中，关键字参数 (keyword arguments) 允许你在调用函数时通过参数名指定参数值，使得代码更具可读性和灵活性。关键字参数可以使这些复杂的接口更容易使用和扩展，因为它们允许通过名称而不是通过位置来标识参数，具有关键字参数的函数在签名中**使用分号定义**：

```
function myplot(x, y; style="solid", width=1, color="black")  
    ###  
end
```

例如上述定义的一个绘制直线的函数。这个函数有很多选项，用于控制线条样式、宽度、颜色。如果我们希望只修改宽度和颜色，可以这样调用： `myplot (x, y, color = "red", width = 2)`。这样做有两个目的：

1. 这个调用更容易阅读，因为我们可以用它的含义来标记一个参数；
2. 可以以任意顺序传递大量参数的任意子集。

```
function myprint(a::String, b::String; Delimiter::String = " , ",  
    EndOfLine::String = "\n")  
    println(a * Delimiter * b * EndOfLine)  
end
```

```
myprint("Hello", "Julia", Delimiter = " ")  
myprint("Hello", "Julia", EndOfLine=".")  
myprint("Hello", "Julia")
```

```
julia> function myprint(a::String, b::String; Delimiter::String=" , ", End  
OfLine::String="\n")  
    println(a * Delimiter * b * EndOfLine)  
end  
myprint (generic function with 1 method)  
  
julia> myprint("Hello", "Julia", Delimiter=" ")  
Hello Julia  
  
julia> myprint("Hello", "Julia", EndOfLine=".")  
Hello , Julia.  
  
julia> myprint("Hello", "Julia")  
Hello , Julia
```

1 函数基础 – 函数的返回值

return 关键字

默认情况下，返回值函数定义体中的最后一个表达式。

在函数 `f` 中，返回表达式 `x + y` 的值：

```
function f(x, y)
    x + y
end
```

```
f(1, 2)
```



```
julia> function f(x, y)
    x + y
end
f (generic function with 1 method)

julia> f(1, 2)
3
```

与许多其他语言中一样，`return` 关键字会导致函数立即返回，提供一个返回值的表达式：

```
function g(x, y)
    return x + y
    x * y
end
```

```
g(1, 2)
```



```
julia> function g(x, y)
    return x + y
    x * y
end
g (generic function with 1 method)

julia> g(1, 2)
3
```

1 函数基础 – 函数的返回值

返回值类型声明

与输入参数的类型声明类似，可以使用 `::TypeName` 在函数声明中指定返回类型。这会将返回值转换为指定的类型。

```
function g(x, y)::Int8
    return x * y
end
```

```
typeof(g(1, 2))
```



```
julia> function g(x, y)::Int8
           return x * y
       end
g (generic function with 1 method)

julia> typeof(g(1, 2))
Int8
```

返回 nothing

对于不需要任何返回值的函数，Julia 中的写法为返回值 `nothing`：

```
function printx(x)
    println("x = $x")
    return nothing
end

r = printx("hello!")
println(r === nothing)
```



```
julia> function printx(x)
           println("x = $x")
           return nothing
       end
printx (generic function with 1 method)

julia> r = printx("hello!")
x = hello!

julia> println(r === nothing)
true
```


1 函数基础 – 函数的返回值

多返回值

可以通过返回元组或其他可迭代值来从函数返回多个值。例如，下面的函数返回两个值：

```
function foo(a, b)
    a + b, a * b
end

# 如果在交互式会话中调用它，而没有任何地方分配返回值，您将看到元组返回
foo(2, 3)

# 将每个值提取到一个变量中：
x, y = foo(2, 3)
```

```
julia> function foo(a, b)
           a + b, a * b
       end
foo (generic function with 1 method)

julia> foo(2, 3)
(5, 6)

julia> x, y = foo(2, 3)
(5, 6)

julia> x
5

julia> y
6
```

1 函数基础 – 操作符

操作符

在 Julia 中，大多数操作符只不过是支持特殊语法的函数（&& 和 || 等具有特殊评估语义的操作符除外），因此，你也可以使用带括号的参数列表来使用它们，就和任何其他函数一样：

```
julia> 1 + 2 + 3
```

```
6
```

```
julia> +(1,2,3)
```

```
6
```

你可以对操作符，例如 + 和 *，进行赋值和传参，就像其它函数传参一样：

```
julia> f = +;
```

```
julia> f(1,2,3)
```

```
6
```

1 函数基础 – 匿名函数

匿名函数

函数可以用作参数，也可以当作返回值。函数也可以不带函数名称地匿名创建，使用语法如下：

```
julia> x -> x^2 + 2x - 1
#1 (generic function with 1 method)

julia> function (x)
    x^2 + 2x - 1
end
#3 (generic function with 1 method)
```

这样就创建了一个接受一个参数 x 并返回当前值的多项式 x^2+2x-1 的函数。

匿名函数最主要的用法是传递给接收函数作为参数的函数。一个经典的例子是 `map`，为数组的每个元素应用一次函数，然后返回一个包含结果值的新数组：

```
julia> map(round, [1.2, 3.5, 1.7])
3-element Vector{Float64}:
 1.0
 4.0
 2.0
```

1 函数基础 – 匿名函数

匿名函数

如果做为第一个参数传递给 map 的转换函数已经存在，那直接使用函数名称是没问题的。但是通常要使用的函数还没有定义好，这样使用匿名函数就更加方便：

```
julia> map(x -> x^2 + 2x - 1, [1, 3, -1])  
3-element Vector{Int64}:  
 2  
14  
-2
```

接受多个参数的匿名函数写法可以使用语法 $(x,y,z) \rightarrow 2x+y-z$ ，而无参匿名函数写作 $() \rightarrow 3$ 。

1 函数基础 – 向量化

向量化

在科学计算语言中，通常会有函数的「向量化」版本，这种语法便于数据处理，但在其它语言中，向量化通常也是性能所需要的：如果循环很慢，函数的「向量化」版本可以调用由低级语言编写的、快速的库代码。任何 Julia 函数 f 能够以元素方式作用于任何数组（或者其它集合），这通过语法 $f(A)$ 实现。例如， $\sin$ 可以作用于向量 A 中的所有元素：

```
julia> A = [1.0, 2.0, 3.0];

julia> sin.(A)
3-element Vector{Float64}:
 0.8414709848078965
 0.9092974268256817
 0.1411200080598672
```

你如果为 f 编写了一个专门的「向量化」方法，例如通过 $f(A::AbstractArray) = \text{map}(f, A)$ ，可以省略点号，这和 $f(A)$ 一样高效。但这种方法要求你事先决定要进行向量化的函数。

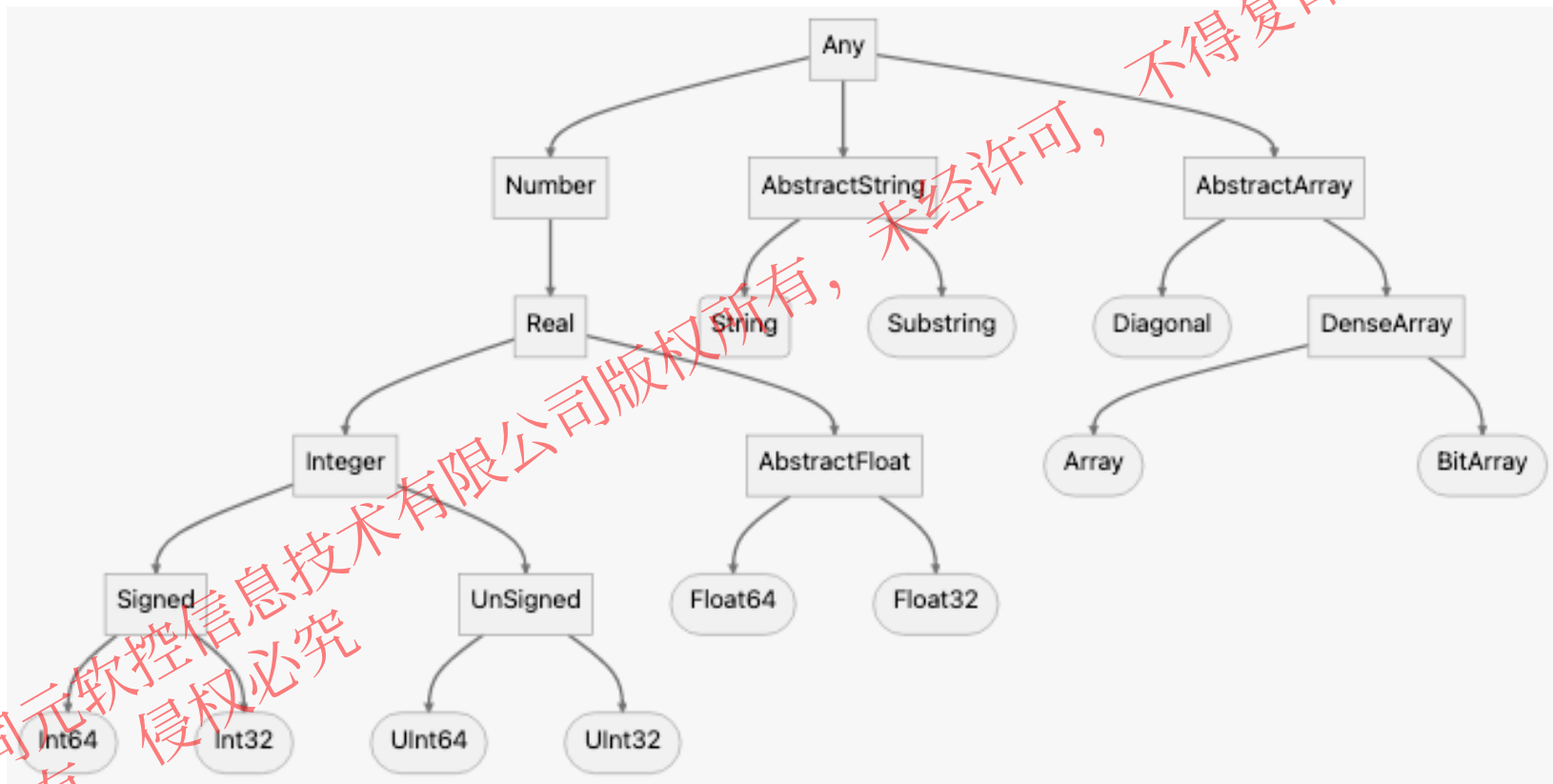
02 类型与结构体

©苏州同元软控信息技术有限公司版权所有，未经许可，不得复印、传播或以其他方式使用

@苏州同元软控信息技术有限公司
版权所有，侵权必究

2 类型与结构体 – 常用数值类型

整数和浮点值是算术和计算的基础。这些数值的内置表示被称作原始数值类型（numeric primitive），且整数和浮点数在代码中作为立即数时称作数值字面量（numeric literal）。例如，1 是个整型字面量，1.0 是个浮点型字面量，它们在内存中作为对象的二进制表示就是原始数值类型。



2 类型与结构体 – 常用数值类型

整数类型:

类型	是否带符号	比特数	最小值	最大值
Int8	√	8	-2^7	2^7-1
UInt8		8	0	2^8-1
Int16	√	16	-2^{15}	$2^{15}-1$
UInt16		16	0	$2^{16}-1$
Int32	√	32	-2^{31}	$2^{31}-1$
UInt32		32	0	$2^{32}-1$
Int64	√	64	-2^{63}	$2^{63}-1$
UInt64		64	0	$2^{64}-1$
Int128	√	128	-2^{127}	$2^{127}-1$
UInt128		128	0	$2^{128}-1$
Bool	N/A	8	false(0)	true(1)

```
julia> a = 1
1

julia> typeof(a) #操作系统为64位
Int64

julia> max = typemax(Int64)
9223372036854775807

julia> max + 1 #超限
-9223372036854775808

julia> min = typemin(Int64)
-9223372036854775808

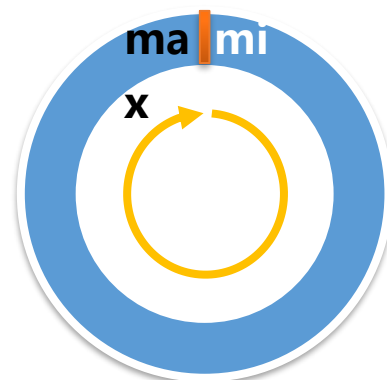
julia> min - 1 #超限
9223372036854775807

julia> x = typeof(0x123)
UInt16

julia> Int64(0x123) #转化为Int64
291
```

说明:

- 根据操作系统不同, 整数 Int 可能是 Int32 或 Int64
- 超出一个类型可表示的范围会导致环绕
- 无符号整数会使用0x为前缀的十六进制来表示
- 可以使用T()进行数据类型转换



Julia 的内置变量 `Sys.WORD_SIZE` 表明了目标系统是 32 位还是 64 位架构

2 类型与结构体 – 常用数值类型

浮点类型：用于表示小数

类型	精度	比特数
Float16	半(half)	16
Float32	单(single)	32
Float64	双(double)	64

```
julia> a = 5.2  
5.2
```

```
julia> typeof(a) #操作系统为64位  
Float64
```

```
julia> typeof(1e5) #操作系统为64位  
Float64
```

```
julia> typeof(5.2f0) #使用f则为32位  
Float32
```

```
julia> Float32(a)  
5.2f0
```

说明：

- 浮点数的默认类型取决于电脑系统是32位还是64位
- 浮点数可以用科学记数法表示
- 使用“f”替代“e”或Float32()可以得到Float32的浮点数
- 半精度一般采用软件模拟，性能较差，不推荐使用

2 类型与结构体 - 常用数值类型

特殊浮点数：正零、负零、正无穷、负无穷、NaN

[illegible]

Float16	Float32	Float64	名称	描述
Inf16	Inf32	Inf	正无穷	大于所有有限浮点数的值
-Inf16	-Inf32	-Inf	负无穷	小于所有有限浮点数的值
NaN16	NaN32	NaN	非数	不等于任何浮点数，甚至不等于自己

```
julia> typemax(Float64)
Inf
```

```
julia> typemin(Float64)
-Inf
```

```
julia> 0/0
NaN
```

julia> Inf/Inf
NaN

```
julia> 0 * Inf
NaN
```

```
julia> NaN + 1 #传染性
NaN
```

```
julia> Inf + 1 #传染性
Inf
```

```
julia> NaN == NaN #NaN不等于任何值
false
```

```
julia> Inf == Inf
true
```

说明:

- Inf和NaN具有传染性
- NaN甚至不等于它自己

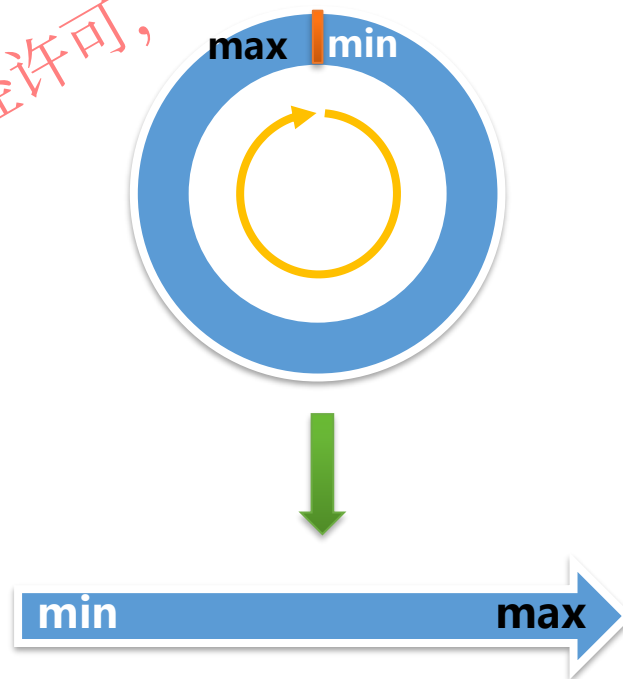
2 类型与结构体 - 常用数值类型

任意精度计算: BigInt 和 BigFloat 分别提供了任意精度的整数和浮点数

[illegible]

可使用以下语句对数值进行扩位:

- BigInt(x) 或 BigFloat(x)
- big""

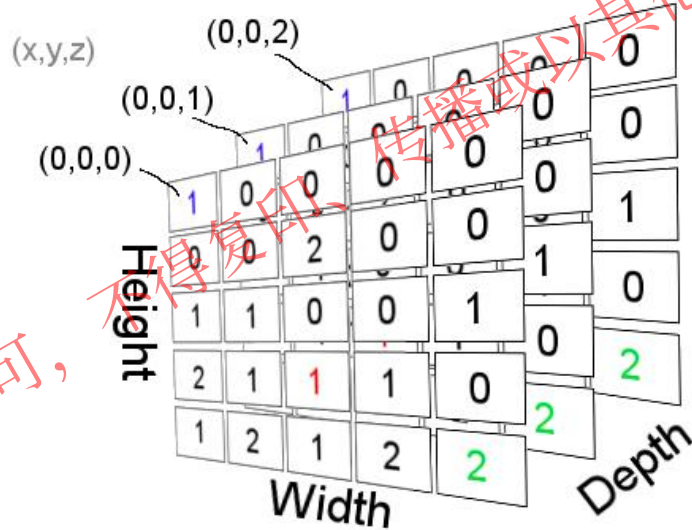


2 类型与结构体 - 数组类型

数组：用于容纳同种类型的元素。

Julia支持多维数组:

- 1维数组：向量，如： $\begin{bmatrix} 1, & 2, & 3 \end{bmatrix}$
- 2维数组：矩阵，如： $\begin{bmatrix} 1 & 2 & 3 \\ 2 & 1 & 2 \\ 3 & 2 & 1 \end{bmatrix}$
-
- n维数组



容器类型	元素类型	元素顺序	元素可变性	形式
元组 (Tuple)	任意类型	有序	不可变	(a,b,c)
字典 (Dict)	任意类型的键值对	无序	可变	Dict(a =>n, b =>m)
集合 (Set)	同种类型 (互不相等)	无序	可变	Set((a,b,c))
数组 (Array)	相同类型 (允许不同类型)	有序	可变	[a, b, c]

四种容器对比

2 类型与结构体 – 数组类型

数组：用于容纳同种类型的元素。

向量常用构造形式：

1. `[a,b,c,d]`

```
julia> a = [1,2,3]
3-element Vector{Int64}:
 1
 2
 3
```

2. `[a;b;c;d]`

```
julia> b = [1;2;3]
3-element Vector{Int64}:
 1
 2
 3
```

```
julia> c = Float64[1;2] #构造数组同时约束数值类型
2-element Vector{Float64}:
 1.0
 2.0
```

说明：

- 两种表示形式结果一致，但是分号表示为拼接
- Julia 的一维向量为列向量

2 类型与结构体 – 数组类型

数组：用于容纳同种类型的元素。

矩阵常用构造形式：

1. [a b;c d]

```
julia> a = [1 2 3]
1×3 Matrix{Int64}:
 1  2  3
```

```
julia> a1=[1;; 2;; 3;; 4]
1×4 Matrix{Int64}:
 1  2  3  4
```

```
julia> b = [1 2;3 4]
2×2 Matrix{Int64}:
 1  2
 3  4
```

2. [a b
c d]

```
julia> c = [1 2
            3 4]
2×2 Matrix{Int64}:
 1  2
 3  4
```

说明：

- 分号、换行符表示纵向(行, 第一维度)拼接
- 空格、制表符表示横向(列, 第二维度)拼接
- 使用双分号进行水平连接时, 先纵向拼接再横向拼接
- 空格、制表符的优先级高于分号

2 类型与结构体 – 数组类型

数组：用于容纳同种类型的元素。

构造列表：

1. `start:step:stop`

- `step`可缺省，缺省时`step=1`

2.

`range( start, stop, length, step)`

`range(start; stop, length, step)`

`range(start, stop; length, step)`

`range(start, stop, length::Integer)`

```
julia> 1:2
```

```
1:2
```

```
julia> e=range(1,3,3)
```

```
1.0:1.0:3.0
```

```
julia> f=range(1,step=1,length=2)
```

```
1:1:2
```

列表构造向量

1. `[a:b...]`或`[a:b;]`

```
julia> a = [1:3...]
```

```
3-element Vector{Int64}:
```

```
1
```

```
2
```

```
3
```

2. `collect(r)`

- `r`为列表

```
julia> b = collect(1:2:3)
```

```
2-element Vector{Int64}:
```

```
1
```

```
3
```

说明：

- 列表的运算方式与向量一致
- 列表可以避免不必要的内存分配，效率更高

2 类型与结构体 – 数组类型

数组拼接

纵向拼接：使用**分号、换行符**

```
julia> [1:2, 4:5] # 这里有一个逗号，因此并不会发生矩阵的拼接。  
range 本身就是元素  
2-element Vector{UnitRange{Int64}}:  
 1:2  
 4:5
```

```
julia> [1:2; 4:5]  
4-element Vector{Int64}:  
 1  
 2  
 4  
 5
```

```
julia> [1:2  
        4:5]  
4-element Vector{Int64}:  
 1  
 2  
 4  
 5
```

横向拼接：使用**空格、制表符**

```
julia> [1:2 4:5 7:8]  
2×3 Matrix{Int64}:  
 1 4 7  
 2 5 8  
  
julia> [[1,2] [4,5] [7,8]]  
2×3 Matrix{Int64}:  
 1 4 7  
 2 5 8
```

```
julia> [1 2 3] # 数字可以被横向拼接  
1×3 Matrix{Int64}:  
 1 2 3
```

使用双分号进行水平连接时，先纵向拼接再横向拼接

```
julia> [1; 2;; 2; 3;; 3; 4;; 4; 5]  
2×4 Matrix{Int64}:  
 1 2 3 4  
 2 3 4 5
```


2 类型与结构体 – 数组类型

数组：特殊数组的构造

函数	描述
<code>Array{T}(undef, dims)</code>	一个没有初始化的密集Array
<code>Matrix{T}(I,m,n)</code>	m行n列的单位矩阵
<code>zeros(T, dims)</code>	每个元素均为0的Array
<code>ones(T, dims)</code>	每个元素均为1的Array
<code>true(dims)</code>	每个元素均为true的BitArray
<code>false(dims)</code>	每个元素均为false的BitArray
<code>rand(T, dims)</code>	一个随机Array，元素值是[0,1)半开区间中均匀分布且服从一阶独立分布
<code>randn(T,dims)</code>	一个随机 Array，元素为标准正态分布，服从独立同分布
<code>range(start, stop =stop, length = n)</code>	从start到stop的带有n个线性间隔元素的范围
<code>fill(x, dims)</code>	一个被值x填充的Array

说明：

- T为数据类型
- dims为数组维度

```
julia> a = Array{Float64}(undef,2,1)
2×1 Matrix{Float64}:
 6.9522955403409e-310
 6.95229553427217e-310
```

```
julia> using LinearAlgebra
julia> b = Matrix{Bool}(I,2,3)
2×3 Matrix{Bool}:
 1  0  0
 0  1  0
```

```
julia> c=true(1,3)
1×3 BitMatrix:
 1  1  1
```

```
julia> d=rand(1,3)
1×3 Matrix{Float64}:
 0.483502  0.491862  0.613609
```

```
julia> e=range(1,3,3)
1.0:1.0:3.0
```

```
julia> f=range(1,step=1,length=2)
1:1:2
```

```
julia> [f;] #还可以使用[f...]
2-element Vector{Int64}:
 1
 2
```

说明：

- `Array{T}(undef, dims)`可以完全确定数组类型，Array可以改为Vector和Matrix。

2 类型与结构体 – 数组类型

数组：特殊数组的构造

函数	描述
<code>fill!(A, x)</code>	用值x填充数组A
<code>reshape(A, dims)</code>	一个包含跟A相同数据但维度不同的数组
<code>copy(A)</code>	拷贝A
<code>similar(A, T, dims)</code>	一个与A具有相同类型(密集、稀疏等)的未初始化数组，但具有制定的元素类型和维度，T和dims可缺省
<code>reinterpret(T, A)</code>	与A具有相同二进制数据的数组，但元素类型为T

```
julia> a=[1,1,1]; b=fill!(Vector{Vector{Int}}(undef, 3), a)
```

```
3-element Vector{Vector{Int64}}:
```

```
[1, 1, 1]
```

```
[1, 1, 1]
```

```
[1, 1, 1]
```

```
julia> c= reshape(a,1,3)
```

```
1×3 Matrix{Int64}:
```

```
1 1 1
```

```
julia> reinterpret(Bool, 0xff)
```

```
true
```

```
julia> reinterpret(Bool, 0x00)
```

```
false
```

2 类型与结构体 – 数组类型

数组索引：Julia数组使用方括号[]进行索引和切片。

说明：

- `I_n` 可以为标量整数、整数数组或其他支持的索引类型
- 数组索引： `I_n` 均为为标量整数时，结果X为数组A中的单个元素
- 数组切片： `I_n` 不全为标量整数时，结果X为数组A中的一个区域数组

```
julia> A = reshape(collect(1:4),(2,2))
2×2 Matrix{Int64}:
 1  3
 2  4
julia> A[1,2];A[3] #数组元素位置从1开始; 支持线性索引
 3
julia> A[1,[1,2]] # [1,2]表示此维度中第1个和第2个量
2-element Vector{Int64}:
 1
 3
julia> A[[1],[1,2]]
1×2 Matrix{Int64}:
 1  3
```

```
julia> A[1,:] #使用:取维度中所有量
2-element Vector{Int64}:
 1
 3
julia> A[1,1:2] #1:2表示此维度中第1个到第2个量
2-element Vector{Int64}:
 1
 3
julia> A[end,2] #end表示此维度中最后1个量
 4
julia> A[[false,true],:] # [false,true]表示此维度中对应位置为true的量
1×2 Matrix{Int64}:
 2  4
julia> A[CartesianIndex(2, 1)] #CartesianIndex(2, 1) 表示使用笛卡尔坐标索引
 2
```

2 类型与结构体 – 数组类型

数组修改：使用索引与切片的方式改变数组的值。

```
julia> X = [1, 2, 3, 4]
2-element Vector{Int64}:
```

```
1
2
```

```
julia> X[1] = 3; #将X的第1个值改为3
```

```
julia> X
```

```
4-element Vector{Int64}:
```

```
3
2
3
4
```

```
julia> X[2:3] = [0,0];
```

```
julia> X
```

```
4-element Vector{Int64}:
```

```
3
0
0
4
```

```
julia> X = [1, 2]
```

```
2-element Vector{Int64}:
```

```
1
2
```

```
julia> Y = X
```

```
2-element Vector{Int64}:
```

```
1
2
```

```
julia> Y[1]=5 #X的值是否有变化?
```

```
5
```

```
julia> X #X的值也会跟随Y的值变化
```

```
2-element Vector{Int64}:
```

```
5
2
```

```
julia> pointer(X) == pointer(Y) #地址相同
true
```

```
julia> X = [1, 2]; Y = copy(X)
```

```
2-element Vector{Int64}:
```

```
1
2
```

```
julia> pointer(X) == pointer(Y) #地址不同
false
```

```
julia> X == Y #数值相同
```

```
true
```

```
julia> Y[1]=5 #X的值是否有变化?
```

```
5
```

```
julia> X #X的值不会跟随Y的值变化
```

```
2-element Vector{Int64}:
```

```
1
2
```

注意：

- Julia 数组在分配给另一个变量时不会被复制。在A = B之后，改变B的元素也会改变A的元素。

2 类型与结构体 – 数组类型

数组修改： Julia 不会在赋值语句中自动增长数组。

```
julia> a = rand(3,3)
3×3 Matrix{Float64}:
 0.317134  0.808967  0.912479
 0.241704  0.420218  0.245492
 0.00407314 0.447554  0.506671

julia> a[3,1] #索引第3行第1列的值
0.0040731381120721055

julia> a[4,1] = 3.4 #索引第4行第1列的值不存在
ERROR: BoundsError: attempt to access 3×3 Matrix{Float64} at
index [4, 1]
Stacktrace:
 [1] getindex(::Matrix{Float64}, ::Int64, ::Int64)
       @ Base .\array.jl:862
 [2] top-level scope
       @ REPL[3]:1
```

2 类型与结构体 – 数组类型

运算符

类型	表达式	名称	描述
一元运算符	+	一元加法	全等操作
	-	一元减法	将各元素取反
二元运算符	-	减	各元素相减
	+	加	各元素相减
	*	乘	数组乘法（线性代数）
	/	除	数组除法（线性代数）
	\	反向除法	数组反向除法（线性代数）
	^	幂	数组幂运算
比较操作符	==	相等	任意元素不等则为false
	!=	不相等	任意元素不等则为true
	≈	约等	任意元素不约等则为false
	≈/	不约等于	任意元素不约等则为true

```
julia> a=[1,2,3]; b=[2,3,4];

julia> a+b
3-element Vector{Int64}:
 3
 5
 7

julia> a/b
3×3 Matrix{Float64}:
0.0689655 0.103448 0.137931
0.137931 0.206897 0.275862
0.206897 0.310345 0.413793

julia> ans*b
3-element Vector{Float64}:
1.0
2.0
3.0

julia> c=[1+1e-10,2,3];a≈c
true
```

2 类型与结构体 – 数组类型

数组运算相关函数

类别	函数	功能描述
矩阵属性	norm	计算范数
	det	矩阵行列式
	rank	矩阵的秩
	tr	对角线元素之和
	cond	逆运算的条件数
	condskeel	相对条件数
	condeig	与特征值有关的条件数
	rref	简化的行阶梯形矩阵（Gauss-Jordan 消元法）
	subspace	两个子空间之间的角度
线性方程	\	对线性方程组 $Ax = B$ 求解 x
	/	对线性方程组 $xA = B$ 求解 x
	linsolve	对线性方程组求解
	inv	矩阵求逆
	pinv	Moore-Penrose 伪逆
	lsconv	存在已知协方差情况下的最小二乘解
	sylvester	求 Sylvester 方程 $AX + XB = C$ 的 X 解

类别	函数	功能描述
特征值与奇异值	eigvals	返回A的特征值
	eigen	计算 A 的特征值分解，返回一个特征分解对象 F
	svd	计算 A 的奇异值分解 (SVD) 并返回一个 SVD 对象。
	svdvals	按降序返回 A 的奇异值
	ordeig	拟三角矩阵的特征值
	polyeig	多项式特征值问题
	hessenberg	矩阵的 Hessenberg 形式
	schur	Schur 分解
	schur!	Schur 分解
	rsf2csf	将实数Schur形式变换为复数Schur形式
矩阵分解	cdf2rdf	拟三角矩阵的特征值
	lu	LU 矩阵分解
	ldlt	实对称三对角矩阵S的LDLt分解
	cholupdate	Cholesky 分解的秩 1 更新
	qrdelete	从 QR 分解中删除列或行
	qrinsert	将列或行插入 QR 分解
	qrupdate	QR 分解的秩 1 更新
	planerot	Givens 平面旋转

类别	函数	功能描述
矩阵运算	transpose	转置向量或矩阵
	adjoint	复共轭转置
	sqrt	矩阵平方根
	exp	矩阵指数运算
	log	矩阵对数运算
	kron	Kronecker 张量积
	cross,x	叉积
	dot,·	点积
矩阵结构	bandwidth	矩阵的上下带宽
	tril	矩阵的下三角形部分
	triu	矩阵的上三角形部分
	isbanded	确定矩阵是否在特定带宽范围内
	isdiag	确定矩阵是否为对角矩阵
	ishermitian	确定矩阵是 Hermitian 矩阵
	issymmetric	确定矩阵是对称矩阵
	istril	确定矩阵是否为下三角矩阵
	istriu	确定矩阵是否为上三角矩阵

具体函数使用详见《Syslab帮助文档》

2 类型与结构体 – 字符串类型

字符串构造

字符串字面量：由双引号或三重双引号分隔。

```
julia> str = "Hello, world.\n"  
"Hello, world.\n"
```

```
julia> """Contains "quote" characters"""  
"Contains \"quote\" characters"
```

使用内置函数构造和操作字符串。

```
julia> s = join(["Hello", "World"], ", ") # "Hello, World "  
"Hello, World"
```

```
julia> s = string("The answer is ", 42) # "The answer is 42 "  
The answer is 42
```

字符串拼接

使用 string 函数可以将多个值拼接成一个字符串。

```
julia> greet = "Hello"  
"Hello"
```

```
julia> whom = "world"  
"world"
```

```
julia> string(greet, ", ", whom, ".\n")  
"Hello, world.\n"
```

使用 * 运算符连接多个字符串。

```
julia> greet * ", " * whom * ".\n"  
"Hello, world.\n"
```


2 类型与结构体 – 字符串类型

字符串比较

可以使用标准的比较操作符按照字典顺序比较字符串

```
julia> "abracadabra" < "xylophone"  
true
```

```
julia> "abracadabra" == "xylophone"  
false
```

```
julia> "Hello, world." != "Goodbye, world."  
true
```

```
julia> "1 + 2 = 3" == "1 + 2 = $(1 + 2)"  
true
```

字符串索引

你可以使用 **findfirst** 与 **findlast** 函数搜索特定字符的索引

```
julia> findfirst(isequal('o'), "xylophone")  
4
```

```
julia> findlast(isequal('o'), "xylophone")  
7
```

```
julia> findfirst(isequal('z'), "xylophone")
```

带上第三个参数用 **findnext** 与 **findprev** 函数来在给定偏移量处搜索字符

```
julia> findnext(isequal('o'), "xylophone", 1)  
4
```

```
julia> findnext(isequal('o'), "xylophone", 5)  
7
```

```
julia> findprev(isequal('o'), "xylophone", 5)  
4
```

```
julia> findnext(isequal('o'), "xylophone", 8)
```

2 类型与结构体 – 字符串类型

字符串匹配

用 **occursin** 函数检查在字符串中某子字符串可否找到，支持正则表达式查找。

```
julia> occursin("world", "Hello, world.")
true

julia> occursin("a", "Xylophon")
false

julia> occursin('o', "Xylophon")
true

julia> occursin(r"^\s*(?:#|$)", "not a comment")
false

julia> occursin(r"^\s*(?:#|$)", "# a comment")
true
```

要捕获正则表达式匹配的信息，可以用 **match** 函数。

```
julia> m = match(r"(a|b)(c)?(d)", "ad")
RegexMatch("ad", 1="a", 2=nothing, 3="d")

julia> m.match
"ad"

julia> m.captures
3-element Vector{Union{Nothing, SubString{String}}}:
"a"
nothing
"d "

julia> m.offset
1

julia> m.offsets
3-element Vector{Int64}:
1
0
2
```

2 类型与结构体 – 字符串类型

其他数据类型转为字符串

string 函数：将任意类型转换为字符串。

```
julia> num = 42;  
julia> str = string(num)  
"42"
```

字符串插值：使用 \$ 符号将任意类型插入到字符串中。

```
julia> num = 42;  
julia> str = "$num"  
42
```

将字符串转换为其他数据类型

parse 函数：将字符串解析为指定的类型。

```
julia> str = "42";  
julia> num = parse{Int, str}  
42
```

```
julia> str = "3.14";  
julia> num = parse{Float64, str}  
3.14
```

tryparse 函数：类似于 parse，但如果转换失败，不会抛出错误，而是返回 nothing。

```
julia> str = "42";  
julia> num = tryparse{Int, str}  
42
```

```
julia> invalid_str = "abc";  
julia> invalid_num = tryparse{Int, invalid_str}
```

2 类型与结构体 – 字符串类型

格式化

@sprintf 宏可以将格式化的字符串存储在变量中。

```
using Printf                                     @sprintf.jl

# 格式化整数
num = 42
formatted_str = @sprintf("Integer: %d", num)
println(formatted_str) # 输出: "Integer: 42"

# 格式化浮点数
flt = 3.14159
formatted_str = @sprintf("Float: %.2f", flt)
println(formatted_str) # 输出: "Float: 3.14"

# 格式化字符串
name = "Alice"
formatted_str = @sprintf("Name: %s", name)
println(formatted_str) # 输出: "Name: Alice"
```

```
julia> 正在运行 @sprintf.jl
Integer: 42
Float: 3.14
Name: Alice
```

@printf 宏提供了类似于 C 语言的格式化输出功能。

```
using Printf                                     @printf.jl

# 格式化整数
num = 42
@printf("Integer: %d\n", num)

# 格式化浮点数
flt = 3.14159
@printf("Float: %.2f\n", flt)

# 格式化字符串
name = "Alice"
@printf("Name: %s\n", name)
```

```
julia> 正在运行 @printf.jl
Integer: 42
Float: 3.14
Name: Alice
```

2 类型与结构体 – 枚举类型

@enum EnumName[::BaseType] value1[=x] value2[=y]

创建一个 EnumName{BaseType} 类型，名称为 EnumName，可选 BaseType 为成员值类型，BaseType 默认为 Int32，必须是 Integer 的子类型。枚举成员值为 value1和 value2，可选赋值分别为 x 和 y。

```
julia> @enum Fruit apple=1 orange=2 banana=3
```

```
julia> f(x::Fruit) = "I'm a Fruit with value: $(Int(x))"  
f (generic function with 1 method)
```

```
julia> f(apple)  
"I'm a Fruit with value: 1"
```

创建一个枚举值

```
julia> my_fruit = Fruit(1)  
apple::Fruit = 1
```

也可以在 begin 块中指定值:

```
julia> @enum Fruit begin  
    apple=1  
    orange=2  
    banana=3  
end
```

检查枚举值

```
julia> if my_fruit == apple  
    println("It's an apple!")  
else  
    println("It's not an apple.")  
end  
It's an apple!
```

列出枚举的所有实例使用 instances

```
julia> instances(Fruit)  
(apple, orange, banana)
```

可以从枚举实例构造符号:

```
julia> Symbol(apple)  
:apple
```

2 类型与结构体 – 结构体

struct

Julia 中最常用的类型是 struct，它被指定为一个名称和一组字段。使用 struct 关键字定义。

```
struct Point
    x
    y
end
```

字段可以有类型限制，没有类型限制的字段默认为 Any，可以保存任何类型的值：

```
struct Point{X}
    x
    y::Float64
end
```

结构体还可以通过 <: 语法声明抽象超类型：

```
struct Point <: AbstractPoint
    x
    y
end
```

在默认情况下，struct 是**不可变的**；构造之后不能修改这些类型之一的实例。可以使用 mutable struct 来声明实例可以修改的类型。

2 类型与结构体 – 结构体

struct

类似于其它编程语言中的 struct, class, Julia 可以用 struct 或者 mutable struct 定义自己的复合数据类型:

```
julia> struct Foo
        bar
        baz::Int
        qux::Float64
    end
```

Foo 类型的新对象是通过将 Foo 类型的对象像函数一样应用于其字段的值来创建的, 也被称为构造函数。实际上, Julia 会自动生成两个构造函数。其中一个接受任何参数, 并调用 Convert 将它们转换为对应的字段类型, 如果 Convert 失败则会报错, 另一个接受与字段类型完全匹配的参数。

```
julia> foo1 = Foo("Hello, world.", 23, 1.5) # 该构造函数接受与字段类型完全匹配的参数
Foo("Hello, world.", 23, 1.5)

julia> foo2 = Foo("Hello, world.", 23.0, 1) # 该构造函数调用 Convert 将对应的参数转换为对应的字段类型
Foo("Hello, world.", 23, 1.0)

julia> foo3 = Foo("Hello, world.", 23.5, 1) # 该构造函数调用 Convert 将 baz 字段转换为 Int 类型失败, 报错
ERROR: InexactError: Int64(23.5)
```

2 类型与结构体 – 结构体

struct

用 struct 声明的复合对象是不可变的，在构造之后不能修改它们。使用不可变结构体能改善程序的性能，并且它们是线程安全的，因为它们在跨线程使用时不需要同步。

```
julia> foo1.bar = "hello"  
ERROR: setfield!: immutable struct of type Foo cannot be changed
```

要得到字段名列表可以使用 fieldnames 函数：

```
julia> fieldnames(Foo)  
(:bar, :baz, :qux)
```

要访问某字段的值可以用点操作：

```
julia> foo1.bar  
"Hello, world."
```

```
julia> foo1.baz  
23
```

```
julia> foo1.qux  
1.5
```


2 类型与结构体 – 结构体

mutable struct

如果使用 mutable struct 声明复合类型，那么可以修改它的实例：

```
julia> mutable struct Bar
    baz
    qux::Float64
end

julia> bar = Bar("Hello", 1.5);

julia> bar.qux = 2.0
2.0
```

如果已知可变结构的一个或多个字段是不可变的，那么可以使用 const 来声明这些字段：

```
julia> mutable struct Baz
    a::Int
    const b::Float64
end

julia> baz = Baz(1, 1.5);

julia> baz.a = 2
2

julia> baz.b = 2.0
ERROR: setfield!: const field .b of type Baz cannot be changed
```

2 类型与结构体 – 结构体

Base.@kwdef

julia语言中结构体无法直接提供默认值，需要通过 Base.@kwdef 宏设置默认值：

```
julia> Base.@kwdef struct Person
    name::String = "Tom"
    age::Int = 30
    gender::String = "Unknown"
end
```

```
julia> p = Person();
```

```
julia> println("Name: ", p.name, "\nAge: ", p.age, "\nGender: ",
p.gender)
Name: Tom
Age: 30
Gender: Unknown
```

创建一个 Person 对象时，通过指定变量初始化结构体值。

```
julia> p = Person(name="Alice", age=18);
```

```
julia> println("Name: ", p.name, "\nAge: ", p.age, "\nGender: ",
p.gender)
Name: Alice
Age: 18
Gender: Unknown
```

03 多重派发

@苏州同元软控信息技术有限公司版权所有，未经许可，不得复印、传播或以其他方式使用
版权所有，侵权必究

3 多重派发

多重派发

从一个函数的多种方法选择一个方法来执行的过程称为派发(dispatch)。Julia 在派发问题上与其它的面向对象语言有很大区别，Julia 是根据参数个数不同以及所有参数类型的不同来选择方法的，这称为多重派发(multiple dispatch)。

```
julia> f(x::Int) = "This is an Int: $(x)";  
julia> f(x::Float64) = "This is a Float: $(x)";  
julia> f(x::Any) = "This is a generic fallback";  
julia> f(10)  
"This is an Int: 10 "  
julia> f("cat")  
"This is a generic fallback"
```

可以在已有的 f 函数基础上进行扩充：

```
julia> f(x::String) = "This is an String: $(x)";  
julia> f(x::Number) = "This is a Number: $(x)";  
julia> f("cat")  
"This is an String: cat"  
julia> f(1 + 3im)  
"This is a Number: 1 + 3im"
```

04 参数化类型

©苏州同元软控信息技术有限公司版权所有，未经许可，不得复印、传播或以其他方式使用

@苏州同元软控信息技术有限公司
版权所有，侵权必究

4 参数化类型 - 参数复合类型

参数化 (parametric) 是 Julia 类型系统中的一个非常重要且强大的特性。它允许类型自身包含参数，并使得一个这样的类型就可以代表整个类型族群。

比如 `Array` 就是参数复合类型，它可以代表的类型的数量是无限的，`Array{T}` 我们可以使用任何一个类型的名称替换 `T`，从而表示一种具体的类型，比如：`Array{String}`，`Array{Int}` 等等。`Array` 本身也是一个有效的类型对象，而它所有的实例都作为它的子类型：

```
julia> Array{String} <: Array
true
julia> Array{Int} <: Array
true
```

需要注意的是，虽然 `Float64 <: Real`，但是**没有** `Array{Float64} < Array{Real}`，换成类型理论说法，Julia 的类型参数是不变的，而不是协变的（或甚至是逆变的）。

由于 `Array{Float64}` 不是 `Array{Real}` 的子类型，因此下面的 `mynorm1` 不适用于类型为 `Array{Float64}` 的参数：

```
julia> function mynorm1(p::Array{Real})
    sqrt(p)
end
```

定义函数接受 `Array{T}` 类型，
其中 `T` 是 `Real` 的子类型



```
julia> function mynorm2(p::Array{<:Real})
    sqrt(p)
end
```

4 参数化类型 – 参数复合类型

自定义一个参数复合类型 `Point`，类型参数在类型名称后引入，用大括号括起来，定义参数化类型时并不一定必须将字段类型限制为参数类型 `T`。你可以自由选择是否使用参数类型 `T`，或者将字段定义为具体类型或其他参数类型。在示例当中，我们将字段类型都限制为 `T`：

```
julia> struct Point{T}
    x::T
    y::T
end
```

由于 `Point{Float64}` 类型等价于在 `Point` 声明时用 `Float64` 替换 `T` 得到的具体类型，它可以相应地作为构造函数使用：

```
julia> p = Point{Float64}(1.0, 2.0)
Point{Float64}(1.0, 2.0)
```

```
julia> typeof(p)
Point{Float64}
```

对于默认的构造函数，必须为每个字段提供一个参数：

```
julia> Point{Float64}(1.0)
ERROR: MethodError: no method matching Point{Float64}(::Float64)
[...]
```

参数类型只生成一个默认的构造函数，因为它无法覆盖。这个构造函数接受任何参数并将它们转换为字段的类型。

4 参数化类型 – 参数复合类型

许多情况下，没有必要提供想要构造的 Point 对象的类型，因为构造函数调用参数的类型已经隐式地提供了类型信息。因此，你也可以将 Point 本身用作构造函数，前提是参数类型 T 的隐含值是明确的：

```
julia> p1 = Point(1.0,2.0)
Point{Float64}(1.0, 2.0)
```

```
julia> typeof(p1)
Point{Float64}
```

```
julia> p2 = Point(1,2)
Point{Int64}(1, 2)
```

```
julia> typeof(p2)
Point{Int64}
```

在 Point 的例子中，当且仅当 Point 的两个参数类型相同时，T 的类型才确实是隐含的。如果不是这种情况，构造函数将失败并出现 MethodError：

```
julia> Point(1,2.5)
ERROR: MethodError: no method matching Point{::Int64, ::Float64}
Closest candidates are:
  Point{::T, !Matched::T} where T at none:2
```


4 参数化类型 – 参数抽象类型

参数抽象类型声明使用 `abstract type`，定义一个参数抽象类型 `Pointy`：

```
julia> abstract type Pointy{T} end
```

在此声明中，对于每个类型 `T`，`Pointy{T}` 都是不同的抽象类型。与参数复合类型一样，每个此类型的实例都是 `Pointy` 的子类型：

```
julia> Pointy{Int64} <: Pointy
true
```

```
julia> Pointy{1} <: Pointy
true
```

参数抽象类型是不变的，就像参数复合类型：

```
julia> Pointy{Float64} <: Pointy{Real}
false
```

```
julia> Pointy{Float64} <: Pointy{<:Real}
true
```

4 参数化类型 – 参数抽象类型

正如之前的普通抽象类型用于在具体类型上创建实用的类型层次结构一样，参数抽象类型在参数复合类型上具有相同的用途。例如，我们可以将 `Point{T}` 声明为 `Pointy{T}` 的子类型，如下所示：

```
julia> struct Point{T} <: Pointy{T}
    x::T
    y::T
end
```

鉴于此类声明，对每个 `T`，都有 `Point{T}` 是 `Pointy{T}` 的子类型。

```
julia> Point{Float64} <: Pointy{Float64}
true
julia> Point{Real} <: Pointy{Real}
true
```

下面的关系依然不变：

```
julia> Point{Float64} <: Pointy{Real}
false
julia> Point{Float64} <: Pointy{<:Real}
true
```

4 参数化类型 – 参数抽象类型

参数抽象类型（比如 Pointy）的用途是什么？接下来再定义一个参数复合类型，用来表示在对角线上的点：

```
julia> struct DiagPoint{T} <: Pointy{T}
    x::T
end
```

如果希望有一个方法 calculate，计算更靠近 x 轴的点到 x 轴的距离：

```
julia> calculate(a::Pointy, b::Pointy) = abs(a.x) - abs(b.x) > 0 ? b.x : a.x
calculate (generic function with 1 method)
julia> calculate(Point(1,2), DiagPoint(2))
1
julia> calculate(Point(1,2), Point(0, 1))
0
julia> calculate(DiagPoint(3), DiagPoint(2))
2
```

4 参数化类型 – 参数原始类型

在Julia中，原始类型（Primitive Types）是最基本的、不可分割的数据类型，它们通常直接映射到底层机器表示。原始类型的定义和操作是 Julia 语言的基础，用于构建更复杂的数据结构和类型。原始类型通常是由系统预定义的，用户可以在其上扩展和操作。

以下是一些常见的原始类型：

- Int8, Int16, Int32, Int64, Int128：有符号整数类型，分别占用 8, 16, 32, 64, 128 位。
- UInt8, UInt16, UInt32, UInt64, UInt128：无符号整数类型，分别占用 8, 16, 32, 64, 128 位。
- Float16, Float32, Float64：浮点数类型，分别占用 16, 32, 64 位。
- Char：字符类型，通常占用 32 位。
- Bool：布尔类型，占用 8 位，取值为 true 或 false。

在Julia中，用户也可以定义自己的原始类型。定义原始类型的语法如下：

```
primitive type TypeName <: SuperType bits end
```

其中 TypeName 是要定义的类型名，SuperType 是其父类型，bits 是类型占用的位数。

4 参数化类型 – 参数原始类型

以下是一个更详细的示例，定义一个 16 位的原始类型来表示一个自定义的标识符（Identifier）：

Identifier.jl

```
# 定义一个 16 位的原始类型 Identifier
primitive type Identifier 16 end

# 定义一个函数来创建 Identifier 实例
function Identifier(value::UInt16)
    return reinterpret(Identifier, value)
end

# 定义函数来获取 Identifier 的原始值
function value(id::Identifier)
    return reinterpret(UInt16, id)
end
```

test.jl

```
include("Identifier.jl")

# 创建一个 Identifier 实例
id = Identifier(0x1234)

# 获取 Identifier 的原始值
println("Identifier value: ", value(id))
```



```
julia> 正在运行 test.jl
Identifier value: 4660
```

05 常用的宏

@苏州同元软控信息技术有限公司版权所有，未经许可，不得复印、传播或以其他方式使用
版权所有，侵权必究

5 常用的宏

@assert

@assert 是一个用于断言 (assertions) 的宏。它用于在代码中进行条件检查，如果条件不满足，则会抛出一个错误。@assert 通常用于调试和验证程序的正确性。

```
@assert condition [message]
```

- condition 是一个布尔表达式，如果其值为 false，则会触发断言错误。
- message 是一个可选的参数，用于提供错误信息，帮助调试。

以下是一些使用 @assert 的示例：

简单的断言

```
x = 5
@assert x > 0 "x 必须为正数"
```

没有提供错误信息

```
z = -1
@assert z >= 0
```

提供错误信息

```
@assert z >= 0 "z 必须非负数"
```



调试控制台	终端	注释
<pre>julia> x = 5 5</pre>		
<pre>julia> @assert x > 0 "x 必须为正数"</pre>		
<pre>julia> z = -1 -1</pre>		
<pre>julia> @assert z >= 0 ERROR: AssertionError: z >= 0 Stacktrace: [1] top-level scope @ REPL[79]:1</pre>		
<pre>julia> @assert z >= 0 "z 必须非负数" ERROR: AssertionError: z 必须非负数 Stacktrace: [1] top-level scope @ REPL[80]:1</pre>		

5 常用的宏

@show

@show 用于调试和检查变量的值。它会打印出表达式及其结果，这对快速检查代码中的变量状态非常有帮助。

```
@show expression
```

- expression 是你想要检查的表达式或变量。

以下是一些使用 @show 的示例：

show.jl

```
# 定义一些变量
a = 10

# 使用 @show 打印变量的值
@show a

# 计算并打印表达式的结果
result = a + 5
@show result

# 在函数中使用 @show
function add_and_show(x, y)
    result = x + y
    @show result # 打印 result 的值
    return result
end

sum = add_and_show(5, 7)
```



调试控制台 终端 注释

```
julia> 正在运行 show.jl
a = 10
result = 15
result = 12
12
```


5 常用的宏

@info

@info 是一个用于记录信息级别日志的宏。它是 Julia 内置的日志记录机制的一部分，提供了一种标准化的方式来记录和管理。

```
@info "message" [key1=value1, key2=value2, ...]
```

- "message" 是你想要记录的日志消息。
- [key1=value1, key2=value2, ...] 是可选的键值对，用于提供额外的上下文信息。

以下是一些使用 @info 的示例：

```
# 基本信息日志
@info "Starting the process"

# 带有变量的日志
x = 42
@info "The value of x is" x

# 带有上下文信息的日志
y = 100
@info "Calculating sum" value1 = x value2 = y result = (x + y)

# 在函数中使用 @info
function add_and_log(a, b)
    result = a + b
    @info "Adding numbers" a = a b = b result = result
    return result
end
sum = add_and_log(5, 7)
```



```
调试控制台 终端 注释
julia> 正在运行 info.jl
[ Info: Starting the process
[ Info: The value of x is
      x = 42
[ Info: Calculating sum
      value1 = 42
      value2 = 100
      result = 142
[ Info: Adding numbers
      a = 5
      b = 7
      result = 12
12
```

5 常用的宏

@which

@which 用于查找某个函数或方法的定义位置。它有助于理解函数的来源和它们的具体实现，特别是在调试和代码阅读过程中。

```
@which function_name(arguments...)
```

- function_name 是你想要查找的函数名称。
- arguments... 是该函数的参数，用于匹配特定的方法签名。

以下是一些使用 @which 的示例：

```
# 查找标准库函数的定义  
@which sqrt(4)
```

```
# 定义一个简单的函数  
function add(a, b)  
    return a + b  
end  
  
# 使用 @which 查找函数定义  
@which add(1, 2)
```

```
julia> @which sqrt(4)  
sqrt(x::Real)  
@ Base.Math math.jl:1488
```

```
julia> function add(a, b)  
    return a + b  
end  
add (generic function with 1 method)  
  
julia> @which add(1, 2)  
add(a, b)  
@ Main REPL[84]:1
```

5 常用的宏

@kwdef

Base.@kwdef 可以为结构体自动生成带关键字参数的构造函数。这个宏简化了定义结构体的过程，特别是在结构体包含多个字段时。

```
Base.@kwdef struct StructName
    field1::Type1
    field2::Type2 = default_value2
    field3::Type3 = default_value3
end
```

- StructName 是结构体的名称。
- field1, field2, field3 是结构体的字段名称。
- Type1, Type2, Type3 是字段的类型。
- default_value2, default_value3 是字段的默认值。

以下是使用 Base.@kwdef 的示例：

```
using Base: @kwdef

@kwdef struct Person
    name::String
    age::Int = 30
end

p1 = Person(name="Alice")
p2 = Person(name="Bob", age=25)
```

类型声明：字段类型声明是可选的，但建议提供，以便更好地进行类型检查和代码优化。

默认值：如果字段没有默认值，必须在创建实例时提供该字段的值。



```
julia> p1 = Person(name="Alice")
Person("Alice", 30)

julia> p2 = Person(name="Bob", age=25)
Person("Bob", 25)
```

5 常用的宏

@_FILE__

@_FILE__ 是一个内置的宏，它在代码中返回当前文件的路径。当你的代码涉及文件操作、日志记录或调试时，这个宏非常有用。

@_FILE__

- @_FILE__ 返回当前文件的绝对路径，类型为 String。

以下是一些使用 @_FILE__ 的示例：

- 例1：打印当前文件路径

CurrentFile.jl

```
println("Current file path: ", @_FILE__)
```

- 例2：创建日志文件

create_log_fil.jl

```
function create_log_file()
    log_file = joinpath(dirname(@_FILE__), "logfile.txt")
    open(log_file, "w") do file
        write(file, "Log entry")
    end
    println("Log file created at: ", log_file)
end

create_log_file()
```

julia> 正在运行 currentFile.jl
Current file path: c:\Users\TR\Desktop\Penpen\M\julia Basic Grammar\examples\03_函数基础\05_常用的宏\06_@_FILE__\currentFile.jl

logfile.txt x
1 Log entry

调试控制台 终端 注释 + v Julia REPL (v1.9.3) ❏ ❏ ... ^ x

julia> 正在运行 create_log_file.jl
Log file created at: c:\Users\TR\Desktop\Penpen\M\julia Basic Grammar\examples\03_函数基础\05_常用的宏\06_@_FILE__\example2\logfile.txt

5 常用的宏

@_DIR_

@_DIR_ 是一个内置的宏，它返回当前文件所在的目录路径。这在处理相对文件路径、模块资源文件定位、日志记录等方面非常有用。

@_DIR_

- @_DIR_ 返回当前文件的目录路径，类型为 String。

以下是一些使用 @_DIR_ 的示例：

- 例1：打印当前文件所在的目录路径

CurrentDir.jl

```
println("Current directory: ", @_DIR_)
```

- 例2：加载相对路径的文件

```
function load_data_file()
    data_file = joinpath(@_DIR_, "data.txt")
    open(data_file, "r") do file
        for line in eachline(file)
            println(line)
        end
    end
end

load_data_file()
```

julia> 正在运行 CurrentDir.jl

Current directory: c:\Users\TR\Desktop\Penpen\M\julia Basic Grammar\examples\03_函数基础\05_常用的宏\07__DIR__

data.txt ×

```
1 data.txt
2
```

调试控制台 终端 注释 + ▾ Julia REPL (v1.9.3) □ □ ... ^ ×

julia> 正在运行 load_data_file.jl
data.txt

5 常用的宏

@view

@view 是一个内置的宏，用于创建数组的视图（view）。视图是对数组的子集的引用，而不是创建新的数组副本。使用视图可以提高内存和计算效率，特别是在处理大数组时，因为它避免了数据的复制。

```
@view array[slice]
```

- array 是你要创建视图的数组。
- slice 是数组的切片或索引范围。

以下是使用 @view 的示例：

view.jl

```
B = [1 2 3; 4 5 6; 7 8 9]
```

```
v = @view B[1:2, 2:3]
```

```
println(v) # 输出: [2 3; 5 6]
```

```
# 修改视图中的值
```

```
v[1, 1] = 20
```

```
println(B)
```

在这个示例中，v 是二维数组 B 的视图，修改 v 中的值会直接影响 B。



```
julia> 正在运行 view.jl
```

```
[2 3; 5 6]
```

```
[1 20 3; 4 5 6; 7 8 9]
```

5 常用的宏

@error

@error 是一个用于记录错误消息的宏，它属于 Julia 的日志记录功能的一部分。使用 @error 可以帮助你在程序运行时记录错误信息，方便调试和错误追踪。

```
@error "Error message" key1=value1 key2=value2 ...
```

- "Error message" 是你要记录的错误消息。
- key1=value1, key2=value2, ... 是可选的键值对参数，用于提供额外的上下文信息。

以下是一些使用 @error 的示例：

- 例1：记录简单的错误消息

```
@error "Something went wrong"
```

- 例2：记录带上下文信息的错误消息

```
x = 42  
y = "invalid input"  
@error "Failed to process input" x=x y=y
```

```
julia> @error "Something went wrong"  
[ Error: Something went wrong  
  @ Main REPL[93]:1
```

```
julia> x = 42  
42  
  
julia> y = "invalid input"  
"invalid input"  
  
julia> @error "Failed to process input" x = x y = y  
[ Error: Failed to process input  
  x = 42  
  y = "invalid input"  
  @ Main REPL[96]:1
```


5 常用的宏

@time

@time 是一个用于测量代码块执行时间和内存分配的宏。它可以帮助你了解代码的性能，尤其是在进行性能调优时。

```
@time expression
```

- expression 是你要测量执行时间和内存分配的代码块。

以下是一些使用 @time 的示例：

- 例1：测量简单表达式的执行时间

```
@time sum(1:1000000)
```

- 例2：在函数中使用 @time

```
function compute()  
    # 生成 1000x1000 的随机矩阵  
    @time A = rand(1000, 1000)  
  
    # 矩阵乘法  
    @time B = A * A  
    return B  
end  
  
R = compute();
```

```
julia> @time sum(1:1000000)  
0.000001 seconds  
500000500000
```

```
julia> 正在运行 compute.jl  
0.040508 seconds (2 allocations: 7.629 MiB, 90  
.07% gc time)  
0.021285 seconds (2 allocations: 7.629 MiB)
```


5 常用的宏

@less

@less 宏用于显示函数或表达式的源代码。它是由 InteractiveUtils 模块提供的，非常有助于理解和调试代码，尤其是当你需要查看某个函数的具体实现时。

```
@less function_name(args...)
```

- function_name(args...) 是你要查看源代码的函数以及其参数。

以下是一些使用 @less 的示例：

- 例1：查看内置函数的源代码

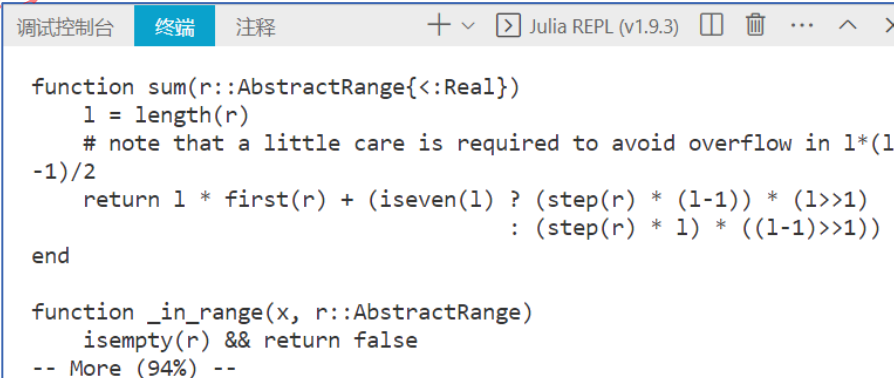
```
@less sum(1:10)
```

- 例2：查看自定义函数的源代码

less.jl

```
function add(a, b)
    return a + b
end
```

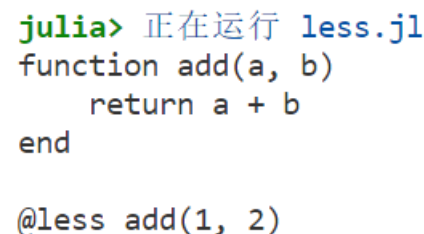
```
@less add(1, 2)
```



调试控制台 终端 注释 + Julia REPL (v1.9.3)

```
function sum(r::AbstractRange{<:Real})
    l = length(r)
    # note that a little care is required to avoid overflow in l*(l
-1)/2
    return l * first(r) + (iseven(l) ? (step(r) * (l-1)) * (l>>1)
                                     : (step(r) * l) * ((l-1)>>1))
end

function _in_range(x, r::AbstractRange)
    isempty(r) && return false
-- More (94%) --
```



```
julia> 正在运行 less.jl
function add(a, b)
    return a + b
end

@less add(1, 2)
```

Syslab Julia 模块化

Syslab 语言组

苏州同元软控信息技术有限公司

2024-08-08



TONGYUAN
Software and Control

目录

1. 包管理器

2. 模块

3. 包

4. 文档

01 包管理器

@苏州同元软控信息技术有限公司版权所有，未经许可，不得复印、传播或以其他方式使用

版权所有，侵权必究

1. 包管理器

Julia 包管理器 Pkg.jl

在 Julia REPL 中按下] 进入 REPL 的包管理模式，要返回 Julia REPL，请按 Ctrl+C 或退格键。在进入包管理器后，REPL的提示符会变为 “pkg >” 。

在包管理器中输入 “st” ，即可查看当前环境中已经安装的包：

```
(@v1.9) pkg> st  
Status `<path-to-julia-environment>`  
...
```

通过 “add” 命令安装包（此处以JSON包为例）：

```
(@v1.9) pkg> add JSON  
Updating registry at `<path-to-julia-registry>`  
Resolving package versions...  
...
```

通过 “update” 命令更新包：

```
(@v1.9) pkg> update JSON  
Updating registry at `<path-to-julia-registry>`  
...
```

1. 包管理器

Julia 包管理器 Pkg.jl

通过 “rm” 命令删除包:

```
(@v1.9) pkg> rm JSON
Updating registry at `<path-to-julia-registry>`
[682c06a0] - JSON v0.21.4
...
```

通过 “help” 命令获取有关可用命令及其用法的帮助信息:

```
(@v1.9) pkg> help
Welcome to the Pkg REPL-mode. To return to the julia> prompt, either press backspace when the input line
is empty or press Ctrl+C.
...
Synopsis

pkg> cmd [opts] [args]
...
Commands

activate: set the primary environment the package manager manipulates
add: add packages to project
```

1. 包管理器

Julia 包管理器 Pkg.jl

Julia 语言将包管理器独立发布为 Pkg 包，所有在 REPL 包管理模式中进行的操作都可以通过直接调用 Pkg 包的接口来完成。例如：可以通过 `Pkg.status()` 查看当前环境中已经安装的软件包，`Pkg.update()` 用于更新当前环境中的所有包。

```
julia> import Pkg

julia> Pkg.status()
Status `~/julia/environments/v1.9/Project.toml`
...

julia> Pkg.update()
Updating registry at `C:/Users/Public/TongYuan/julia\registries\General.toml`
Updating registry at `C:/Users/Public/TongYuan/julia\registries\Syslab.toml`
...
```

02 模块

©苏州同元软控信息技术有限公司版权所有，未经许可，不得复印、传播或以其他方式使用

@苏州同元软控信息技术有限公司
版权所有，侵权必究

2. 模块

模块的创建

- Julia 中的模块有助于将代码组织成分离的单元，Main 模块是 REPL 和全局命名空间的默认容器。所有在 REPL 中定义的变量、函数和模块默认都在 Main 模块中。
- 模块在语法上用 `module ... end` 来定义。每个模块都有该模块专属的命名空间，该命名空间的模块成员在模块外部默认不可见。
- `export` 关键字用于指定哪些名称（函数、类型、常量等）应该被导出，使得这些名称在该模块被导出时能够被引用到。
- 模块能够嵌套，在父模块中可以通过完整路径来调用子模块的名称（函数、类型、常量等）。

```
julia> module M
    export M_member
    M_member() = "Hello belong to M!"
end
Main.M

julia> module A
    module B
    export hello
    hello() = "Hello"
    end # end of B
    end # end of A
Main.A
```

2. 模块

模块的导入与导出

Julia 中的模块每个模块分属于不同的命名空间，如果要使用其他模块中的内容，需要使用 `import` 或者 `using` 导入模块。

`import` 和 `using` 的区别在于：

- 通过 `import` 导入模块：仅导入模块，但不将其方法导入当前命名空间，需要通过 `模块名.成员名` 访问模块的成员。
- 通过 `using` 导入模块：将模块和模块中的所有导出名字（在模块内通过 `export` 声明）导入当前命名空间，导出名字可以直接调用。

```
julia> module A
      module B
      export hello
      hello() = "Hello"
      end # end of B
      end # end of A
```

```
julia> import .A.B # 注意：导入当前命名空间的模块时需要在模块路径前方加上 . 以启用相对路径导入
```

```
julia> B.hello() # 此时无法直接调用 hello() 函数，因为import不会将模块的成员导入至当前命名空间
```

```
julia> using .A.B
```

```
julia> hello() # 此时可以直接调用 hello() 函数，因为using会把模块B的导出成员导入当前命名空间
```

```
julia> A.B.hello()
```

2. 模块

解决导入模块时的命名冲突

在将其他模块的定义方法导入至当前命名空间时，导入的方法名可能和当前命名空间的已有定义发生冲突，此时方法导入将会失败。

```
julia> foo() = "Hello I'm foo";

julia> module A
    export foo
    foo() = "Hello I'm foo from A"
end

julia> using .A
WARNING: using A.foo in module Main conflicts with an existing identifier.
```

此时可以使用 `as` 关键字对导入的名字进行重命名，也可以导入后通过 `模块名.成员名` 的形式显式的访问模块内的方法。

```
julia> using .A: foo as foo1

julia> foo1()
"Hello I belong to A"

julia> A.foo()
"Hello I belong to A"
```

2. 模块

解决导入模块时的命名冲突

同时导入多个模块时，若多个模块同时导出了相同的名字，此时Julia会发现导出的名字存在歧义，导入将会失败。

```
julia> module A
    export f
    f() = 1
end
julia> module B
    export f
    f() = 2
end
julia> using .A, .B # 同时导入模块 A 和 B
julia> f()
WARNING: both B and A export "f"; uses of it in module Main must be qualified
ERROR: UndefVarError: `f` not defined
```

此时可以通过 `as` 对导入的名称进行重命名，也可以导入后通过 `模块名.成员名` 的形式显式的访问模块内的方法。

```
julia> using .A: f as f; f()
1
julia> using .B: f as g; g()
2
julia> A.f()
1
julia> B.f()
2
```

03 包

©苏州同元软控信息技术有限公司版权所有，未经许可，不得复印、传播或以其他方式使用

@苏州同元软控信息技术有限公司
版权所有，侵权必究

3. 包

开发Julia包

Julia REPL 的包管理模式提供了一组用于开发 Julia 项目的命令。

```
(@v1.9) pkg> generate MyProject
Generating project MyProject:
  MyProject/Project.toml
  MyProject/src/MyProject.jl
```

```
shell> tree MyProject/
MyProject/
├── Project.toml
└── src
    └── MyProject.jl
```

1 directory, 2 files

```
shell> cat MyProject/src/MyProject.jl
module MyProject
```

```
  greet() = print("Hello World!")
```

```
end # module MyProject
```

通过 generate 命令在当前文件夹下生成名为 MyProject 的 Julia 项目模板。

项目文件夹包含一个项目描述文件 Project.toml，和一个与项目名称同名的julia源代码文件，该源代码文件包含一个简单的打印程序。

3. 包

激活不同的环境

包管理器 (Pkg) 允许用户创建和管理不同的项目环境。每个环境都有自己独立的一组依赖项，这样可以避免不同项目之间的依赖冲突。项目模板创建完毕后，即可在包管理模式下通过activate命令激活该项目对应的Julia环境。

```
(@v1.9) pkg> activate MyProject
Activating project at `<path-to-julia-environment>`

(MyProject) pkg> st
Project MyProject v0.1.0
Status `<path-to-julia-environment>`
```

此时在该环境安装的依赖项不会影响其他项目，不同的项目可以安装不同版本的包而不互相干扰。此时用户可以在该环境中安装Test依赖，用于项目的单元测试。

```
(MyProject) pkg> add Test
```

3. 包

开发项目的功能函数和相应的测试用例

此时用户可以在MyProject文件夹下的 src/MyProject.jl 定义用户函数，在 test/runtests.jl 下编写单元测试。

src/MyProject.jl

```
module MyProject
export my_sum
function my_sum(first::Integer, last::Integer)
    if first > last
        return 0
    end
    return div((first + last) * (last - first + 1), 2)
end
end# module MyProject
```

test/runtests.jl

```
using Test
using MyProject
# 定义测试用例
test_cases = [
    (1, 10), (0, 0), (-5, 5),
    (100, 200), (-100, -50), (1, 1),
    (50, 50), (-10, 0), (-50, -40),
    (999, 1000)
]
# 运行测试用例
for (first, last) in test_cases
    @test my_sum(first, last) == sum(first:last)
end
```


3. 包

运行项目的测试

功能函数和单元测试编写完毕后，即可通过包管理模式的 test 命令运行项目的单元测试。

```
(MyProject) pkg> test
Testing MyProject
...
Testing Running tests...
Testing MyProject tests passed
```

测试完毕后，会打印 “MyProject tests passed” 表示 MyProject 所有单元测试运行成功。

04 文档

©苏州同元软控信息技术有限公司版权所有，未经许可，不得复印、传播或以其他方式使用

@苏州同元软控信息技术有限公司
版权所有，侵权必究

4. 文档

为 Julia 模块添加文档

- 在 Julia 中，可以使用三重引号 (""") 和 Markdown 语法为函数添加文档字符串。文档字符串通常放在函数定义之前，Julia 的文档系统会自动识别并解析这些文档字符串。在 REPL 中使用 `?` 进入帮助模式可以查看函数的文档。

src/MyProject.jl

```
module MyProject
export my_sum

"""
## my_sum

my_sum(first, last)

计算 [first, last] 之间的所有整数的求和
"""
function my_sum(first::Integer, last::Integer)
    if first > last
        return 0
    end
    return div((first + last) * (last - first + 1), 2)
end
end
```

```
help?> my_sum
search: my_sum

my_sum
=====

my_sum(first, last)

计算 [first, last] 之间的所有整数的求和
```

Syslab Julia 核心风格

Syslab 语言组

苏州同元软控信息技术有限公司

2024-08-14



TONGYUAN
Software and Control

@苏州同元软控信息技术有限公司版权所有，侵权必究
未经许可，不得复印、传播或以其他方式使用

目录

1. 动态编译

2. 多重派发

3. 类型稳定

4. 向量化编程

01

动态编译

©苏州同元软控信息技术有限公司版权所有，未经许可，不得复印、传播或以其他方式使用

@苏州同元软控信息技术有限公司
版权所有，侵权必究

1. 从1+2=3开始：C

C 是一门**静态编译型**语言：代码的编译与执行分别在两个进程内完成



C 编译器是开发期依赖：

- 代码的编译发生开发侧
- 代码的执行发生在用户侧
(不需要 C 编译器即可使用)

严格来说，机器码才是可执行代码，但大部分时候为了可读性的考量，也可以使用汇编码来表达同样的涵义

1. 从1+2=3开始：Python

Python 是一门**动态解释型**语言：代码通过 Python 解释器执行



允许动态更新函数实现，验证数值结果

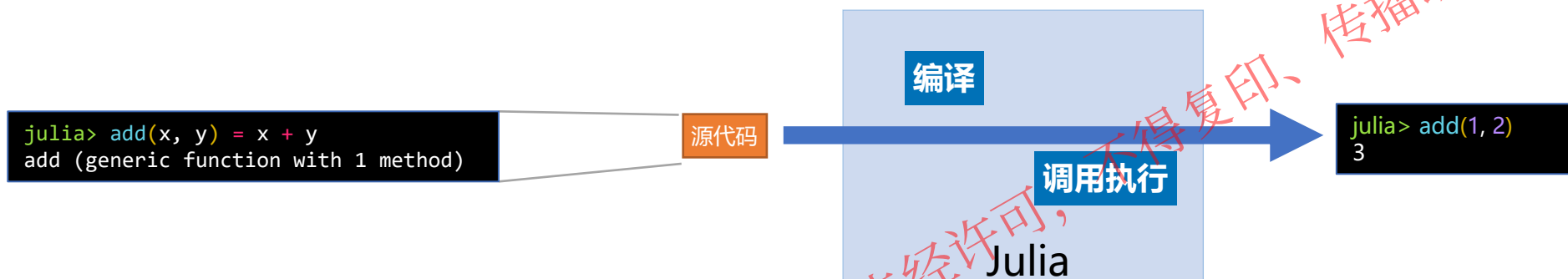
```
In [1]: def add(x, y):  
...:     return x + y  
...:  
  
In [2]: add(1, 2)  
Out[2]: 3  
  
In [3]: def add(x, y):  
...:     return x + y + 3  
...:  
...:  
  
In [4]: add(1, 2)  
Out[4]: 6
```

Python 解释器是一个运行时依赖：

- 代码的执行发生在用户侧
- 函数定义可以动态更新

1. 从1+2=3开始：Julia

Julia 是一门**动态编译型**语言：代码的编译与执行在同一个进程内完成



允许动态更新函数实现，验证数值结果

```
julia> add(x, y) = x + y
add (generic function with 1 method)

julia> add(1, 2)
3

julia> add(x, y) = x + y + 3
add (generic function with 1 method)

julia> add(1, 2)
6
```

Julia 编译器是一个运行时依赖：

- 代码的编译与执行均发生在用户侧
- 代码执行前进行按需代码编译
- 函数定义可以动态更新
(代码的编译结果会被自动更新)

1. 从1+2=3开始: Julia

尽管 Julia 编程的流程与 Python 类似, 但 Julia 是一门性能极高的编译型语言

$$\text{Kurtosis 峰值 } \text{Kurt}(X) = \frac{E[(X - \mu)^4]}{E[(X - \mu)^2]^2}$$

Julia

11.45ms

```
# Julia 的 kurtosis 实现
using Statistics

function kurtosis(X)
    mu = mean(X)
    n = length(X)

    tmp = (X .- mu).^2
    # 对 tmp 中的每个元素求平方并求和, 它等价于 sum(tmp.^2)
    # 但是会更高效率
    fourth_moment = sum(abs2, tmp) / n
    second_moment = sum(tmp) / n
    return fourth_moment / (second_moment^2)
end
```

Python

1739ms

```
# 使用 Python numpy 的 kurtosis 实现
import numpy as np

def kurtosis(X):
    mu = np.mean(X)
    n = len(X)

    tmp = (X - mu)**2
    fourth_moment = np.sum(tmp**2) / n
    second_moment = np.sum(tmp) / n
    return fourth_moment / (second_moment**2)
```

Julia

(for 循环优化版)

5.012ms

```
using Statistics

function kurtosis_loop(X)
    E = 0.0
    S = 0.0
    n = length(X)

    mu = mean(X)
    @inbounds @simd for i in 1:n
        x = X[i]
        x2 = (x - mu)^2
        x4 = x2^2
        E += x4
        S += x2
    end
    E = E / n
    S = S / n
    return E / (S^2)
end
```

C++

18.087ms

```
#include <vector>
#include <numeric>

double kurtosis(const std::vector<double>& X) {
    double mu = std::accumulate(X.begin(), X.end(), 0.0) /
X.size();
    double E = 0.0, S = 0.0;
    for (auto x : X) {
        double x2 = (x - mu) * (x - mu);
        double x4 = x2 * x2;
        E += x4;
        S += x2;
    }
    E /= X.size();
    S /= X.size();
    return E / (S * S);
}
```

02

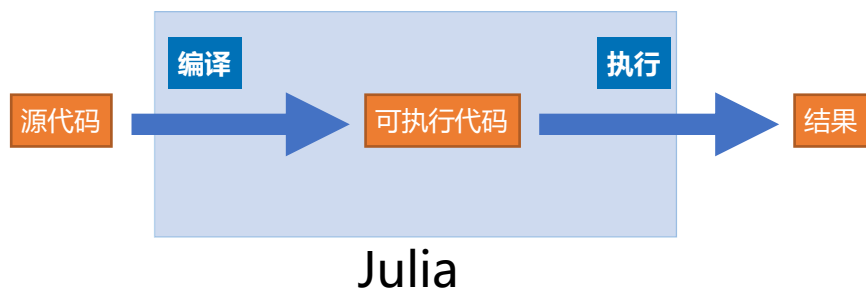
多重派发

©苏州同元软控信息技术有限公司版权所有，未经许可，不得复印、传播或以其他方式使用

@苏州同元软控信息技术有限公司
版权所有，侵权必究

2. 多重派发

Julia 函数调用遇到新的输入类型组合时会触发代码编译



代码编译在何时被触发?

```
julia> add(x, y) = x + y ✗  
add (generic function with 1 method)  
  
julia> add(1, 2) ✓  
3  
  
julia> add(3, 4) ✗  
7  
  
julia> add(1.0, 2.0) ✓  
3.0  
  
julia> add(1.0, 2) ✓  
3.0
```

✓ 触发编译 ✗ 未触发编译

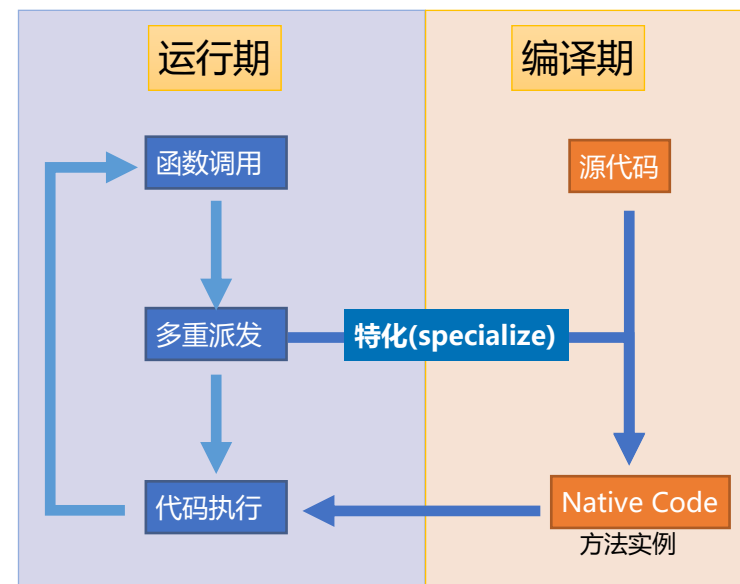
函数编译的结果称为**方法实例**

```
julia> using MethodAnalysis  
  
julia> add(x, y) = x + y ✗  
add (generic function with 1 method)  
  
julia> methodinstances(add)  
Core.MethodInstance[]  
  
julia> add(1, 2) ✓  
3  
  
julia> methodinstances(add)  
1-element Vector{Core.MethodInstance}:  
MethodInstance for add(::Int64, ::Int64)  
  
julia> add(3, 4) ✗  
7  
  
julia> methodinstances(add)  
1-element Vector{Core.MethodInstance}:  
MethodInstance for add(::Int64, ::Int64)  
  
julia> add(1.0, 2.0) ✓  
3.0  
  
julia> methodinstances(add)  
2-element Vector{Core.MethodInstance}:  
MethodInstance for add(::Int64, ::Int64)  
MethodInstance for add(::Float64, ::Float64)  
  
julia> add(1.0, 2) ✓  
3.0  
  
julia> methodinstances(add)  
3-element Vector{Core.MethodInstance}:  
MethodInstance for add(::Int64, ::Int64)  
MethodInstance for add(::Float64, ::Float64)  
MethodInstance for add(::Float64, ::Int64)
```

方法实例 (method instance)

- 方法实例具有完整的类型信息
- 编译新的方法实例的过程叫做**特化 (specialize)**
- 如果方法实例已经存在, 则不需要重新编译
- 根据**全部**输入的类型来判断调用哪个方法实例的策略称为**多重派发 (multiple dispatch)**

Julia 的函数调用、编译与执行围绕**多重派发**进行



2. 多重派发

方法实例与多重派发的概念意味着：即使不加类型限定，也不会影响 Julia 代码的性能

```
function mysum_v1(X)
    rst = 0.0
    for x in X
        rst += x
    end
    return rst
end
```

```
function mysum_v2(X::Vector{Float64})
    rst = 0.0
    for x in X
        rst += x
    end
    return rst
end
```

性能完全一致

```
julia> using BenchmarkTools
julia> X = rand(4096);

julia> @btime mysum_v1($X);
3.525 μs (0 allocations: 0 bytes)

julia> @btime mysum_v2($X);
3.525 μs (0 allocations: 0 bytes)
```

可以通过 `@code_llvm mysum_v1(X)` 来查看所调用的编译后结果

2. 多重派发

Julia **函数** (function) 由多个**方法** (method) 共同定义，多重派发调用最匹配的方法定义

不同参数（数目、类型）的组合对应的是同一个**函数**的不同**方法**

```
julia> is_number(x) = false
is_number (generic function with 1 method) 1

julia> is_number(x::Number) = true
is_number (generic function with 2 methods) 2

julia> is_number(x::Int) = true
is_number (generic function with 3 methods) 3

julia> is_number(1)
julia> is_number(1.0)
julia> is_number([2])
julia> is_number("hello world")
```

编译**方法实例**时会使用对应的方法

```
julia> methodinstances(is_number)
4-element Vector{Core.MethodInstance}:
MethodInstance for is_number(::Int64) 3
MethodInstance for is_number(::Float64) 2
MethodInstance for is_number(::Vector{Int64}) 1
MethodInstance for is_number(::String) 1
```

方法覆盖只发生在类型组合相同的情况下

```
julia> is_number(x::Number) = false
is_number (generic function with 3 methods)

julia> is_number(1)
true
```

2. 多重派发

多重派发允许我们针对特定的数据类型给出特定的优化实现

通过 Diagonal 类型告诉编译器：只有对角线有值

```
julia> using LinearAlgebra

julia> Xd = Diagonal([1 2 3 4])
4x4 Diagonal{Int64, Vector{Int64}}:
 1 . . .
 . 2 . .
 . . 3 .
 . . . 4

julia> X = collect(Xd)
4x4 Matrix{Int64}:
 1 0 0 0
 0 2 0 0
 0 0 3 0
 0 0 0 4

julia> Xd == X
true
```

```
julia> Xd = Diagonal(rand(512))
julia> X = collect(Xd);

julia> @btime Xd * Xd; 调用 Diagonal 版本的乘法
398.651 ns (2 allocations: 4.14 KiB)

julia> @btime X * X;
863.433 μs (2 allocations: 2.00 MiB)

julia> Xd * Xd == X * X
true
```

```
julia> function mysum(A)
    rst = zero(eltype(A))
    @simd for i in eachindex(A)
        rst += A[i]
    end
    return rst
end

mysum (generic function with 1 method)

julia> @btime mysum(Xd)
75.197 μs (1 allocation: 16 bytes)

julia> @btime mysum(X)
30.326 μs (1 allocation: 16 bytes)
```

```
julia> mysum(X::Diagonal) = mysum(X.diag)
mysum (generic function with 2 methods)

julia> @btime mysum(Xd) 调用 Diagonal 版本的 mysum
43.735 ns (1 allocation: 16 bytes)
```

函数的不同方法实例之间可以递归调用

2. 多重派发

一个实际的 Julia 项目可能由大量的小函数和多重派发组成

```
# Reshaping OffsetArrays can "pop" the original OffsetArray wrapper and return
# an OffsetArray(reshape(...)) instead of an OffsetArray(reshape(OffsetArray(...)))
# Short-circuit for AbstractVectors if the axes are compatible to get around the Base restriction
# to 1-based vectors
function _reshape(A::AbstractVector, inds::Tuple{OffsetAxis})
    @noinline throw_dimerr(ind::Integer) = throw(
        DimensionMismatch("parent has $(size(A,1)) elements, which is incompatible with length $ind"))
    @noinline throw_dimerr(ind) = throw(
        DimensionMismatch("parent has $(size(A,1)) elements, which is incompatible with indices $ind"))
    _checksize(first(inds), size(A,1)) || throw_dimerr(first(inds))
    A
end
_reshape(A, inds) = _reshape2(A, inds)
_reshape2(A, inds) = reshape(A, inds)
# avoid a stackoverflow by relegating to the parent if no_offset_view returns an offsetarray
_reshape2(A::OffsetArray, inds) = reshape(parent(A), inds)
_reshape_nov(A, inds) = _reshape(no_offset_view(A), inds)

Base.reshape(A::OffsetArray, inds::Tuple{OffsetAxis,Vararg{OffsetAxis}}) =
    OffsetArray(_reshape(parent(A), inds), map(_toaxis, inds))
# And for non-offset axes, we can just return a reshape of the parent directly
Base.reshape(A::OffsetArray, inds::Tuple{Union{Integer,Base.OneTo},Vararg{Union{Integer,Base.OneTo}}}) =
    _reshape_nov(A, inds)
Base.reshape(A::OffsetArray, inds::Dims) = _reshape_nov(A, inds)
Base.reshape(A::OffsetVector, ::Colon) = A
Base.reshape(A::OffsetVector, ::Tuple{Colon}) = A
Base.reshape(A::OffsetArray, inds::Union{Int,Colon}...) = reshape(A, inds)
Base.reshape(A::OffsetArray, inds::Tuple{Vararg{Union{Int,Colon}}}) = _reshape_nov(A, inds)
# The following two additional methods for Colon are added to resolve method ambiguities to
# Base: https://github.com/JuliaLang/julia/pull/45387#issuecomment-1132859663
Base.reshape(A::OffsetArray, inds::Colon) = reshape(parent(A), inds)
Base.reshape(A::OffsetArray, inds::Tuple{Colon}) = reshape(parent(A), inds)
```

OffsetArrays.jl 源码

<https://github.com/JuliaArrays/OffsetArrays.jl>

Julia 内置标准库中定义了两百多种加法

```
julia> methods(+)  
# 207 methods for generic function "+" from Base:  
[1] +(x::Dates.CompoundPeriod, y::Dates.CompoundPeriod)  
    @ Dates /Applications/Julia-1.10.app/Contents/Resources/julia/shar  
tdlib/v1.10/Dates/src/periods.jl:334  
[2] +(x::Dates.CompoundPeriod, y::Dates.TimeType)  
    @ Dates /Applications/Julia-1.10.app/Contents/Resources/julia/shar  
tdlib/v1.10/Dates/src/periods.jl:362  
[3] +(x::Dates.CompoundPeriod, y::Dates.Period)  
    @ Dates /Applications/Julia-1.10.app/Contents/Resources/julia/shar  
tdlib/v1.10/Dates/src/periods.jl:332
```


基于多重派发，Julia 允许我们构建一个渐进式的性能优化体验：

- 第一天只需要给出通用的代码实现，确保模型正确
- 进一步挖掘模型中的固有信息，将这些信息以 Julia 数据类型的方式编码
- 围绕新创建的数据类型，通过多重派发引入优化实现 —— 这只需要增加一些新的方法，而不需要对已有的系统进行大规模重构

Julia 性能优化的核心在于**渐进式优化**：

- 原型实现只要能跑，不需要关心是否使用了最佳实践，也不需要过度在意它是否使用了高效的算法实现
- 在有需要的时候，针对性地引入性能优化最佳实践、或者通过多重派发引入特定的优化实现

03

类型稳定

©苏州同元软控信息技术有限公司版权所有，未经许可，不得复印、传播或以其他方式使用

@苏州同元软控信息技术有限公司
版权所有，侵权必究

3. 类型稳定

max 函数在不同编程语言下的典型实现

python

```
def mymax(a, b):  
    if a > b:  
        return a  
    else:  
        return b
```

Julia

```
function mymax(a, b)  
    if a > b  
        return a  
    else  
        return b  
    end  
end
```

C/C++

```
double maxd(double a, double b) {  
    if (a > b) {  
        return a;  
    } else {  
        return b;  
    }  
}
```

MATLAB

```
function c = mymax(a, b)  
    if a > b  
        c = a  
    else  
        c = b  
    end  
end
```

语言	类型	泛型支持	性能
C/C++	静态编译型语言	差	优
Python/MATLAB	动态解释型语言	优	差
Julia	动态编译型语言	优	优

问：Julia 如何在泛型的基础上做到高性能？



3. 类型稳定

代码编译期间只能够获得类型信息，而无法获得值信息

函数输入 `mymax(1, 2)` 在编译阶段是 `mymax(::Int, ::Int)`

Julia

```
function mymax(a, b)
    if a > b
        return a
    else
        return b
    end
end
```

`mymax(::Int, ::Int)`

`mymax(::Float64, ::Float64)`

```
julia> @code_warntype mymax(1, 2)
MethodInstance for mymax(::Int64, ::Int64)
  from mymax(a, b) @ Main REPL[1]:1
Arguments
  #self# :: Core.Const(mymax)
  a :: Int64
  b :: Int64
Body::Int64
1 - %1 = (a > b) :: Bool
   goto #3 if not %1
2 - return a
3 - return b
```

输入 Int, Int 类型
输出 Int 类型

```
julia> @code_warntype mymax(1.5, 2.5)
MethodInstance for mymax(::Float64, ::Float64)
  from mymax(a, b) @ Main REPL[1]:1
Arguments
  #self# :: Core.Const(mymax)
  a :: Float64
  b :: Float64
Body::Float64
1 - %1 = (a > b) :: Bool
   goto #3 if not %1
2 - return a
3 - return b
```

输入 Float64, Float64 类型
输出 Float64 类型

实际上，编译器还可以获得一些常量信息。

3. 类型稳定

问：编译器如何判断 `mymax(1.5, 2)` 与 `mymax(1, 2.5)` 的返回值类型？

编译器不知道输入的具体值 1.5, 2.5, 1, 2

Julia

```
function mymax(a, b)
    if a > b
        return a
    else
        return b
    end
end
```

代码编译

代码执行

```
julia> typeof(mymax(1.5, 2))
Int64
julia> typeof(mymax(1, 2.5))
Float64
```

尽管实际代码运行的数值类型是确定的，但是在编译阶段，Julia 编译器进行编译优化基于的是 `Union{Float64, Int64}` 的摇摆类型来做，因此难以做到完全的性能优化。

```
julia> @code_warntype mymax(1.5, 2)
MethodInstance for mymax(::Float64, ::Int64)
  from mymax(a, b) @ Main REPL[1]:1
Arguments
  #self#::Core.Const(mymax)
  a::Float64
  b::Int64
Body::Union{Float64, Int64}
1 - %1 = (a > b)::Bool
   |      goto #3 if not %1
2 -      return a
3 -      return b
```

输出在 Int 与 Float64 类型之间摇摆

```
julia> @code_warntype mymax(1, 2.5)
MethodInstance for mymax(::Int64, ::Float64)
  from mymax(a, b) @ Main REPL[1]:1
Arguments
  #self#::Core.Const(mymax)
  a::Int64
  b::Float64
Body::Union{Float64, Int64}
1 - %1 = (a > b)::Bool
   |      goto #3 if not %1
2 -      return a
3 -      return b
```

输出在 Int 与 Float64 类型之间摇摆

@code_warntype 会用黄色/红色标记类型不稳定的情况

3. 类型稳定

对于类型稳定的代码来说，Julia 能够根据输入的类型来推断输出的类型

类型不稳定的代码

```
julia> function mymax(a, b)
    if a > b
        return a
    else
        return b
    end
end
mymax (generic function with 1 method)
```

类型稳定的代码

```
julia> function mymax_stable(a, b)
    a_, b_ = promote(a, b)
    if a_ > b_
        return a_
    else
        return b_
    end
end
mymax_stable (generic function with 1 method)
```

```
julia> @code_warntype mymax_stable(1.5, 2)
MethodInstance for mymax_stable(::Float64, ::Int64)
  from mymax_stable(a, b) @ Main REPL[62]:1
Arguments
  #self#::Core.Const(mymax_stable)
  a::Float64
  b::Int64
Locals
  @_4::Int64
  b_::Float64
  a_::Float64
Body::Float64
1 - %1 = Main.promote(a, b)::Tuple{Float64, Float64}
  | %2 = Base.indexed_iterate(%1, 1)::Core.PartialStruct{
  | Any[Float64, Core.Const(2)]}
  |   (a_ = Core.getfield(%2, 1))
  |   (@_4 = Core.getfield(%2, 2))
  | %5 = Base.indexed_iterate(%1, 2, @_4::Core.Const(2))::Core.PartialStruct{
  | le{Float64, Int64}, Any[Float64, Core.Const(3)]}
  |   (b_ = Core.getfield(%5, 1))
  | %7 = (a_ > b_)::Bool
  | goto #3 if not %7
2 - return a_
3 - return b_

```

3. 类型稳定

对于类型稳定的代码来说，Julia 能够根据输入的类型来推断输出的类型

类型不稳定的代码

```
julia> function mymax(a, b)
    if a > b
        return a
    else
        return b
    end
end
mymax (generic function with 1 method)
```

类型稳定的代码

```
julia> function mymax_stable(a, b)
    a_, b_ = promote(a, b)
    if a_ > b_
        return a_
    else
        return b_
    end
end
mymax_stable (generic function with 1 method)
```

Julia 性能优化核心建议：写类型稳定的代码

类型不稳定会影响后续代码的性能

```
julia> A = rand{Float64, 64, 64}

julia> B = rand{0:1, 64, 64}

julia> @btime mymax(A[1], B[1])
53.695 ns (2 allocations: 32 bytes)
```

```
julia> @btime mymax_stable(A[1], B[1])
55.763 ns (2 allocations: 32 bytes)
```

```
julia> @btime sum(mymax.(A, B))
136.630 μs (6209 allocations: 161.06 KiB)
```

```
julia> @btime sum(mymax_stable.(A, B))
2.826 μs (4 allocations: 32.09 KiB)
```

通过 using BenchmarkTools 来获得 @btime 性能测试工具

3. 类型稳定

类型稳定(type stability) 理解1: 函数的返回值类型在不同情况下是不变的

```
julia> f() = rand{Bool} ? 0 : 1.0
f (generic function with 1 method)
```

```
julia> @code_warntype f()
MethodInstance for f()
  from f () in Main at REPL[6]:1
Arguments
  #self#::Core.Const(f)
Body::Union{Float64, Int64}
  %1 = Main.rand(Main.Bool)::Bool
  goto #3 if not %1
  return 0
  return 1.0
```

```
julia> @btime f();
3.910 ns (0 allocations: 0 bytes)
```

```
julia> @btime sum($([f() for i in 1:1024]));
21.852 μs (1023 allocations: 15.98 KiB)
```

```
julia> f() = rand{Bool} ? 0.0 : 1.0
f (generic function with 1 method)
```

```
julia> @code_warntype f()
MethodInstance for f()
  from f () in Main at REPL[10]:1
Arguments
  #self#::Core.Const(f)
Body::Float64
  %1 = Main.rand(Main.Bool)::Bool
  goto #3 if not %1
  return 0.0
  return 1.0
```

```
julia> @btime f();
6.828 ns (0 allocations: 0 bytes)
```

```
julia> @btime sum($([f() for i in 1:1024]));
69.255 μs (0 allocations: 0 bytes)
```


3. 类型稳定

类型稳定(type stability) 理解 2: 函数的返回值 (及中间结果) 类型可以通过函数输入的类型来唯一推断

```
julia> function f(x)
    if x < 0.5
        return Int(0)
    else
        return Float64(1.0)
    end
end
```

f (generic function with 2 methods)

```
julia> @code_warntype f(rand())
MethodInstance for f(::Float64)
  from f(x) in Main at REPL[16]:1
Arguments
  #self#::Core.Const(f)
  x::Float64
Body::Union{Float64, Int64}
```

```
%1 = (x < 0.5)::Bool
    goto #3 if not %1
%3= Main.Int(0)::Core.Const(0)
    return %3
%5 = Main.Float64(1.0)::Core.Const(1.0)
    return %5
```

```
julia> @btime sum($([f(rand()) for i in 1:1024]));
22.534 μs (1023 allocations: 15.98 KiB)
```

```
julia> function f(x)
    if x < 0.5
        return zero(0)
    else
        return one(1.0)
    end
end
```

f (generic function with 2 methods)

```
julia> @code_warntype f(rand())
MethodInstance for f(::Float64)
  from f(x) in Main at REPL[35]:1
Arguments
  #self#::Core.Const(f)
  x::Float64
Body::Float64
```

```
%1 = (x < 0.5)::Bool
    goto #3 if not %1
%3= Main.zero(x)::Core.Const(0.0)
    return %3
%5 = Main.one(x)::Core.Const(1.0)
    return %5
```

```
julia> @btime sum($([f(rand()) for i in 1:1024]));
70.824 ns (0 allocations: 0 bytes)
```

返回与 x 类型相同的 0 或 1

```
julia> @code_warntype f(1)
MethodInstance for f(::Int64)
  from f(x) in Main at REPL[35]:1
Arguments
  #self#::Core.Const(f)
  x::Int64
Body::Int64
1 - %1 = (x < 0.5)::Bool
    goto #3 if not %1
2 - %3 = Main.zero(x)::Core.Const(0)
    return %3
3 - %5 = Main.one(x)::Core.Const(1)
    return %5
```

```
julia> @code_warntype f(1.0)
MethodInstance for f(::Float64)
  from f(x) in Main at REPL[35]:1
Arguments
  #self#::Core.Const(f)
  x::Float64
Body::Float64
1 - %1 = (x < 0.5)::Bool
    goto #3 if not %1
2 - %3 = Main.zero(x)::Core.Const(0.0)
    return %3
3 - %5 = Main.one(x)::Core.Const(1.0)
    return %5
```

3. 类型稳定

输入矩阵 A 是否为空矩阵会影响返回值的类型

类型不稳定的代码

```
julia> function mysum(A)
    rst = 0
    @simd for i in eachindex(A)
        rst += A[i]
    end
    return rst
end
mysum (generic function with 1 method)

julia> A = rand(1024, 1024);

julia> @btime mysum($A);
1.869 ms (0 allocations: 0 bytes)
```

```
julia> @code_warntype mysum(rand(4, 4))
MethodInstance for mysum(::Matrix{Float64})
  from mysum(A; dims) in Main at REPL[27]:1
Arguments
  #self#::Core.Const{mysum}
  A::Matrix{Float64}
Locals
  @_3::Union{Nothing, Tuple{Int64, Int64}}
  rst::Union{Float64, Int64}
  i::Int64
Body::Union{Float64, Int64}
1 --      (rst = 0)
   %2  = Main.eachindex(A)::Base.OneTo{Int64}
   (@_3 = Base.iterate(%2))
   %4  = (@_3 === nothing)::Bool
   %5  = Base.not_int(%4)::Bool
   goto #4 if not %5
2 -- %7  = @_3::Tuple{Int64, Int64}
   (i = Core.getfield(%7, 1))
   %9  = Core.getfield(%7, 2)::Int64
   %10 = rst::Union{Float64, Int64}
   %11 = Base.getindex(A, i)::Float64
   (rst = %10 + %11)
   (@_3 = Base.iterate(%2, %9))
   %14 = (@_3 === nothing)::Bool
   %15 = Base.not_int(%14)::Bool
   goto #4 if not %15
3 --      goto #2
4 --      return rst
```

类型稳定的代码

```
julia> function mysum(A)
    rst = zero(eltype(A))
    @simd for i in eachindex(A)
        rst += A[i]
    end
    return rst
end
mysum (generic function with 1 method)

julia> A = rand(1024, 1024);

julia> @btime mysum($A);
131.004 μs (0 allocations: 0 bytes)
```

3. 类型稳定

类型不稳定很容易从底层传播出来影响整体函数性能

```
julia> function mysum(A)
    rst = zero(eltype(A))
    @simd for i in eachindex(A)
        rst += A[i]
    end
    return rst
end

mysum (generic function with 1 method)

julia> gen_data(f, n::Int) = [f() for i in 1:n, j in 1:n]
gen_data (generic function with 1 method)

julia> f_slow = rand(Bool) ? 0 : 1.0 ::Union{Int,Float64}
f_slow (generic function with 1 method)

julia> h(n) = mysum(gen_data(f_slow, 1024))
h (generic function with 1 method)

julia> @btime h(1024)
35.263 ms (1571575 allocations: 39.98 MiB)
```

f_slow 类型不稳定导致 h 类型不稳定且性能下降

```
julia> f_fast = rand(Bool) ? 0.0 : 1.0
f_fast (generic function with 1 method)

julia> h(n) = mysum(gen_data(f_fast, 1024))
h (generic function with 1 method)

julia> @btime h(1024)
5.317 ms (2 allocations: 8.00 MiB)
```

未经许可，不得复印、传播或以其他方式使用

3. 类型稳定

大部分类型不稳定来源于条件分支可能返回不同类型，但它并不一定是类型不稳定

这个条件分支只需要类型就可以判断

```
julia> function f(x)
    if x isa AbstractFloat
        return 3.0 * x
    elseif x isa Integer
        return 2 * x
    else
        return zero(x)
    end
end
f (generic function with 1 method)
```

```
julia> g(x::AbstractFloat) = 3.0 * x
g (generic function with 1 method)

julia> g(x::Integer) = 2 * x
g (generic function with 2 methods)

julia> g(x) = zero(x)
g (generic function with 3 methods)
```

```
julia> @code_warntype f(1.0)
MethodInstance for f(::Float64)
  from f(x) @ Main REPL[50]:1
Arguments
  #self#::Core.Const(f)
  x::Float64
Body::Float64
1 - %1 = (x isa Main.AbstractFloat)::Core.Const(true)
   └─ goto #3 if not %1
2 - %3 = (3.0 * x)::Float64
   └─ return %3
3 - Core.Const:(x isa Main.Integer))
   └─ Core.Const:(goto %9 if not %5))
   └─ Core.Const:(2 * x)
   └─ Core.Const:(return %7))
   └─ Core.Const:(Main.zero(x))
   └─ Core.Const:(return %9))
```

@code_llvm f(3) 与 @code_llvm g(3) 的结果是一致的

3. 类型稳定

类型稳定并不需要你将所有地方都添加类型标记

```
julia> function mysum_v1(A::Matrix{Float64}) ::Float64
    rst = 0.0::Float64
    @simd for i in eachindex(A)
        rst += A[i]::Float64
    end
    return rst::Float64
end
mysum_v1 (generic function with 1 method)

julia> function mysum_v2(A)
    rst = zero(eltype(A))
    @simd for i in eachindex(A)
        rst += A[i]
    end
    return rst
end
mysum_v2 (generic function with 1 method)
```

对于类型稳定的 Julia 代码来说，添加额外的类型标记不会让代码变得更快

```
julia> A = rand(64, 64);

julia> @btime mysum_v1($A);
237.043 ns (0 allocations: 0 bytes)

julia> @btime mysum_v2($A);
237.026 ns (0 allocations: 0 bytes)
```

经验观察：C/C++ 程序员要比 Python/MATLAB 程序员更容易写出类型稳定的代码

判断代码类型稳定的思路：

- 仅通过输入的类型（而非输入的值）能否推断输出以及中间结果的类型
- 使用 `@code_warntype` 来验证类型不稳定行为

类型不稳定会导致：

- Julia 编译器难以做充足的编译优化
- 额外的运行时类型检查和开销
- 类型不稳定往往影响的是函数调用者的性能

编程建议：

- 尽量避免类型不稳定的代码
- 类型稳定不是通过过度添加类型标记来实现的，而是通过函数的合理拆分与设计
- 在进行任何其他性能优化之前优先评估代码类型不稳定对性能的影响

04 向量化编程

@苏州同元软控信息技术有限公司版权所有，未经许可，不得复印、传播或以其他方式使用
版权所有，侵权必究

4. 向量化编程

- ❑ 向量化编程：通过一次性处理整个数组或数据集，而非使用循环对元素逐个进行操作的编程范式。
- ❑ 对于Matlab/Python来说，通过向量化可以获得性能加速；对于Julia来说，虽然Julia有很好的向量化编程支持，但它的主要目的是为了简化写法而不是加速代码。

for循环版本

```
@time begin
    t = 0:.01:10
    y = Vector{Float64}(undef, length(t))
    for i in eachindex(t)
        y[i] = sin(t[i])
    end
end
# 耗时约 0.000118 seconds
```

```
function loop_sum()
    s = 0.0
    for k = 1:1000_000
        s += 1.0 / (k * k)
    end
    return s
end
@time y = loop_sum()
# 0.001474 seconds (1 allocation: 16 bytes)
```

向量化版本

```
@time begin
    t = 0:.01:10
    y = sin.(t)
end
# 耗时约 0.000036 seconds
```

```
function vec_sum()
    k = 1:1:1000_000
    s = sum(1 ./ (k .* k))
    return s
end
@time y = vec_sum()
# 0.001764 seconds (3 allocations: 7.629 MiB)
```


4. 向量化编程

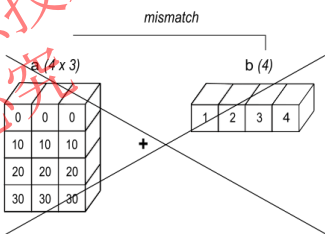
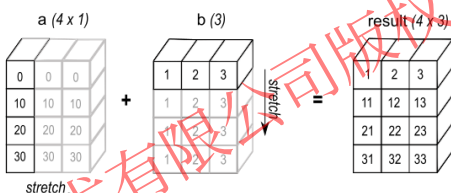
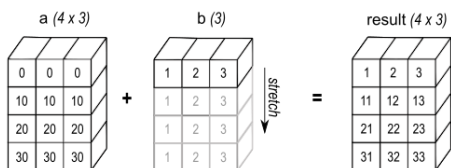
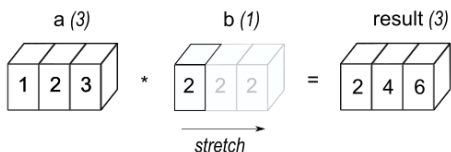
- 在不同尺寸的数组上执行元素对元素的操作时，Julia 的广播会将参数中低维度的参数扩展，使得其与其他维度匹配，且不会使用额外的内存，并将所给的函数逐元素地应用。
- Julia 的广播会尽可能多地融合连续的点运算符进一个 for 循环：更少的内存开销和更高的性能

```
# ok
[1 2 3] .* 2

# ok
a = [0 0 0
      10 10 10
      20 20 20
      30 30 30]
b = [1 2 3]
a .+ b

# ok
a = [0 10 20 30]'
b = [1 2 3]
a .+ b
```

```
# 报错
a = [0 0 0
      10 10 10
      20 20 20
      30 30 30]
b = [1 2 3 4]
a .+ b
```



5种等价写法及性能比较：

```
X = rand(10^6)

@time y1 = sin.(cos(X))
# 0.013138 seconds (6 allocations: 7.630 MiB)

@time y2 = @. sin(cos(X))
# 0.011807 seconds (6 allocations: 7.630 MiB)

@time y3 = broadcast(x -> sin(cos(x)), X)
# 0.038181 seconds (66.00 k allocations: 12.112 MiB)

@time y4 = [sin(cos(x)) for x in X]
# 0.038181 seconds (66.00 k allocations: 12.112 MiB)

@time y5 = map(x -> sin(cos(x)), X)
# 0.030999 seconds (25.80 k allocations: 9.350 MiB)
```

Syslab Julia 文件 IO

Syslab 语言组

苏州同元软控信息技术有限公司

2024-08-08



TONGYUAN
Software and Control

@苏州同元软控信息技术有限公司版权所有，侵权必究
未经许可，不得复印、传播或以其他方式使用

目录

1. 文件IO简介

2. 文件IO操作函数

3. 场景示例

01 文件IO简介

©苏州同元软控信息技术有限公司版权所有，未经许可，不得复印、传播或以其他方式使用

@苏州同元软控信息技术有限公司
版权所有，侵权必究

1. 文件IO简介

- 文件 IO（输入/输出）是计算机程序与外部文件系统之间数据交换的过程。在Julia语言中，文件IO操作允许程序读取和写入文件，这对于数据分析、数据处理、日志记录、配置管理等应用至关重要。Julia 提供了一系列灵活且强大的工具和函数，使得文件IO操作既简单又高效。
- 在 Julia 中，对文件的操作都被抽象成操作文件 IO 流对象。IO 流是一种抽象概念，它在程序与文件之间建立了一个数据传输通道，允许程序按顺序读取或写入文件内容，用于表示数据的连续流动，而不必关心数据的来源（如文件、网络连接等）或去向。

1. 文件IO简介 – IO流 : IOStream

- Julia 用抽象 IO 类来表示 IO 流，在Julia REPL中输入如下命令，可知 IOStream 是 IO 类的子类。

```
supertypes(IOStream)
```



```
julia> supertypes(IOStream)
(IOStream, IO, Any)
```

- IOStream 是处理文件 IO 流的主要类型之一，提供了丰富的函数接口来操作文件数据，后面会详细介绍。

例如：使用 open 函数以只读模式打开一个名为“example.txt” 的文本文件时，会返回 IOStream 对象，这个对象就代表了所操作的文件。

```
io = open("example.txt", "r")
```



```
example.txt X
1 Welcome to Syslab.
2 Here is a tutorial for file IO.
3 Hope you 学习愉快。

julia> io = open("example.txt", "r")
IOStream(<file example.txt>)
```

1. 文件IO简介 – IO流 : IOBuffer

- IOBuffer 也是一种常用的流类型，它也是 IO 类的子类。

```
supertypes(IOBuffer)
```

```
julia> supertypes(IOBuffer)
(IOBuffer, IO, Any)
```

- IOBuffer 是用于内存中数据流操作的类型，它提供了一个可增长的缓冲区，用来临时存储数据，模拟文件操作。因此非常适合于需要临时存储数据的场景，比如数据的预处理或传递大量数据时使用。如下示例可创建一个空的 IOBuffer 对象。

```
buf = IOBuffer()
```

```
julia> buf = IOBuffer()
IOBuffer(data=UInt8[...], readable=true, writable=true, seekable=true
, append=false, size=0, maxsize=Inf, ptr=1, mark=-1)
```

02 文件IO操作函数

©苏州同元软控信息技术有限公司版权所有，未经许可，不得复印、传播或以其他方式使用

@苏州同元软控信息技术有限公司
版权所有，侵权必究

2. 文件IO操作函数 - open

open 函数打开一个文件，返回与此文件相关联的 IOStream 对象。它的行为可以通过传递不同的参数来定制，包括文件路径、操作模式（如读、写、追加）以及是否允许多线程访问。首先介绍下列两个重载方法：

```
open(filename::AbstractString, [mode::AbstractString]; lock = true) -> IOStream
```

```
open(filename::AbstractString; lock = true, keywords...) -> IOStream
```

2. 文件IO操作函数 - open

open(filename::AbstractString, [mode::AbstractString]; lock = true) -> IOStream

open(filename::AbstractString; lock = true, keywords...) -> IOStream

mode	描述	对应关键字参数
r	read	none
w	write, create, truncate	write = true
a	write, create, append	append = true
r+	read, write	truncate = true, read = true
w+	read, write, create, truncate	truncate = true, read = true
a+	read, write, create, append	append = true, read = true

关键字参数	描述
write = true	允许写入文件，用于w模式
append = true	追加模式，文件指针位置移到文件尾，用于a和a+模式
read = true	允许读取文件，用于r、r+、w+和a+模式
truncate = true	打开文件时截断到零长度，用在w和w+模式

2. 文件IO操作函数 – open & close

`open(filename::AbstractString, [mode::AbstractString]; lock = true) -> IOStream`

`open(filename::AbstractString; lock = true, keywords...) -> IOStream`

因此，下面两种写法是完全等价的：

```
io = open("example.txt", "a+")
io = open("example.txt"; append=true,
read=true)
```

不得复印、传播或以其他方式使用

未经许可，

```
julia> io = open("example.txt", "a+")
IOStream(<file example.txt>)

julia> io = open("example.txt"; append=true, read=true)
IOStream(<file example.txt>)
```

通常程序结束时，应该调用close函数关闭文件流：

```
close(io)
```

```
julia> close(io)
```

2. 文件IO操作函数 - open

open 重载方法可以接收一个函数对象作为参数，这将允许我们在一个安全的上下文中操作文件，Julia 会确保文件最终会被关闭。重载形式为：open(f::Function, args...; kwargs...) 可以理解为将 open(args...; kwargs...) 返回的 IOStream 对象作为参数传递给函数 f，并返回函数 f 执行后的结果。下面两种写法等价：

```
function f(io::IO)
    str = read(io, String)
    return str
end
content = open(f, "example.txt", "r")
```

```
content = open("example.txt", "r") do file
    read(file, String)
end
println(content)
```

```
julia> function f(io::IO)
           str = read(io, String)
           return str
       end
f (generic function with 1 method)

julia> content = open(f, "example.txt", "r")
"Welcome to Syslab.\r\nHere is a tutorial for file IO.\r\nHope you 学习愉快。"
```

```
julia> content = open("example.txt", "r") do file
           read(file, String)
       end
"Welcome to Syslab.\r\nHere is a tutorial for file IO.\r\nHope you 学习愉快。"

julia> println(content)
Welcome to Syslab.
Here is a tutorial for file IO.
Hope you 学习愉快。
```

2. 文件IO操作函数 - read

read函数用于从文件或其他输入流中读取数据。它提供了多种重载形式，允许读取不同类型的数据，包括字节、字符串、特定数量的字节，以及自定义类型。read函数的灵活性使其成为处理文件和数据流输入的重要工具。主要重载形式如下表所示：

Read重载函数	说明
read(io::IO, T)	从io读取T类型的值
read(filename::AbstractString, arg...)	相当于 open(io->read(io, args...), filename)
read(s::IO, nb=typemax{Int})	从s中最多读取nb个字节，返回Vector{UInt8}类型的值
read(command::Cmd, String)	读取命令的输出，将其作为String类型的值返回

2. 文件IO操作函数 - read

read函数重载方法结果如下所示:

```
io = IOBuffer("Welcome to Syslab.");  
read(io, Char)  
  
io = IOBuffer("Welcome to Syslab.");  
read(io, String)
```



```
julia> io = IOBuffer("Welcome to Syslab.");  
julia> read(io, Char)  
'W': ASCII/Unicode U+0057 (category Lu: Letter, uppercase)  
  
julia> io = IOBuffer("Welcome to Syslab.");  
julia> read(io, String)  
"Welcome to Syslab."
```

```
read("example.txt")
```



```
julia> read("example.txt")  
77-element Vector{UInt8}:  
 0x57  
 0x65  
 0x6c  
 0x63  
 0x6f  
 0x6d  
 0x65  
 0x20  
 0x74  
  ⋮  
 0x84
```

```
read("example.txt", String)
```



```
julia> read("example.txt", String)  
"Welcome to Syslab.\r\nHere is a tutorial for file IO.\r\nHope you 学习愉快."
```

2. 文件IO操作函数 - read

read函数重载方法结果如下所示:

```
io = IOBuffer("Welcome to  
Syslab.");  
read(io, 3)
```



```
julia> io = IOBuffer("Welcome to Syslab.");  
  
julia> read(io, 3)  
3-element Vector{UInt8}:  
 0x57  
 0x65  
 0x6C
```

```
print(read(`ping  
www.bing.com`, String))
```



```
julia> print(read(`ping www.bing.com`, String))  
  
Pinging www.bing.com [202.89.233.100] with 32 bytes of data:  
Reply from 202.89.233.100: bytes=32 time=29ms TTL=116  
Reply from 202.89.233.100: bytes=32 time=27ms TTL=116  
Reply from 202.89.233.100: bytes=32 time=38ms TTL=116  
Reply from 202.89.233.100: bytes=32 time=27ms TTL=116  
  
Ping statistics for 202.89.233.100:  
    Packets: Sent = 4, Received = 4, Lost = 0 (0% loss),  
Approximate round trip times in milli-seconds:  
    Minimum = 27ms, Maximum = 38ms, Average = 30ms
```

2. 文件IO操作函数 – read!

read!函数可以将读取的数据直接填充到提供的数组中，适用于已经有一个预分配的数组，并希望直接将数据读入这个数组中的场景，避免返回一个新的数组。分配的数组字节大小要小于等于文件内容或io字节数。

read!(stream::IO, array::AbstractArray)

read!(filename::AbstractString, array::AbstractArray)

```
io = IOBuffer("Welcome to
Syslab.");
array = Vector{UInt8}(undef, 5);
read!(io, array)
```



```
julia> io = IOBuffer("Welcome to Syslab.");
julia> array = Vector{UInt8}(undef, 5);
julia> read!(io, array)
5-element Vector{UInt8}:
 0x57
 0x65
 0x6c
 0x63
 0x6f
```

```
let
    array = Vector{UInt8}(undef,
5);
    read!("example.txt", array)
end
```



```
julia> let
    array = Vector{UInt8}(undef, 5);
    read!("example.txt", array)
end
5-element Vector{UInt8}:
 0x57
 0x65
 0x6c
 0x63
 0x6f
```


2. 文件IO操作函数 – readeach & skipchars

- `readeach(io::IO, T)` 返回一个生成`read(io, T)`的可迭代对象，可用`for ... in`语法遍历io流中的内容。

```
io = IOBuffer("Welcome to Syslab.");  
for c in readeach(io, Char)  
    c == '\n' && break  
    print(c)  
end
```



```
julia> io = IOBuffer("Welcome to Syslab.");  
julia> for c in readeach(io, Char)  
    c == '\n' && break  
    print(c)  
end  
Welcome to Syslab.
```

- `skipchars(predicate, io::IO; linecomment = nothing)` 用于跳过满足特定条件的字符序列，`predict`是一个函数，接收一个字符作为输入，指示是否应该跳过该字符。`linecomment` 允许指定一个字符，用作行注释的开始，会忽略一整行注释，直到下一行。

```
buf = IOBuffer("    Welcome to Syslab.");  
skipchars(isspace, buf);  
read(buf, String)
```



```
julia> buf = IOBuffer("    Welcome to Syslab.");  
julia> skipchars(isspace, buf);  
julia> read(buf, String)  
"Welcome to Syslab."
```

```
buf = IOBuffer("# This line is a  
comment.\n    Welcome to Syslab.");  
skipchars(isspace, buf; linecomment = '#');  
read(buf, String)
```



```
julia> buf = IOBuffer("# This line is a comment.\n    Welcome to Syslab.");  
julia> skipchars(isspace, buf; linecomment = '#');  
julia> read(buf, String)  
"Welcome to Syslab."
```

2. 文件IO操作函数 – eachline

eachline 用于逐行读取来自于IO对象或文本文件中的文本内容。它返回一个可迭代对象，用于循环处理文本内容。有以下两种重载形式：

- eachline(io::IO=stdin; keep::Bool=false)
- eachline(filename::AbstractString; keep::Bool=false)

其中keep参数用于控制是否保留每行末尾的换行符。

从标准输入逐行读取

```
for line in eachline()
    println("Read line: ",
line)
end
```

```
julia> for line in eachline()
        println("Read line: ", line)
    end
123
Read line: 123
```

从已有IO对象中读取

```
buf = IOBuffer("Welcome to
Syslab.\nHere is...");
for line in eachline(buf)
    println("Read line: ", line)
end
```

```
julia> buf = IOBuffer("Welcome to Syslab.\nHere is ...");
julia> for line in eachline(buf)
        println("Read line: ", line)
    end
Read line: Welcome to Syslab.
Read line: Here is ...
```

从文件中读取

```
for line in eachline("example.txt",
keep=false)
    println("Read line(with newline): ",
repr(line))
end
```

```
for line in eachline("example.txt",
keep=true)
    println("Read line(with newline): ",
repr(line))
end
```

```
julia> for line in eachline("example.txt", keep=false)
        println("Read line (with newline): ", repr(line))
    end
Read line (with newline): "Welcome to Syslab."
Read line (with newline): "Here is a tutorial for file IO."
Read line (with newline): "Hope you 学习愉快."
```

```
julia> for line in eachline("example.txt", keep=true)
        println("Read line (with newline): ", repr(line))
    end
Read line (with newline): "Welcome to Syslab.\r\n"
Read line (with newline): "Here is a tutorial for file IO.\r\n"
Read line (with newline): "Hope you 学习愉快."
```

2. 文件IO操作函数 – readlines

与eachline不同，readlines函数读取来自于IO对象或文本文件中的所有行，并将它们作为字符串数组返回。主要有以下两种重载形式：

- readlines(io::IO=stdin; keep::Bool=false)
- readlines(filename::AbstractString; keep::Bool=false)

其中keep参数用于控制是否保留每行末尾的换行符。

从已有IO对象中读取

```
buf = IOBuffer("Welcome to  
Syslab.\nHere is...");  
lines = readlines(buf)
```

```
julia> buf = IOBuffer("Welcome to Syslab.\nHere is ...");  
  
julia> lines = readlines(buf)  
2-element Vector{String}:  
"Welcome to Syslab."  
"Here is ..."
```

从文件中读取

```
lines = readlines("example.txt")
```

```
julia> lines = readlines("example.txt")  
3-element Vector{String}:  
"Welcome to syslab."  
"Here is a tutorial for file Io."  
"Hope you 学习愉快."
```

2. 文件IO操作函数 – readuntil

readuntil读取来自于IO对象或文本文件中的内容，直到遇到指定分隔符delim。主要有以下两种重载形式：

- readuntil(stream::IO, delim; keep::Bool = false)
- readuntil(filename::AbstractString, delim; keep::Bool = false)

其中keep参数用于控制是否保留每行末尾的换行符。

从已有IO对象中读取

```
buf = IOBuffer("Welcome to  
Syslab.\nHere is...");  
readuntil(buf, "\n")  
readuntil(buf, "\n")
```

```
julia> buf = IOBuffer("Welcome to Syslab.\nHere is ...");  
julia> readuntil(buf, '\n')  
"Welcome to Syslab."  
  
julia> readuntil(buf, '\n')  
"Here is ..."
```

从文件中读取

```
lines = readuntil("example.txt", "\n")
```

```
julia> lines = readuntil("example.txt", "\n")  
"Welcome to syslab.\r"  
  
julia> lines = readuntil("example.txt", "\n")  
"Welcome to syslab.\r"
```

2. 文件IO操作函数 – write

write可以向IO流写入数据，或直接向执行的文件名写入各种类型的数据，包括但不限于字符串、字节数组等。主要有以下两种重载形式：

- write(io::IO, x)
- write(filename::AbstractString, x)

take!函数将io对象中的内容取出，返回字节数组，随后io将被置为初始状态。

```
io = IOBuffer();  
write(io, "Welcome to Syslab.\n",  
      "Second Line");  
print(String(take!(io)))
```



```
julia> io = IOBuffer();  
julia> write(io, "Welcome to Syslab.\n", "Second Line");  
julia> print(String(take!(io)))  
Welcome to Syslab.  
Second Line
```

```
io = IOBuffer();  
write(io, "Welcome to Syslab.\n") +  
write(io, "Second Line")  
print(String(take!(io)))
```



```
julia> io = IOBuffer();  
julia> write(io, "Welcome to Syslab.\n") + write(io, "Second Line")  
30  
julia> print(String(take!(io)))  
Welcome to Syslab.  
Second Line
```

2. 文件IO操作函数 – write

- `write(filename::AbstractString, x)` 直接将数据写入到文件中

```
# 写入字符串  
write("example_string.txt", "Hello, world!\n")
```

```
julia> write("example_string.txt", "Hello, world!\n")  
14
```

```
# 写入字节数组  
write("example_binary", [1, 2, 3, 4])
```

```
julia> write("example_binary", [1, 2, 3, 4])  
32
```

2. 文件IO操作函数 – position & peek & seek & skip ...

在IO对象的内部存在一个标志下一次读写操作位置的指针，这个文件指针位置决定了IO对象的读写行为。Julia 提供了一系列函数来操作这个文件指针位置：

- `position(l::Lexer)` 返回当前文件指针位置。
- `peek(stream[, T=UInt8])` 从流中读取一次T类型的值，但不影响文件指针位置。
- `seek(s, pos)` 将文件指针位置置于pos处。
- `seekstart(s)` 将文件指针位置至于开始位置。
- `seekend(s)` 将文件指针位置置于文件尾。
- `skip(s, offset)` 将当前文件指针位置移动offset。
- `mark(s::IO)` 在当前文件指针位置添加标记，取消用`unmark(s::IO)`，`ismarked(s::IO)`查询是否被标记。
- `reset(s::IO)` 将文件指针重置到标记位置，同时清除标记。

2. 文件IO操作函数 – position & peek & seek & skip ...

```
io = IOBuffer("Welcome to Syslab")

# position: 获取当前位置
println("Current position: ", position(io)) # 应该是0, 因为我们还没有读取任何内容

# peek: 查看下一个字节但不移动位置
peeked_char = peek(io, Char)
println("Peeked character: ", peeked_char)
println("Position after peek: ", position(io)) # 位置不变, 还是0

# read: 读取一个字符, 位置会前进
read_char = read(io, Char)
println("Read character: ", read_char)
println("Position after read: ", position(io)) # 位置前进了一个字符

# mark: 标记当前位置
mark(io)
println("Marked position: ", position(io))

# skip: 跳过指定数量的字节
skip(io, 5) # 跳过 5 个字节
println("Position after skip: ", position(io))
```



```
Current position: 0
Peeked character: W
Position after peek: 0
Read character: W
Position after read: 1
Marked position: 1
Position after skip: 6
```


2. 文件IO操作函数 – position & peek & seek & skip ...

```
# ismarked: 检查是否有标记
println("Is marked? ", ismarked(io))
# reset: 重置到最近的标记
reset(io)
println("Position after reset: ", position(io)) # 回到标记的位置
# unmark: 移除标记
unmark(io)
println("Is marked after unmark? ", ismarked(io))

# seek: 移动到指定位置
seek(io, 10)
println("Position after seek: ", position(io))

# seekstart: 移动到开始
seekstart(io)
println("Position after seekstart: ", position(io))

# seekend: 移动到末尾
seekend(io)
println("Position after seekend: ", position(io))

# 关闭 IOBuffer
close(io)
```



```
Is marked? true
Position after reset: 1
Is marked after unmark? false
Position after seek: 10
Position after seekstart: 0
Position after seekend: 17
```

03 场景示例

©苏州同元软控信息技术有限公司版权所有，未经许可，不得复印、传播或以其他方式使用

@苏州同元软控信息技术有限公司
版权所有，侵权必究

3. 场景示例 – 批量读取文件数据

- 如果文件不是特别大，可以一次性读取所有行到内存中：

```
lines = readlines("example.txt")
for line in lines
    println(line)
end
```

- 对于较大的文件，可以使用迭代器按行读取，这样可以节省内存：

```
open("example.txt", "r") do file
    for line in eachline(file)
        println(line)
    end
end
```

3. 场景示例 – 批量读取文件数据

- 如果需要按块（比如每次1024字节）读取数据：

```
open("example.txt", "r") do file
  while !eof(file)
    data = read(file, 1024) # 读取最多 1024 字节
    # 处理数据...
    println(String(data))
  end
end
```

- 另一种按块读取的方法是使用 IOBuffer 作为缓冲区：

```
open("example.txt", "r") do file
  buf = IOBuffer()
  while !eof(file)
    data = read(file, 1024) # 从文件中读取最多 1024 字节
    write(buf, data) # 将读取的数据写入到 IOBuffer 中
  end
  # 处理buf中的数据
  println(String(take!(buf)))
end
```

3. 场景示例 – 读取制表符“\t”间隔的数据文件

CSV.jl 和 DataFrames.jl 包可用于读取和处理TSV文件，DataFrames.jl 提供了大量的功能来支持数据处
理和分析任务。

下面代码生成一个 output.tsv 文件到当前工作目录，随后读取文件内容后打印。

```
using CSV
using DataFrames
# 设置 delim='\t' 来指定制表符作为字段分隔符
data = DataFrame(Name=["Alice", "Bob", "Charlie"], Age=[25, 30, 35], Salary=[50000, 60000, 70000])
# 导出 DataFrame 到 TSV 文件
CSV.write("output.tsv", data, delim='\t')
```

```
# 读取文件内容
data = CSV.read("output.tsv", DataFrame, delim='\t')
println(first(data, 2))
```



2x3 DataFrame			
Row	Name String7	Age Int64	Salary Int64
1	Alice	25	50000
2	Bob	30	60000

Julia 与其他语言的显著差异

Syslab 语言组

苏州同元软控信息技术有限公司

2024-08-12



TONGYUAN
Software and Control

@苏州同元软控信息技术有限公司版权所有，侵权必究
未经许可，不得复印、传播或以其他方式使用

目录

1. Julia 与 M 语言的显著差异

2. Julia 与 Python 语言的显著差异

Julia 与 M 语言的显著差异

Syslab 语言组

苏州同元软控信息技术有限公司

2024-08-12



TONGYUAN
Software and Control

@苏州同元软控信息技术有限公司版权所有，侵权必究
未经许可，不得复印、传播或以其他方式使用

目录

1. 概览

2. 语法差异

3. 常用函数差异

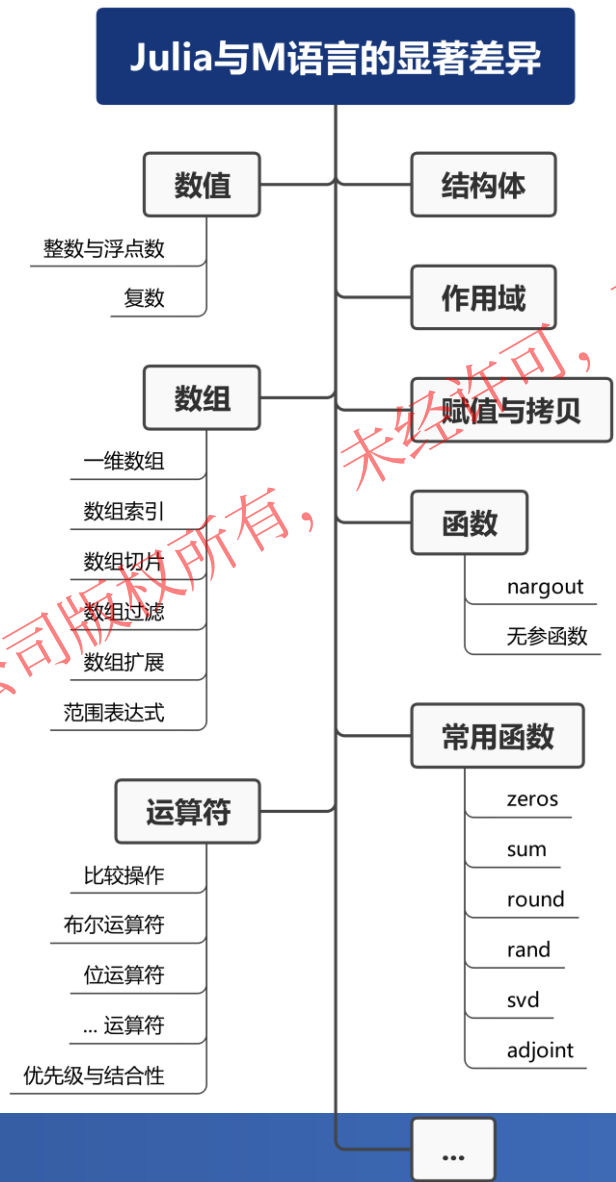
01 概览

©苏州同元软控信息技术有限公司版权所有，未经许可，不得复印、传播或以其他方式使用

@苏州同元软控信息技术有限公司
版权所有，侵权必究

1 概览

Julia 与 M 语言存在一些相似之处，但 Julia 不是 M 语言的克隆，两者存在显著差异。



02 语法差异

©苏州同元软控信息技术有限公司版权所有，未经许可，不得复印、传播或以其他方式使用

@苏州同元软控信息技术有限公司
版权所有，侵权必究

2 语法差异 – 数值

- 在Julia中，没有小数点的数字字面量会**创建整数**而不是浮点数。
- 在Julia中，复数虚部为 **im**，而MATLAB中为 **i** 或 **j**。

Julia

```
a=-1
typeof(a) # Int64

# 报错
2^a
#=
ERROR: DomainError with -1:
Cannot raise an integer x to a negative
power -1.
=#

# ok
2^float(a)

b = 1 + im
```

MATLAB

```
a=-1,
class(a) % 'double'

% ok
2^a

% 报错
a=int64(-1)
2^a
% Error using .^
% Integers can only be raised to positive
integral powers.

b = 1 + i
b = 1 + j
```

2 语法差异 – 数组

- **一维数组**: Julia 拥有真正的一维数组，列向量的大小为 N ，而不是 $N \times 1$ 。

Julia

```
a = [1 2 3]
# 1×3 矩阵

b = [1,2,3]
# 向量

c = [1;2;3]
# 向量

d = [1;2;3;;]
# 3×1 矩阵
```

MATLAB

```
a = [1 2 3]
% 1×3 矩阵

b = [1,2,3]
% 1×3 矩阵

c = [1;2;3]
% 3×1 矩阵
```

2 语法差异 – 数组

- 一维数组：Julia 拥有真正的一维数组，列向量的大小为 N ，而不是 $N \times 1$ 。
- **数组索引**：Julia 数组使用方括号 $A[i,j]$ 进行索引，MATLAB使用 $A(i,j)$ 进行索引。

Julia

```
a=Float64[1 2 3; 4 5 6]
#=
2×3 Matrix{Float64}:
 1.0  2.0  3.0
 4.0  5.0  6.0
=#

a[2,2]
# 5.0
```

MATLAB

```
a=[1 2 3; 4 5 6]
% a =
%      1      2      3
%      4      5      6

a(2,2)
% 5
```

2 语法差异 – 数组

- 一维数组：Julia 拥有真正的一维数组，列向量的大小为 N ，而不是 $N \times 1$ 。
- 数组索引：Julia 数组使用方括号 $A[i,j]$ 进行索引，MATLAB使用 $A(i,j)$ 进行索引。
- **数组切片**：Julia 对行向量($1 \times N$) 提取部分元素，可能返回向量。

Julia

```
x = [1 2 3 4 5 6]
# 1×6 Matrix{Int64}

# 提取部分元素，返回向量
px = x[2:end]
# 5-element Vector{Int64}

# 该用法与MATLAB的x(2:end)或x(:,2:end)等价
px = x[:, 2:end]
# 1×5 Matrix{Int64}
```

MATLAB

```
x = [1 2 3 4 5 6]
% 1x6 矩阵

px = x(2:end)
% 1x5 矩阵

px = x(:,2:end)
% 1x5 矩阵
```


2 语法差异 – 数组

- 数组过滤**: 在 MATLAB 中, 删除不需要的值的惯用方法是使用逻辑索引, 如表达式 $x(x>3)$ 或语句 $x(x>3) = []$ 来 in-place 修改 x 。在 Julia 中, 可以使用高阶函数 `filter` 和 `filter!`, 或类似 $x[x.>3]$ 等价写法。

Julia

```
x = [1 3 5 7 9]
```

```
x[x .> 3]  
# [5, 7, 9]
```

```
x = x[x .<= 3]  
# x = [1, 3]
```

MATLAB

```
x = [1 3 5 7 9]
```

```
x(x>3)  
% [5 7 9]
```

```
x(x>3)=[ ]  
% x = [1 3]
```

2 语法差异 – 数组

- **数组扩展**: Julia **不会在赋值语句中自动增长数组**。而在 MATLAB 中 $a(4) = 3.2$ 可以创建数组 $a = [0 \ 0 \ 0 \ 3.2]$ 。Julia 使用 `push!`、`append!` 来增长 Vector。

Julia

```
# 针对向量
a = [0,0,0,3.2]
push!(a,7)
# [0.0, 0.0, 0.0, 3.2, 7.0]

# 针对矩阵
a = [0 0 0 3.2]
a = [a 7] # 等价于 a = hcat(a, 7)
# [0.0 0.0 0.0 3.2 7.0]
```

MATLAB

```
a = [0 0 0 3.2]
% 1x4 矩阵

a(5) = 7
% a = [0 0 0 3.2000 7.0000]
```

2 语法差异 – 数组

- **范围表达式**: 在 Julia 中, `a:b` 和 `a:b:c` 构造 `AbstractRange` 对象。使用 `collect(a:b)` 构造一个类似 MATLAB 中完整的向量。通常, 不需要调用 `collect`。在大多数情况下, `AbstractRange` 对象将像普通数组一样运行, 但效率更高, 因为它是懒惰求值。

Julia

```
x=1:2:10
# 1:2:9 (不展开, 懒惰求值, 效率高)

# 转换为向量
collect(x)
# [1, 3, 5, 7, 9]

# 创建100个元素的向量
y = LinRange(-5,5,100)

y1 = LinRange(-5,5,7)
# [-5.0, -3.33333, -1.66667, 0.0, 1.66667, 3.33333, 5.0]
```

MATLAB

```
x=1:2:10
% [1 3 5 7 9]

% 默认创建 1x100 矩阵
y = linspace(-5,5)

y1 = linspace(-5,5,7)
% [-5.0000 -3.3333 -1.6667 0 1.6667 3.3333 5.0000]
```

2 语法差异 – 结构体

- Julia的结构体 struct(不可变) 和 mutable struct(可变), 与Matlab的 struct, 差异非常大。
- 在Julia中, 对于 struct 不可变结构体, 创建后不可修改, 但性能高效。

Julia

```
# 不可变结构体
struct Foo
    baz::Int
    qux::Float64
end

# 创建实例
foo = Foo(23, 1.5)

# 不允许修改
foo.qux = 2.0
# ERROR: setfield!:
immutable struct of type
Foo cannot be changed
```

Julia

```
# 可变结构体
mutable struct Bar
    baz::Int
    qux::Float64
end

# 创建实例
bar = Bar(23, 1.5);

# 允许修改
bar.qux = 2.0
```

MATLAB

```
% 方式1: 定义结构体对象
foo.baz = 23
foo.qux = 2.5

% 方式2: 结构体对象
bar = struct("baz", 23, "qux", 1.5)

% 允许修改
bar.qux = 2.0
```

2 语法差异 – 运算符

- 比较操作：**在 Julia 中，如果 A 和 B 是数组，像 `A == B` 这样的逻辑比较运算符不会返回布尔值数组。相反地，请使用 `A .== B`。对于其他的像是 `<`、`>` 的布尔运算符同理。

Julia

```
a=[1,3,5,7,9]
b=[1,3,5,7,9]

# 数组判等
a == b
# true

# 数组元素判等（向量化操作）
a .== b
# Bool[1, 1, 1, 1, 1]

# ok: 需要向量化操作
a .> 3
# Bool[0, 0, 1, 1, 1]
```

MATLAB

```
a=[1,3,5,7,9]
b=[1,3,5,7,9]

% 数组元素判等
a == b
% 1×5 logical array
% 1 1 1 1 1

% 数组判等
isequal(a, b)
% true

a > 3
% 1×5 logical array
% 0 0 1 1 1
```

2 语法差异 – 运算符

- **布尔运算符**: 在Julia中, 运算符采用 **!** (否定)、**&&** (短路与)、**||** (短路或) 进行布尔运算, 分别与 Matlab中的 **~**、**&&**、**||** 等价。注意, Matlab中的 **&** 与 **|** 表示无短路功能的逻辑运算, 不是位运算符。

Julia

```
A = false  
  
# 逻辑 not  
!A  
# true
```

MATLAB

```
A = false  
  
% 逻辑 not  
~A
```

2 语法差异 – 运算符

- **位运算符**: 在 Julia 中, 运算符 `&`、`|` 和 `⊕` (`xor`) 进行按位操作, 分别与 Matlab 中的 `bitand`、`bitor`、`bitxor` 等价。

Julia

```
# 按位取反
~4 # -5

# 按位与
4 & 10 # 0
4 & 12 # 4

# 按位或
4 | 10 # 14
4 | 1 # 5

# 按位异或
xor(4, 7) # 3
```

MATLAB

```
% 按位补码
bitcmp(int8(4)) % int8(-5)

% 按位与
bitand(int8(4), int8(10)) % int8(0)
bitand(int8(4), int8(12)) % int8(4)

% 按位或
bitor(int8(4), int8(10)) % int8(14)
bitor(int8(4), int8(1)) % int8(5)

% 按位异或
bitxor(int8(4), int8(7)) % int8(3)
```

2 语法差异 – 运算符

- **...运算符**：在Julia中，...运算符有两种用法：一是将多个参数组合成一个参数，二是将一个参数分解成多个不同参数；但在Matlab中，...表示续行符。
- Julia没有续行符，Julia中不完整的表达式会自动延续到下一行

Julia

```
# 用法1: 将多个参数组合成一个参数
function printargs(args...)
    println(typeof(args))
    for (i, arg) in
enumerate(args)
        println("Arg # $i$  =  $arg$ ")
    end
end
printargs(10, 20, 30)
#=
Tuple{Int64, Int64, Int64}
Arg #1 = 10
Arg #2 = 20
Arg #3 = 30
=#
```

Julia

```
# 用法2: 将一个参数分解成多个不同参数
function threeargs(a, b, c)
    println("a =  $a::\text{typeof}(a)$ ")
    println("b =  $b::\text{typeof}(b)$ ")
    println("c =  $c::\text{typeof}(c)$ ")
end

x = [1, 2, 3]
threeargs(x...)
#=
a = 1::Int64
b = 2::Int64
c = 3::Int64
=#
```

Julia

```
# Julia没有续行符
y = 1 +
    2 +
    4
# 7
```

MATLAB

```
% ...表示续行符
y = 1 + ...
    2 + ...
    4
% 7
```


2 语法差异 – 运算符

- 运算符优先级：**在 Julia 中， a^b^c 被认为是 $a^{(b^c)}$ 而在 MATLAB 中它是 $(a^b)^c$ ，这是因为 Julia 的 $^$ 是右结合。建议使用 $()$ 指定优先级。

Julia

```
# Julia中，^是右结合
4^3^2 # 等价于 4^(3^2)
# 262144

(4^3)^2
# 4096
```

MATLAB

```
% M语言中，^是左结合
4^3^2 % 等价于 (4^3)^2
% 4096

4^(3^2)
% 262144
```

2 语法差异 – 作用域

- 在 Julia 中，每个模块有自身的全局作用域/命名空间，而在 Matlab 中只有一个全局作用域。
- 在 Julia 中，用户难以理解的问题是 **for 作用域**，参见 Syslab 帮助文档中的【[Syslab>Julia语言概览>变量作用域](#)】。

Julia REPL

```
# 在交互式环境下，如REPL
s = 0
for i = 1:10
    s += i
end
s # 55
```

Julia 文件

```
# 在非交互环境下运行，如文件
s = 0
for i = 1:10
    s += i
end
# ERROR: UndefVarError: `s`
# not defined
```

MATLAB

```
% 在文件中执行，与在命令行窗口
% 执行，表现一致
s = 0
for i = 1:10
    s = s+i;
end
s
% 55
```

2 语法差异 – 赋值与拷贝

- Matlab 使用按值传递 (Call by value) , 防止数组被意外地篡改, 但也会导致不必要的数组拷贝。
- Julia 与 Python、Ruby等语言, 都采用共享调用技术 (Call by sharing) , 详情参见 Syslab 帮助文档中的【[Syslab>Julia语言概览>赋值与拷贝](#)】

Julia

```
# x和y指向同一块内存
x=[1,2,3]
y=x
x[1] = 10
y # [10,2,3]
```

Julia

```
# 数组拷贝
x = [1,2,3]
y = copy(x)
x[1] = 10
y # [1,2,3]
```

MATLAB

```
% 拷贝
x=[1,2,3]
y=x
x(1) = 10
y
% [1,2,3]
```

2 语法差异 – 函数

- 在 Julia 中，能返回多个值并将其赋值为元组，例如 $(a, b) = (1, 2)$ 或 $a, b = 1, 2$ 。在 Julia 中不存在 Matlab 的 `nargout`，它通常在 MATLAB 中用于根据返回值的数量执行可选工作。取而代之的是，用户可以使用可选参数和关键字参数来实现类似的功能。

Julia

```
function addme2(a,b)
    result = a+b
    absResult = abs(result)
    return result, absResult
end
```

```
addme2(1,-3)
# 返回元组 (-2, 2)
```

```
x, = addme2(1,-3)
# x = -2
```

```
x, y = addme2(1,-3)
# x = -2, y = 2
```

MATLAB

```
% 文件1
function [result,absResult] = addme2(a,b)
    result = a+b
    if nargin > 1
        absResult = abs(result);
    end
end
```

```
% 文件2 或 在命令行窗口输入
addme2(1,-3)
% 返回 -2
```

```
[x,y]=addme2(1,-3)
% x = -2, y = 2
```

2 语法差异 – 函数

- 在 Julia 中，调用无参数的函数时必须使用小括号，例如 rand()。
- 在 Matlab 中，调用无参数的函数时可以不带小括号。

Julia

```
# 函数变量
rand
# rand (generic function with 81 methods)

# 函数调用
rand()
# 0.3125316543482983
```

MATLAB

```
% rand 与 rand() 等价
rand % 0.8147
rand() % 0.9058
```

03 常用函数差异

@苏州同元软控信息技术有限公司版权所有，未经许可，不得复印、传播或以其他方式使用

侵权必究

3 常用函数差异 – zeros函数

- zeros(n)、ones(n) 生成矩阵操作，与MATLAB用法存在一些差异。

Julia

Julia返回向量，而Matlab返回方阵

```
zeros(2)
```

```
# [0.0, 0.0]
```

以下用法与Matlab的 zeros(2) 等价

```
zeros(2,2)
```

```
# 2×2 Matrix{Float64}
```

指定数组的数据类型

```
zeros{Int}(2,2)
```

```
# 2×2 Matrix{Int64}
```

MATLAB

```
zeros(2)
```

```
% 返回 2x2 方阵
```

```
zeros(2,'int64')
```

```
% 返回 2x2 int64 方阵
```

3 常用函数差异 – sum函数

- 在 Julia 中，例如 `sum`、`prod` 和 `max` 的归约操作会作用到数组的每一个元素上，当调用时只有一个函数，例如 `sum(A)`，即使 `A` 并不只有一个维度。

Julia

```
A = [1 2 3;4 5 6;7 8 9]
```

```
# 以下用法与Matlab的 sum(A,'all') 等价
```

```
sum(A) # 返回45
```

```
# 以下用法与Matlab的 sum(A) 或 sum(A,1) 等价
```

```
sum(A,dims=1)
```

```
# [12 15 18]
```

```
# 以下用法与Matlab的 sum(A,2) 等价
```

```
sum(A,dims=2)
```

```
#=
```

```
3×1 Matrix{Int64}:
```

```
6
```

```
15
```

```
24
```

```
=#
```

MATLAB

```
A = [1 2 3;4 5 6;7 8 9]
```

```
sum(A,'all') % 45
```

```
sum(A) % [12 15 18]
```

```
sum(A,1) % [12 15 18]
```

```
sum(A,2)
```

```
% ans =
```

```
%      6
```

```
%     15
```

```
%     24
```


3 常用函数差异 – round函数

- Julia和Matlab的默认行为不同，但有方法来控制四舍五入算法。
- Julia中，`round(X)` 按照四舍五入模式将 `X` 舍入为整数值，并返回与`X`相同类型的值。默认情况下这将四舍五入到最近的整数，如果出现平局（with ties，即小数值为0.5），则会被舍入为最近的偶数整数。

Julia 规则

对于Julia，存在以下参数控制：

- RoundNearest (default)
- RoundNearestTiesAway: rounded away from zero in Julia,
- RoundNearestTiesUp
- RoundToZero
- RoundFromZero (BigFloat only)
- RoundUp
- RoundDown

MATLAB 规则

对于Matlab，使用 **TieBreaker** 关键字参数控制（R2022a引入）：

- fromzero (default)
- tozero
- even
- odd
- plusinf
- minusinf

3 常用函数差异 – round函数

- Julia和Matlab的默认行为不同，但有方法来控制四舍五入算法。
- Julia中，`round(X)` 按照四舍五入模式将 `X` 舍入为整数值，并返回与`X`相同类型的值。默认情况下这将四舍五入到最近的整数，如果出现平局（with ties，即小数值为0.5），则会被舍入为最近的偶数整数。

Julia 规则

```
# 默认情况下，两者存在差异
round(0.5) # Julia返回0, Matlab返回1
round(1.5) # Julia返回2, Matlab返回2

# 以下用法与Matlab的`round(X)`等价
round(-1.5, RoundNearestTiesAway) # -2.0
round(-0.5, RoundNearestTiesAway) # -1.0
round(-0, RoundNearestTiesAway)   # 0
round(0, RoundNearestTiesAway)     # 0
round(0.5, RoundNearestTiesAway)   # 1
round(1.5, RoundNearestTiesAway)   # 2
```

MATLAB 规则

```
round(-1.5) % -2
round(-0.5) % -1
round(-0)    % 0
round(0)     % 0
round(0.5)   % 1
round(1.5)   % 2
```

3 常用函数差异 – rand函数

- Julia中的rand函数 与 Matlab中的 rand 函数，默认种子和算法是不一致的。若要两者保持一致，需要对随机种子和算法进行设置。

Julia

```
using TyRandom

# 使用该种子，可以产生与Matlab一致的随机数
rng=MT19937ar(5489)
rand(rng, 2,3)
#=
2×3 Matrix{Float64}:
 0.814724  0.126987  0.632359
 0.905792  0.913376  0.0975404
=#
```

MATLAB

```
rng('default')
r1 = rand(2,3)
% r1 =
%      0.8147      0.1270      0.6324
%      0.9058      0.9134      0.0975
```

3 常用函数差异 – svd函数

- Julia 的 svd 将奇异值作为向量而非密集对角矩阵返回。

Julia

```
using TyMath
A = [1 0 1; -1 -2 0; 0 1 -1]
U, S, V=svd(A)
U
#=
3×3 Matrix{Float64}:
-0.12      -0.809712  0.574427
 0.901753   0.153123  0.404222
-0.415261   0.566498  0.711785
=#

S # 注意: S是第2个返回值, 且为向量
#=
3-element Vector{Float64}:
 2.460504870018764
 1.699628148275318
 0.23912327825655452
=#

V
#=
3×3 adjoint(::Matrix{Float64}) with eltype Float64:
-0.415261  0.566498  0.711785
-0.901753  0.153123 -0.404222
 0.12      -0.809712 -0.574427
=#
```

MATLAB

```
A = [1 0 1; -1 -2 0; 0 1 -1]

% 方法1: 奇异值 S
svd(A)
% ans =
% 2.4605
% 1.6996
% 0.2391

% 方法2: 返回[U,S,V], 其中S为对角矩阵
[U,S,V]=svd(A)
% U =
% -0.1200    -0.8097    0.5744
%  0.9018    0.1531    0.4042
% -0.4153    0.5665    0.7118
%
% S =
% 2.4605         0         0
%         0    1.6996         0
%         0         0    0.2391
%
% V =
% -0.4153   -0.5665    0.7118
% -0.9018    0.1531   -0.4042
%  0.1200   -0.8097   -0.5744
```

3 常用函数差异 – adjoint函数

- 在 Julia 中, adjoint 函数执行共轭转置。
- 在 MATLAB 中, adjoint 提供了经典伴随, 它是余子式的转置。

Julia

```
using TyMath
A = magic(3);

# 公式:  $A \cdot X = \det(A) \cdot \text{eye}(n) = X \cdot A$ 
X = det(A) * inv(A)
#=
3×3 Matrix{Float64}:
-53.0  52.0 -23.0
 22.0  -8.0 -38.0
  7.0 -68.0 37.0
=#
```

MATLAB

```
A = magic(3);
X = adjoint(A)
% X =
% -53.0000    52.0000   -23.0000
%  22.0000    -8.0000  -38.0000
%   7.0000   -68.0000   37.0000
```

Julia与 Python语言的显著差异

Syslab 语言组

苏州同元软控信息技术有限公司

2024-08-12



TONGYUAN
Software and Control

@苏州同元软控信息技术有限公司版权所有，未经许可，不得复印、传播或以其他方式使用
版权所有，侵权必究

目录

1. 概览

2. 语法差异

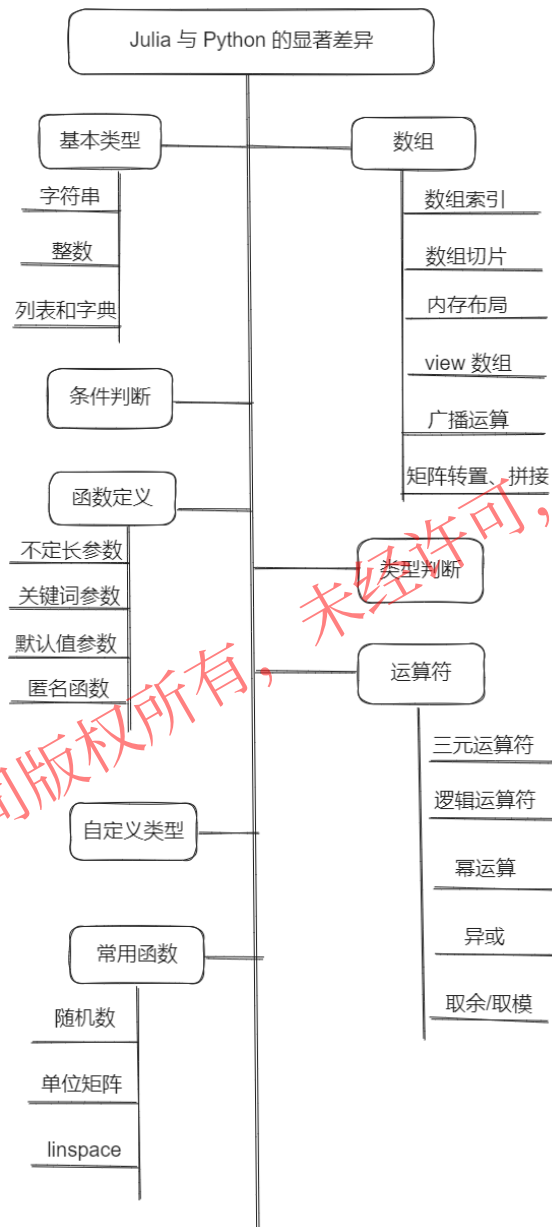
3. 常用函数差异

01 概览

@苏州同元软控信息技术有限公司版权所有，未经许可，不得复印、传播或以其他方式使用
版权所有，侵权必究

1 概览

Python的数组实现较多，
这里采用最常用的
numpy 作为对照。



02 语法差异

©苏州同元软控信息技术有限公司版权所有，未经许可，不得复印、传播或以其他方式使用

@苏州同元软控信息技术有限公司
版权所有，侵权必究

2 语法差异 – 字符串

- **字符串**: 在 Julia 中, 使用双引号 "text" 表示字符串, 单引号 'c' 表示字符
在 Python 中, 字符串可以使用单引号或者双引号来表示

Julia

```
# 字符串
S = "text"

# 字符
C = 'c'

# 报错, '' 只能包含单个字符
C = 'text'
#=
ERROR: syntax: character literal contains
multiple characters
=#
```

Python

```
# 字符串
S = "text"

# 字符串
C = 'text'
```

2 语法差异 – 字符串

- **字符串拼接**: 在 Julia 中, 字符串拼接使用 `*`, 字符串重复n次使用 `^`
在 Python 中, 字符串拼接使用 `+`, 字符串重复n次使用 `*`

Julia

```
# 字符串拼接
S = "text" * "123" # "text123"

# 字符串重复 3 次
S = "ab" ^ 3 # "ababab"
```

Python

```
# 字符串拼接
S = "text" + "123" # "text123"

# 字符串重复 3 次
S = "ab" * 3 # "ababab"
```

2 语法差异 – 整数

- **整数**: 在 Julia 中, Int 类型是 32位/64 位有符号整数, 整数运算可能溢出, 需要根据场景改用大整数 BigInt 或者浮点数类型; 在 Python 中, int 类型是无限精度类型, 但 numpy 中的 int 类型默认为 64/32 位 (linux上默认为 64 位, windows上默认为 32 位)

Julia

```
# 默认为 32 位/64 位整数类型
typeof(2) # Int32/Int64

# 整数运算溢出
2^64 # 0

# 如果需要可使用 BigInt
big(2)^64 # 18446744073709551616
```

Python

```
import numpy as np
# 内置的整数类型不会溢出
2**64 # 18446744073709551616

# numpy 的整数类型为 32 位或者 64 位
# windows 上为 dtype('int32')
# linux 上位 dtype('int64')
x = np.array(2)
x.dtype

# 整数运算可能溢出
x**64 # 0
```

2 语法差异 – 列表和字典

- **存储不同类型的元素**: 在 Python 中, 列表和字典内可以存储不同类型的元素, 在 Julia 中, 存储不同类型元素的容器可以使用 `Vector{Any}` 或者 `Dict{String,Any}`

Julia

```
# 使用 Any 作为容器的元素类型会降低 Julia 代码的运行速度
# 在使用前请确认容器是否真的需要存储不同类型的元素

# 创建可存储任意元素的列表
list = Any[1, "abc", 2.1]
# 往列表内添加元素
push!(list, "text")

# 创建可存储任意元素的字典
dict = Dict{String, Any}(
    "a" => 1, "b" => "abc")
```

Python

```
# 创建可存储任意元素的列表
list = [1, "abc", 2.1]

# 往列表内添加元素
list.append("text")

# 创建可存储任意元素的字典
dict = {"a": 1, "b": "abc"}
```

2 语法差异 – nothing 和 none

- 常见的空值表示：在 Julia 中，常使用 **nothing** 来表示空值
在 Python 中，常使用 **None** 来表示空值

Julia

```
x = nothing

# 使用 !isnothing(x) 来判断值是不是 nothing
if !isnothing(x)
    println("abc")
end
```

Python

```
x = None

# 使用 x is not None 来判断值是不是 None
if x is not None:
    print("abc")
```

2 语法差异 – 数组

- **数组字面量**: 在 Julia 中, 可以使用 `[1,2,3]` 来创建向量或矩阵;
在 Python 中, 可以使用 `np.asarray([1,2,3])` 来从列表中创建数组

Julia

```
# 创建向量 [1,2,3]
x = [1, 2, 3]

# 创建 2x3 的矩阵
x = [1 2 3;
      4 5 6]
```

Python

```
import numpy as np

# 创建向量 [1,2,3]
np.asarray([1,2,3])

# 创建 2x3 的矩阵
np.asarray([[1,2,3],[4,5,6]])
```


2 语法差异 – 数组

- **数组索引**: 在 Julia 中, 数组索引从 1 开始; 在 Python 中, 数组索引从 0 开始
- **数组切片**: 在 Julia 中, 数组切片包含最后一个索引; 在 Python 中, 数组切片不包含最后一个索引
- **负数索引**: 在 Julia 中, 不支持负数索引, 可以使用 end 来索引数组的最后一位

Julia

```
x = [1, 2, 3, 4, 5]

# 取第 1 个元素
x[1] # 1

# 取最后一个 (第 5 个) 元素
x[end] # 5

# 取第 2 个到第 4 个元素
x[2:4] # [2, 3, 4]

# 取第 2 个到最后一个元素
x[2:end] # [2, 3, 4, 5]
```

Python

```
import numpy as np
x = np.asarray([1,2,3,4,5])

# 取第 1 个元素
x[0] # 1

# 取最后一个 (第 5 个) 元素
x[-1] # 5

# 取第 2 个到第 4 个元素
x[1:4] # [2, 3, 4]

# 取第 2 个到最后一个元素
x[1:] # [2, 3, 4, 5]
```

2 语法差异 – 数组

- **切片步长**: Julia 可以使用 `x[start:step:stop]` 来表示间隔 `step` 步长的切片, Python 使用 `x[start:(stop+1):step]` 来表示带步长的切片

Julia

```
x = [1, 2, 3, 4, 5, 6]

# 取偶数位的元素
x[2:2:end] # [2, 4, 6]

# 数组翻转, 也可使用 reverse(x)
x[end:-1:1] # [6, 5, 4, 3, 2, 1]
```

Python

```
import numpy as np
x = np.asarray([1,2,3,4,5,6])

# 取偶数位的元素
x[1::2] # [2, 4, 6]

# 数组翻转, 也可使用 np.flip(x)
x[::-1] # [6, 5, 4, 3, 2, 1]
```

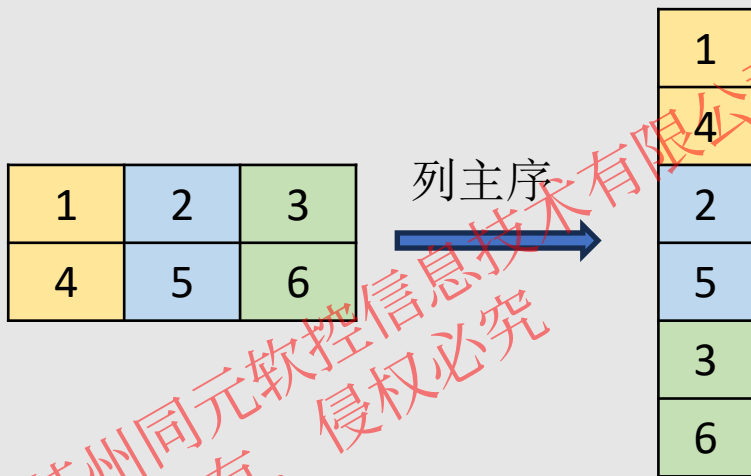
2 语法差异 – 数组

- **数组内存布局**: Julia 使用**列主序**来存储数据和遍历数组, 和Fortran、Matlab相同, Python 默认使用**行主序**来存储数据和遍历数组, 也兼容列主序的内存布局

Julia

```
A = [1 2 3; 4 5 6]
```

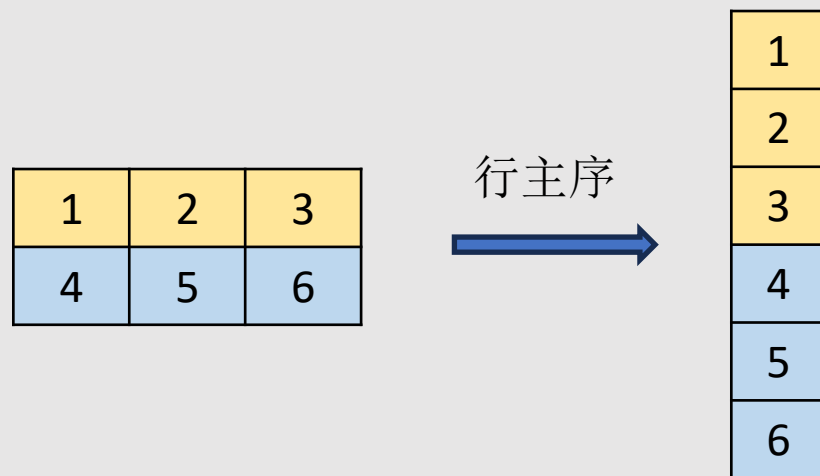
```
vec(A) # [1,4,2,5,3,6]
```



Python

```
import numpy as np  
A = np.asarray([[1,2,3],[4,5,6]])
```

```
A.flatten() # [1,2,3,4,5,6]
```



2 语法差异 – 数组

- **线性索引**: Julia 矩阵的索引 $A[i]$ 表示线性索引, Python 对矩阵取线性索引使用 $A.flat[i]$
- **按行切片**: Julia 使用 $A[i,:]$ 取矩阵的第 i 行, Python 可使用 $A[i]$ 或者 $A[i,:]$ 来取矩阵的第 $i-1$ 行

Julia

```
A = [1 2 3; 4 5 6]

# 线性索引取第 3 个元素
# 按列主序方向遍历
A[3] # 2

# 取矩阵的第 2 行
A[2,:] # [4, 5, 6]
```

Python

```
import numpy as np
A = np.asarray([[1,2,3],[4,5,6]])

# 线性索引取第 3 个元素
# 按行主序方向遍历
A.flat[2] # 3

# 取矩阵的第二行
A[1] # [4,5,6]
A[1,:] # [4,5,6]
```

2 语法差异 – 数组

- **view数组**: Julia 的数组切片默认返回一个新的数组, 可使用 @view 宏来返回数组的视图, 即切片和原数组共享一份数据; Python 的 numpy 数组在切片时默认返回原数组的视图

Julia

```
A = [1 2 3; 4 5 6]

# 创建新的数组, 修改 B 不会影响 A
B = A[1, :]
B[1] = 100

# 创建视图, 修改 C, A 也会被修改
C = @view A[1, :]
C[1] = 100

#=
A = [100 2 3;
     4 5 6]
=#
```

Python

```
import numpy as np
A = np.asarray([[1,2,3],[4,5,6]])

# 创建新的数组, 修改 B 不会影响 A
B = np.copy(A[0, :])
B[0] = 100

# 创建视图, 修改 C, A 也会被修改
C = A[0, :]
C[0] = 100

"""
A = [[100, 2, 3 ],
     [4,   5, 6]]
"""
```

2 语法差异 – 数组

- **广播运算**：Julia 的数组之间广播运算需要使用 `.+, .-, .*, ./` 等带点运算符；
Python 的数组之间广播运算无需加点；
一维向量和二维矩阵进行广播运算时，Julia 和 Python 的向量扩展方向不同。

Julia

```
A = [1 2 3; 4 5 6]

# 每个元素都增加 1
A .+ 1 # [2 3 4; 5 6 7]

# 第 1,2,3 列分别乘以 1,2,3
A .* [1 2 3] # [1 4 9; 4 10 18]

# 和 1 维向量广播时
# 1 维向量扩展为 nx1 的矩阵
# 第 1,2 行分别减去 1,2
A .- [1, 2] # [0 1 2; 2 3 4]
```

Python

```
import numpy as np
A = np.asarray([[1,2,3],[4,5,6]])

# 每个元素都增加 1
A + 1 # # [[2, 3, 4], [5, 6, 7]]

# 第 1,2,3 列分别乘以 1,2,3
A * np.asarray([[1, 2, 3]])
# [[1, 4, 9], [4, 10, 18]]

# 和 1 维向量广播时
# 1 维向量扩展为 1xn 的矩阵
# 第 1,2,3 列分别减去 1,2,3
A - np.asarray([1, 2, 3])
# [[0, 0, 0], [3, 3, 3]]
```

2 语法差异 – 数组

- **矩阵乘法**: Julia 使用 $A * B$ 表示矩阵乘法, Python 使用 $A @ B$ 表示矩阵乘法;

Julia

```
A = [1 2; 3 4]
B = [0 1; 1 0]

# 矩阵乘法
A * B
# [2 1;
#  4 3]
```

Python

```
import numpy as np
A = np.asarray([[1,2],[3,4]])
B = np.asarray([[0,1],[1,0]])

# 矩阵乘法
A @ B
#[[2, 1],
# [4, 3]]
```

2 语法差异 – 数组

- **矩阵转置**: Julia 使用 A' 表示矩阵的共轭转置, `transpose(A)` 表示矩阵的转置, Python 使用 `A.T` 表示矩阵的转置, `A.T.conjugate()` 表示矩阵的共轭转置

Julia

```
A = [1.0+1.0im 2.0+1.0im;  
      4.0+1.0im 5.0+1.0im]
```

转置

```
transpose(A)  
permutedims(A)
```

```
# [1.0+1.0im 4.0+1.0im;  
#   2.0+1.0im 5.0+1.0im]
```

共轭转置

A'

```
adjoint(A)  
# [1.0-1.0im 4.0-1.0im;  
#   2.0-1.0im 5.0-1.0im]
```

Python

```
import numpy as np  
A = np.asarray([[1.0+1.0j, 2.0+1.0j],  
                [4.0+1.0j, 5.0+1.0j]])
```

转置

```
A.T  
np.transpose(A)  
# [[1.+1.j, 4.+1.j],  
#   [2.+1.j, 5.+1.j]]
```

共轭转置

```
A.T.conjugate()  
# [[1.-1.j, 4.-1.j],  
#   [2.-1.j, 5.-1.j]]
```


2 语法差异 – 数组

- **数组拼接**: Julia 可以使用 `[x y]`, `[x; y]` 或者 `vcats(x,y)`, `hcats(x,y)` 来拼接矩阵, 而在 Python 中需要使用 `np.vstack((x,y))` 或者 `np.hstack((x,y))`。

Julia

```
x = [1. 2. 3.;
      4. 5. 6.]

# 垂直拼接
[x; ones(1,3)]
vcats(x, ones(1,3))
# [1. 2. 3.;
#   4. 5. 6.;
#   1. 1. 1.]

# 水平拼接
[x ones(2,1)]
hcats(x, ones(2,1))
# [1. 2. 3. 1.;
#   4. 5. 6. 1.]
```

Python

```
import numpy as np
x = np.asarray([[1.,2.,3.],[4.,5.,6.]])

# 垂直拼接
np.vstack((x, np.ones((1,3))))
# [[1.,2.,3.],
#   [4.,5.,6.],
#   [1.,1.,1.]]

# 水平拼接
np.hstack((x, np.ones((2,1))))
# [[1.,2.,3.,1.],
#   [4.,5.,6.,1.]]
```

2 语法差异 – 条件判断语句

- **if 语句**: 在 Julia 中, if 关键词之后一定需要一个 Bool 类型的表达式
在 Python 中, if 关键词之后的表达式会隐式转换为 bool 类型

Julia

```
# 报错, if 关键词后不能使用非 Bool 类型变量
txt = "something"
if txt
    println("abc")
end

# ERROR: TypeError: non-boolean (String)
# used in boolean context

# 使用 !isempty(txt) 来表示字符串非空的条件
txt = "something"
if !isempty(txt)
    println("abc")
end
```

Python

```
# txt 字符串非空时, 条件成立
txt = "something"
if txt:
    print("abc")

# txt 字符串为空, 条件不成立
txt = ""
if txt:
    print("edf") # 不打印
```

2 语法差异 – 函数参数

- **不定长参数**: 在 Julia 中, 定义函数时, 在最后一个形参名后加上 ... 表示不定长参数
在 Python 中, 定义函数时, 在最后一个形参名前加上 * 表示不定长参数

Julia

```
# 最后一个形参名 args 后加 ... 表示不定长参数
function add(x, args...)
    res = 0
    for i in args
        res += i
    end
    return res + 2*x
end
println(add(1, 2, 3, 4, 5)) # 16
```

Python

```
# 最后一个形参名 args 前加 * 号表示不定长参数
def add(x, *args):
    res = 0
    for i in args:
        res += i
    return res + 2*x
print(add(1, 2, 3, 4, 5)) # 16
```

2 语法差异 – 函数参数

- **关键词参数**：在 Julia 中，定义函数时，在 ; 之后的参数为关键词参数，kwargs... 表示不定长的关键词参数。
在 Python 中，定义函数时，无需特意指定关键词参数，不定长的关键词参数使用 **kwargs 表示。

Julia

```
# 关键词参数在 ; 之后
function mul(; x, y, kwargs...)
    z = 0
    if haskey(kwargs, :z)
        z = kwargs[:z]
    end
    return x + 2 * y + z
end

mul(y = 2, x = 1, z = 1)
# 2 + 1*2 + 1 = 6
```

Python

```
# 不定长的关键词参数使用 **kwargs 表示
def mul(x, y, **kwargs):
    z = 0
    if "z" in kwargs:
        z = kwargs["z"]

    return x + 2 * y + z

mul(y = 2, x = 1, z = 1)
# 1 + 2*2 + 1 = 6
```

2 语法差异 – 函数参数

- **函数的默认值参数**：在 Julia 中，默认值参数**每次**运行时都会执行；
在 Python 中，默认值参数仅在**第一次**运行时执行

Julia

```
function func(x=rand())  
    println(x)  
end
```

```
# 两次运行 x 的值不相同  
# 每次运行都会调用一次 rand()  
func()  
func()
```

Python

```
import numpy as np  
def func(x=np.random.rand()):  
    print(x)
```

```
# 两次运行时的 x 都相同  
# np.random.rand() 仅执行了一次  
func()  
func()
```

2 语法差异 – 匿名函数

- **匿名函数**: 在 Julia 中, 使用 $(x) \rightarrow f(x)$ 表示匿名函数
在 Python 中, 使用 `lambda x: f(x)` 表示匿名函数

Julia

```
# 匿名函数, () 内为形参  
myf = (x) -> x+1  
myf(1) # 2
```

Python

```
# 匿名函数, : 之前为形参  
myf = lambda x: x+1  
myf(1) # 2
```

2 语法差异 – 类型判断

- Julia 使用 `isa` 函数判断类型，Python 使用 `isinstance` 函数判断类型

Julia

```
function func(x)
    # 使用 isa 判断 x 是否是 Int 类型
    if x isa Int
        println("Int")
    else
        println("not Int")
    end
end

func(1) # "Int"
```

Python

```
def func(x):
    # 使用 isinstance 判断 x 是否是 int 类型
    if isinstance(x, int):
        print("int")
    else:
        print("not int")

func(1) # "int"
```

2 语法差异 – 结构体

- Julia 没有 class，但可以使用 struct / mutable struct 定义结构体，结构体定义后不能增减字段，类似 C 的 struct 定义。
- Python 使用 class 定义类，类定义后能增减属性。

Julia

```
# 定义可变结构体
mutable struct Foo
    baz::Int
    qux::Float64
end

# 创建实例
foo = Foo(23, 1.5)

# 不允许增加字段
foo.abc = 3
# ERROR: type Foo has no field abc
```

Python

```
# 定义 class
class Foo():
    baz: int
    qux: float
    def __init__(self, baz, qux):
        self.baz = baz
        self.qux = qux

# 创建实例
foo = Foo(23, 1.5)

# 新增字段
foo.abc = 3

foo.abc # 3
```


2 语法差异 – 方法分派

- Julia 使用多重分派机制，根据所有参数的运行时类型决定调用的方法实现
- Python 一般根据方法接收者的类型，或者 instance 函数来实现分派

Julia

```
struct Bar end

# 定义两个方法实现
func(bar::Bar, y::Int) = 1
func(bar::Bar, y::String) = 2

# 根据所有参数的类型来派发
func(Bar(), 1)      # 1
func(Bar(), "abc") # 2
```

Python

```
class Bar:
    def func(self, x):
        if isinstance(x, int):
            return 1
        elif isinstance(x, str):
            return 2

bar = Bar()
bar.func(1)      # 1
bar.func("abc") # 2
```

2 语法差异 – 三元运算符

- **三元运算符**: 在 Julia 中, 三元运算符使用 `cond ? then_val : else_val` 表示, 和 C 相同
在 Python 中, 三元运算符使用 `then_val if cond else else_val` 表示

Julia

```
# 三元运算符  
true ? 1 : 0 # 1
```

Python

```
# 三元运算符  
1 if True else 0 # 1
```

2 语法差异 – 运算符

- 在 Julia 中, ! 表示取非, ^ 表示幂运算, ÷ 表示整除, ∨ 表示异或运算, % 表示取余
- 在 Python 中, not 表示取非, ** 表示幂运算, // 表示整除, ^ 表示异或运算, % 表示取模

Julia

```
!false    # 取非, true
2 ^ 2     # 幂运算, 4
3 ÷ 2     # 整除 (使用 \div<tab> 输入), 1
1 ∨ 0     # 异或 (使用 \xor<tab> 输入), 1
7 % -2    # 取余, 1
```

Python

```
not False # 取非, true
2 ** 2    # 幂运算, 4
3 // 2    # 整除, 1
1 ^ 0     # 异或, 1
7 % -2    # 取模, -1
```

03 常用函数差异

@苏州同元软控信息技术有限公司版权所有，未经许可，不得复印、传播或以其他方式使用
版权所有，侵权必究

3 常用函数差异 – rand函数

- 在 Julia 中, 使用 Random 模块相关函数生成随机数
- 在 Python 中, 使用 numpy.random 模块生成随机数

Julia

```
import TyRandom, Random
# 设置全局随机数生成器的 seed
Random.seed!(1234)

# 使用全局随机数, 生成 2x3 的均匀分布随机数
randn((2,3))

# 新建 seed 为 42 的随机数生成器
rng = TyRandom.MT19937ar(42)

#使用随机数生成器rng, 生成 2x4 的正态分布随机数
randn(rng, (2,4))

#使用随机数生成器rng, 生成 2x3 的均匀分布随机数
rand(rng, (2, 3))
```

Python

```
import numpy as np
# 设置全局随机数生成器的 seed
np.random.seed(1234)

# 使用全局随机数, 生成 2x3 的均匀分布随机数
np.random.normal(size=(2,3))

# 新建 seed 为 42 的随机数生成器
rng = np.random.default_rng(seed=42)

# 使用随机数生成器rng, 生成 2x4 的正态分布随机数
rng.normal(size=(2, 4))

# 使用随机数生成器rng, 生成 2x3 的均匀分布随机数
rng.random(size=(2, 3))
```

3 常用函数差异 – 单位矩阵

- 在 Julia 中，可使用 LinearAlgebra 模块内的 `I` 变量表示单位矩阵
- 在 Python 中，使用 `numpy.eye` 函数创建单位矩阵

Julia

```
using LinearAlgebra
A = [1. 2.; 3. 4.]
# 可使用 I 表示单位矩阵
A + I
# [2.0  2.0
#   3.0  5.0]
```

Python

```
import numpy as np
A = np.asarray([[1.,2.],[3.,4.]])
A + np.eye(2)
# [[2., 2.],
#   [3., 5.]]
```

3 常用函数差异 – linspace

- 在 Julia 中, 可使用 `range(start, stop, length)` 函数实现 numpy 的 `linspace` 功能

Julia

```
collect(range(-0.1, 0.3, length=5))  
# [-0.1, 0. , 0.1, 0.2, 0.3]
```

Python

```
import numpy as np  
np.linspace(-0.1, 0.3, num=5)  
# [-0.1, 0. , 0.1, 0.2, 0.3]
```

建立知识规范，营造协同生态
积累工业模型，发展可控平台
融入中国创新，打造先进软件

感谢聆听